

Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

UNIVERSITI TUNKU ABDUL RAHMAN
UGCS1633 Fundamentals of Programming
PROJECT REPORT

Trimester : June 2023

Faculty : Faculty of Engineering and Green Technology (FEGT)

Programme : Bachelor of Engineering (Honours) Electronic Engineering

Practical Group : P1

Lecturer's Name : Dr. Toh Pek Lan

Submission Date : 19th September 2023

Group Members :

No.	Name	Student ID No.	Year / Trimester
1.	Poh Zheng Yang	[redacted]	Y1T3
2.	Anson Kok Ming Jin	[redacted]	Y1T3
3.	Joshua Ng Yu Meng	[redacted]	Y1T2
		Total Marks	

Project title:
Digital Electronic Toolkit

Introduction:

Digital electronics is a fundamental area of study for students pursuing careers in electrical engineering, computer science, and related fields. It deals with the design, analysis, and application of digital circuits and systems, which form the building blocks of modern electronic devices and computing systems. From simple logic gates to complex microprocessors, digital electronics encompasses a wide range of concepts and principles.

To have a better understanding of digital electronics, this program acts as an essential tool for students to learn basic knowledges for digital electronics. Such as input and output for components, conversion between digits, and parity of different devices.

Digital Electronic toolkit, a program designed to assist students in the field of digital electronics. Whether students are just starting their journey or seeking to deepen understanding, this program is here to support them.

This program aims to provide a simple assist to students that are new in the field of digital electronics. It offers multiple function like checking and conversion. Furthermore, students can test various digital devices, experiment with different inputs, and observe the corresponding outputs. Users will have access to a comprehensive library of logic gates, flip-flops, counters and other essential components.

Throughout research, there is some examples of digital electronic assistance program that have been developed by programmers around the world. First, a programmer called Kamayani have developed a programme that can convert between number system in different form, from binary to hexadecimal, from decimal to octal, from decimal to hexadecimal and from octal to binary. Second example is a programme that been developed by Dinesh Kumar that is able to do parity checking by verify it is odd or even parity. Our program includes both of this function with a better operation and have some additional functions such as electronic components outcome function, truth table generation and sign magnitude complement system to assists electronic engineering students to have better understanding of the courses.

One of digital electronic assistance program is decimal to binary converter. The program allows the user to key in a decimal value and convert it to binary value. However, this program cannot convert the negative decimal value to its binary form. The program can

only convert a positive decimal number to its binary form in sign and magnitude system (BinaryHexConverter, n.d.). Therefore, a program that can be introduced is binary related issue page. This program has an 8-bit signed binary numbers' systems converter that can convert a decimal number between -127 to 127 to binary form in three binary system, which are sign and magnitude system, 1's complement system and 2's complement system. Thus, this converter has more function and more flexible compare to the program above. Therefore, the user can convert the decimal number to three binary system through this program without doing calculation themselves. This will reduce time usage and increase the efficiency when dealing with this conversion. The user also can use this program to verify their answer when they doing conversion themselves by using calculation. In addition, this program has an additional function which is parity system test simulator. This simulator helps the user more understand the parity system and know that what kind of binary code is satisfied with the corresponding parity system.

Another mentionable of digital electronic assistance program is Number system converter. Number system converter is a versatile program designed to simplify number system conversions. It enables users to seamlessly convert between four fundamental number systems: Binary (Base 2), Octal (Base 8), Decimal (Base 10), and Hexadecimal (Base 16) (BYJUS, 2023). Whether you're a student tackling mathematical problems or a programmer working on digital applications, Number system converter streamlines the process, making numerical conversions quick and effortless. Additionally, the programs also including function for truth table generation. Truth table generator is a user-friendly program designed to generate truth tables for seven essential logic gates: AND, OR, NOT, NAND, NOR, XOR, and XNOR. Users can define the number of input variables, and the program automatically generates the corresponding truth table, simplifying digital logic analysis. Whether you're a student studying logic circuits or an engineer designing complex systems, Truth table generator makes it easy to understand and work with logic gates.

Problem statements:

1. In today's fast-paced and interconnected digital world, the ability to convert numbers between different numerical systems has become increasingly essential. From students learning the foundations of computer science to engineers designing complex systems and programmers developing software applications, there is a ubiquitous need for a reliable and efficient program that can seamlessly handle numerical system conversions. Consider the scenario of a student studying computer science. They may be learning about binary numbers, the fundamental building blocks of digital systems. To comprehend the significance of binary values, the student would benefit greatly from a program that can convert binary numbers to their decimal equivalents. This would allow them to understand how binary data relates to everyday decimal values and strengthen their understanding of digital systems. Moreover, professionals working in fields such as telecommunications, cryptography, data analysis, and network administration also require a tool that can convert numbers seamlessly across various numerical systems. The program should cater to their needs by providing a user-friendly interface, accurate results, customizable precision settings, and efficient performance. By developing a program that addresses these requirements, we can meet the growing demand for a reliable numerical system conversion tool in today's digital landscape. Whether it's students, engineers, programmers, or professionals from various domains, such a program will serve as an indispensable resource, empowering users to effortlessly convert numbers between binary, hexadecimal, octal, and decimal systems with convenience, accuracy, and efficiency.

   In Kamayani’s programme, there is an obvious weakness that can been faced by person who use this program, which is less versatile between number conversion. The program only can convert between 4 different way of combination numbers system, from binary to hexadecimal, from decimal to octal, from decimal to hexadecimal and from octal to binary only. Let’s imagine an engineer’s project are to developing a crypto device that must use a lot of calculation and conversion between binary number system, but this programme only able to convert binary numbers to hexadecimal.

2. ASCII code is a type of alphanumeric code which can represent each character with a 7-bit code. The characters can be symbols, numbers or letters. An 8th bit is added to the ASCII character to show its parity. This parity bit is used for bit error detection. An odd

parity bit makes the total number of 1s in an 8-bit code odd, and an even parity bit makes the total number of 1s even. If a system operates on odd parity, a check is made on each group of bits received to ensure the total number of 1s in that particular group is odd. If there is an even number of 1s, an error has occurred. The problem with this parity system is many students are not familiar with this system and its operation. Students may not understand when an error occurred and the reason for its occurrence. Some students also do not know how to differentiate the even parity system and odd parity system. Consequently, students will have doubts about it, and they will not know how to use electronic devices in a correct parity. This will lead to error generation when using electronic devices. It is undeniable that this problem must be addressed so that a solution can be formulated in order to help the students understand the parity system more. The problem can be solved by creating a parity system test simulator. This simulator can enable the user to enter the parity system used by the electronic device and an 8-bit binary code. The program will check the number of 1s in the 8-bit binary code. If the parity system does match with the number of 1s, there will be no error. If the parity system does not match with the number of 1s, there will be an error.

Dinesh Kumar's program weakness is the users only able to determine whether it is odd or even parity but couldn't detect whether there is an error in the parity systems. Without the ability to check for and identify specific errors, the program would not be able to assist students to correct them. Error correction is essential to ensure data integrity and reliability in communication systems.

3. Electronic components refer to a type of physical entity within a system of electronics with the sole purpose of affecting electrons and/or the fields surrounding them. These components come in various forms and are usually industrial products. AND Gate refers to a digital logic gate that implements conjunction from mathematical logic whereas OR Gate refers to a logic gate that implements logical disjunction. NAND Gate on the other hand stands for NOT AND which implements the logical inverse of AND Gate when NOR Gate does the same for OR Gate. Full adder refers to a combinational circuit that allows the addition of 3 inputs to produce 2 outputs when Full subtracter merely subtracts 3 outputs to form 2 inputs. Half adder is similar to full adder, just that the inputs have been decreased to 2 and the outputs have been decreased to 1. SR Flip-Flop refers to a 1-bit storage system when JK Flip flop is a gated version of SR Flip

flop. D Flip-Flop refers to a circuit that can delay the output signal. T flip-flop refers to a logic circuit that will toggle or hold its input according to the input state.

The problem with the status quo is that there are more than 14 types of electronic components and each of them offers different functions and different in/output results it makes students' life extremely difficult in understanding the components and their respective in/output results. This will result in a hindrance in learning which is not optimum for the study of a student. Further, it is difficult to test the components one by one as such action requires a power supply which can only be obtained in a lab. All this extra process of understanding the concept will not only force the students to spend more unnecessary time on it, but it will also decrease the enthusiasm of students when they try to find the in/output result and fail to do so. Our solution to this problem with the status quo is this program that will allow the user to key in the input and the respective output will be delivered. By doing so we can save their time on trying to find the respective in/output. Hence provide the student with a better learning environment. In

Project objectives:

1. Enhance Understanding: The program aims to assist digital electronics students by providing resources and interactive tools that enhance their understanding of fundamental concepts. This includes clear explanations, visualizations, and simulations to help students grasp complex topics and improve their comprehension.

2. Digit Conversion: The program aims to assist digital electronics students in converting between different numeral systems, such as binary, decimal, octal, and hexadecimal. It provides tools and algorithms that enable students to convert numbers seamlessly, enhancing their proficiency in working with different numerical representations commonly used in digital electronics. By surpass the weaknesses of Kamayani's program, we provide a more versatile way to conversion between number systems. The programme allows users to choose whether which type of number systems they would like to make as input, and which type of number systems they would like to choose as output. This programme provides a total of 12 different combination of conversion between number systems.

3. ASCII Code Conversion: The program facilitates the conversion of characters to their corresponding ASCII codes and vice versa. Students can input characters or ASCII codes and obtain the respective output, helping them understand the encoding and decoding process used in digital communication systems and enhancing their ability to work with character-based data in electronic devices.

4. Parity Testing: The program includes functionality to test the parity of electronic devices, such as parity checking circuits used in communication systems. Students can input data or streams of bits and determine if the parity is even or odd, enabling them to verify the integrity of transmitted data and gain hands-on experience with error detection techniques commonly used in digital electronics. To surpass Dinesh Kumar's programme, our program able to detect whether there is an error in the parity systems. If the parity system does match with the number of 1s, there will be no error. If the parity system does not match with the number of 1s, there will be an error.

4. Output Generations: The program assists students in verifying the output responses of logic gates and digital electronic devices based on the provided input values. This helps students validate the correctness and functionality of the logic gates and devices they are studying.

5. Truth Table Analysis: The program facilitates the generation and analysis of truth tables for logic gates and digital electronic devices. Students can input the logic gate configuration or device specifications, and the program generates the corresponding truth table. This enables students to understand the relationship between inputs and outputs, identify patterns, and draw conclusions about the logical operations performed by the gates or devices.

Main Content & development

Scope

Scopes of modules are listed below:

Function Linker (Main Menu) module

In this module, users are allowed to choose which program to start with, including digital conversion function, electronic components outcome function and parity checking. Users are required to enter the number of functions to proceed to the function pages. Users also able to quit the whole program by insert the correct numbers.

Digital Conversion module

In this module, users are able to convert between 4 different number system, including binary (base 2), octal (base 8), decimal (base 10) and hexadecimal (base 16). Users are required to enter the desired input/number to be converted and choose which number system wanted to convert to. The output / converted number will be calculated and shown by the program.

Electronic Components outcome module

In this module, users are able to verify the outcomes of different electronic components such as AND gate, OR gate and NOR gate etc. Users are required to choose which electronic components are required to examine the outcome and insert whether 1 (On) or 0 (Off) as the input and the output will be calculated and shown by the program.

Parity Checking module

In this module, users are able to check the parity of electronic system. Uses are required to select the parity system whether it is odd or even and the 8-bit code for checking. The parity and error will be calculated and determined by the program.

Truth Table generation module

In this module, users are able to generates different truth table with different numbers of input and output. Users are required to choose the electronic components and the numbers of input. The truth table elements will calculate by the program and generates a truth table.

Sign and magnitude module

This module allows users to convert binary numbers from magnitude sign system to its first complements and second complements.

Task allocation:

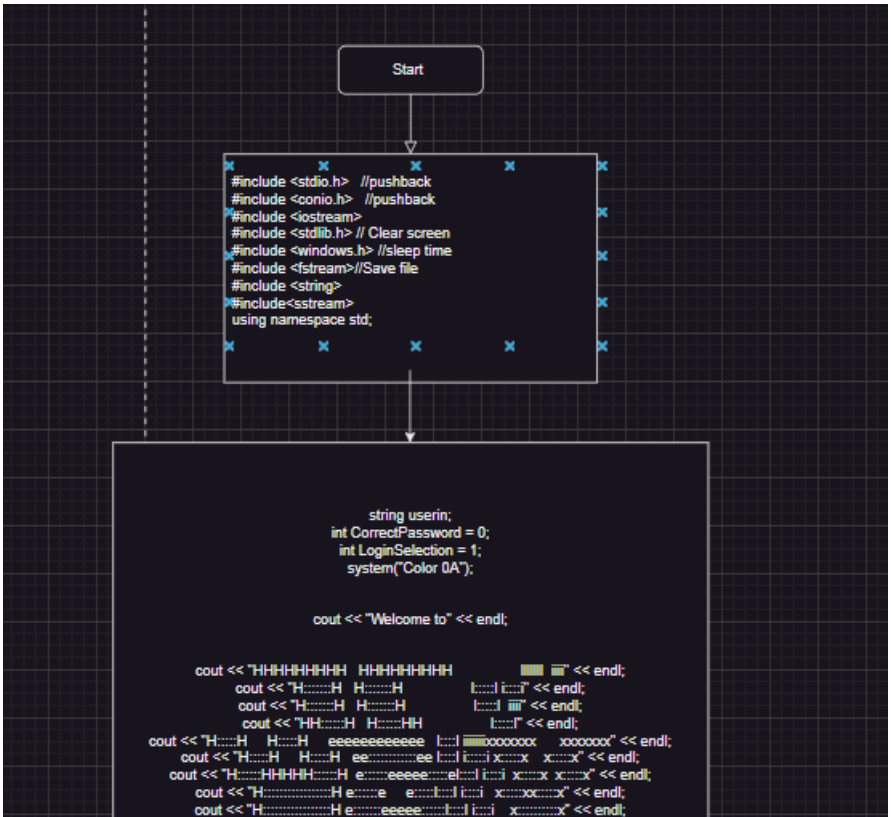
Table 1.0 Task Allocation

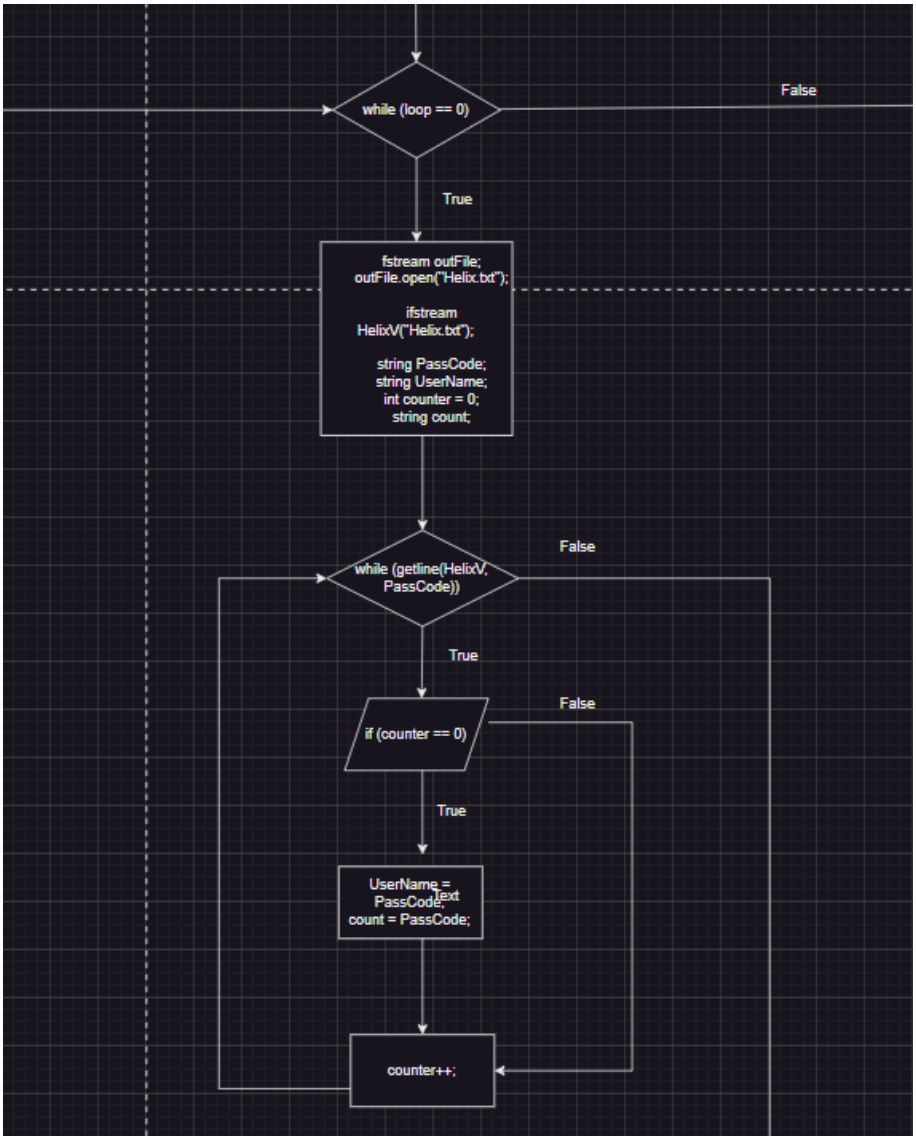
Name	Task allocation and scope
Anson Kok Ming Jin	• Parity system test simulator • 8-bit signed binary numbers' systems converter
Joshua Ng Yu Meng	• Framework • Logic Gate output calculator
Poh Zheng Yang	• Truth table generator • Number system converter

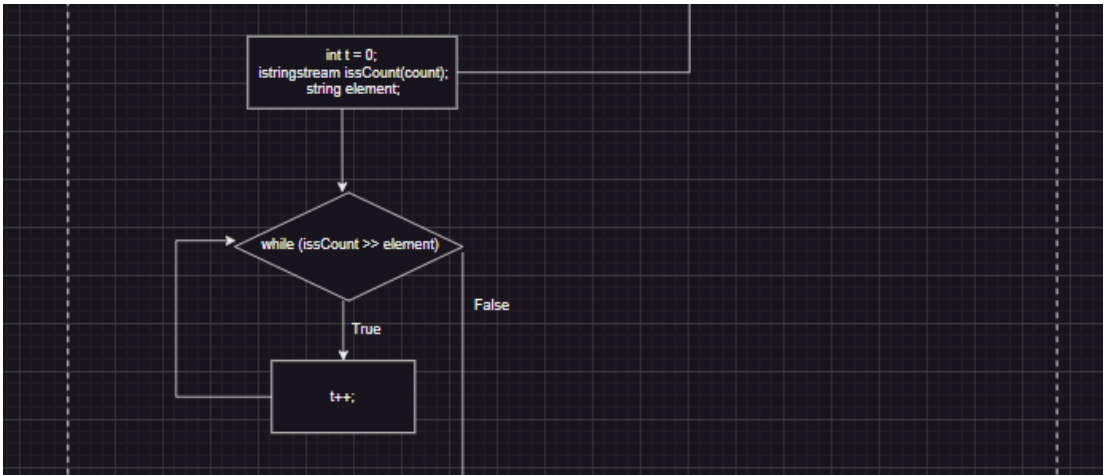
Framework:

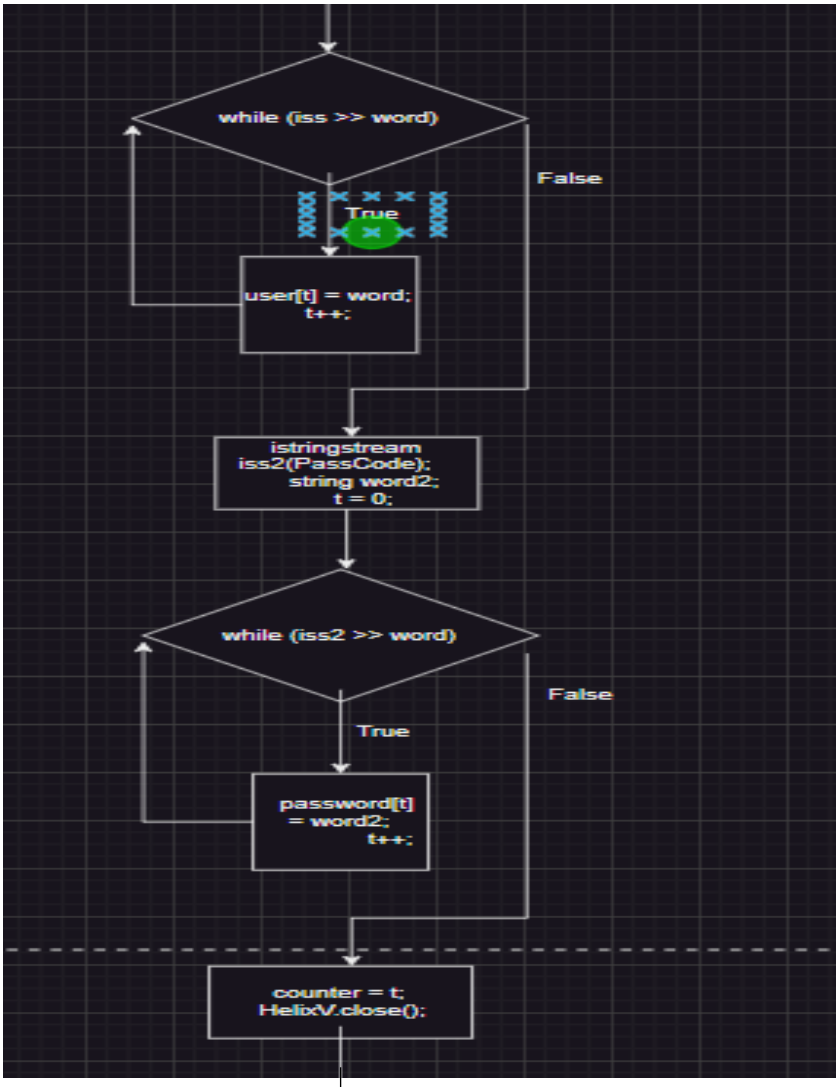
Flowchart

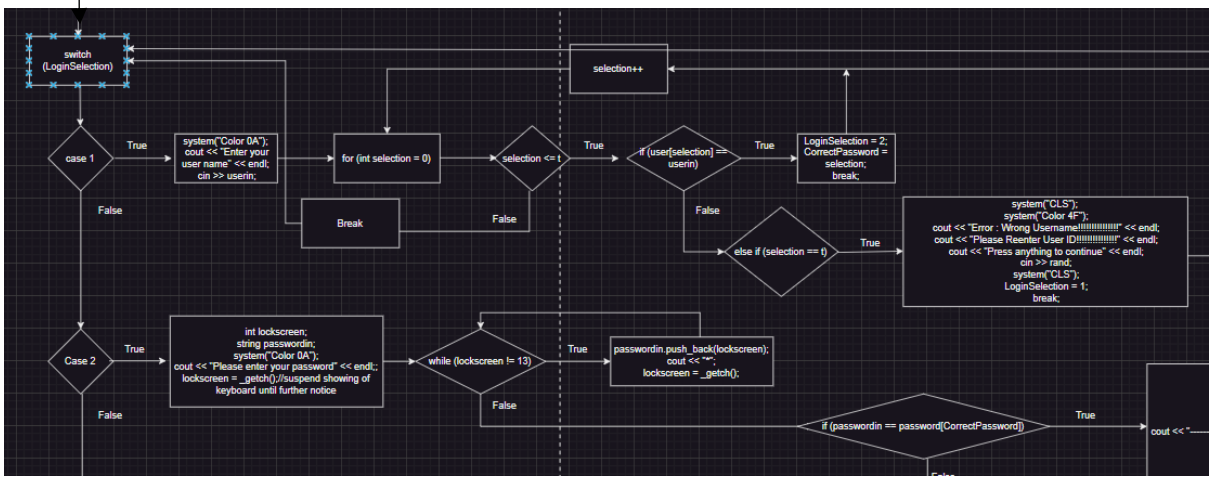
Main Framework



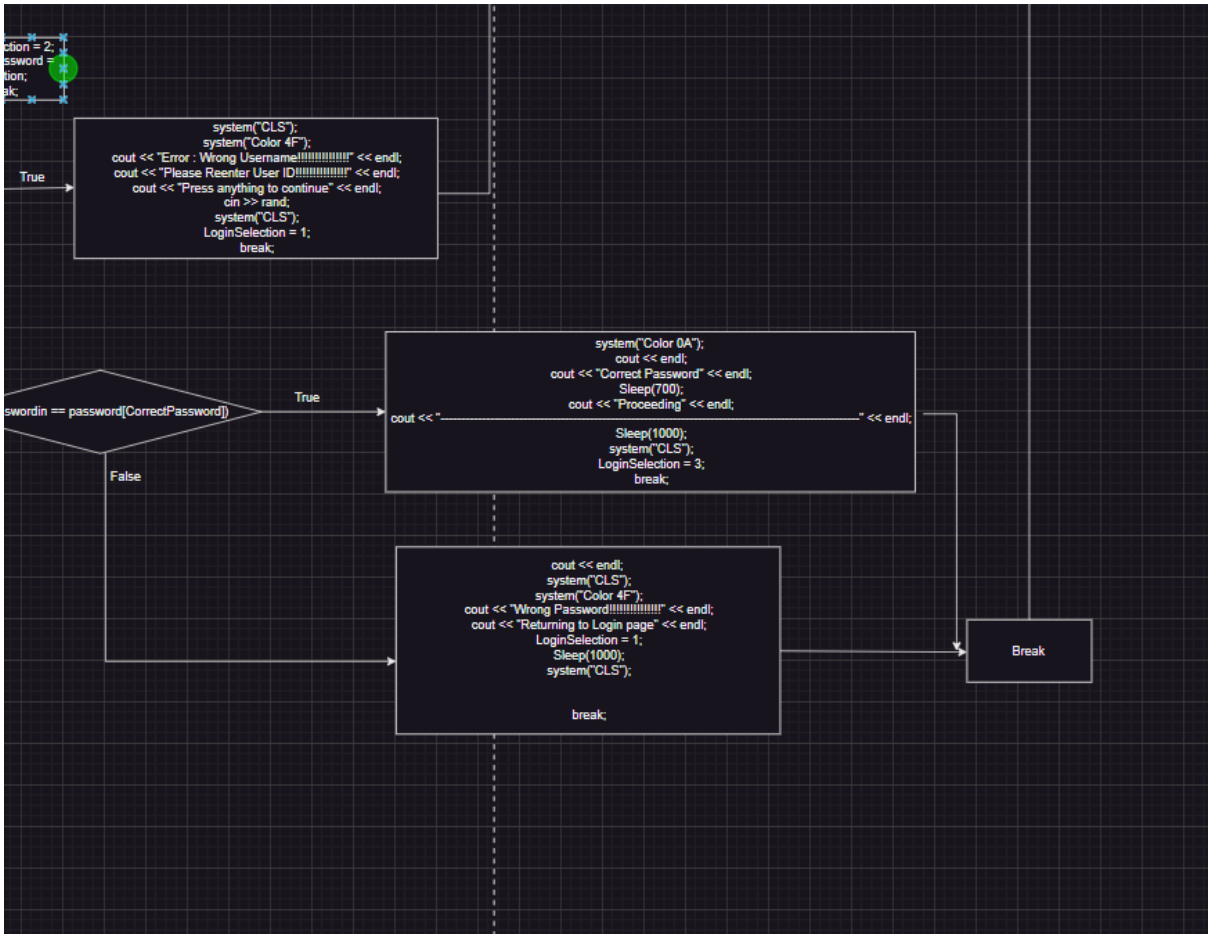

while (loop == 0)
False
True
fstream outFile;
outFile.open("Helix.txt");
ifstream
HelixV("Helix.txt");
string PassCode;
string UserName;
int counter = 0;
string count;
while (getline(HelixV,
PassCode))
False
True
if (counter == 0)
False
True
UserName =
PassCode;
count = PassCode;
counter++;


int t = 0;
istringstream issCount(count);
string element;
while (issCount >> element)
False
True
t++;

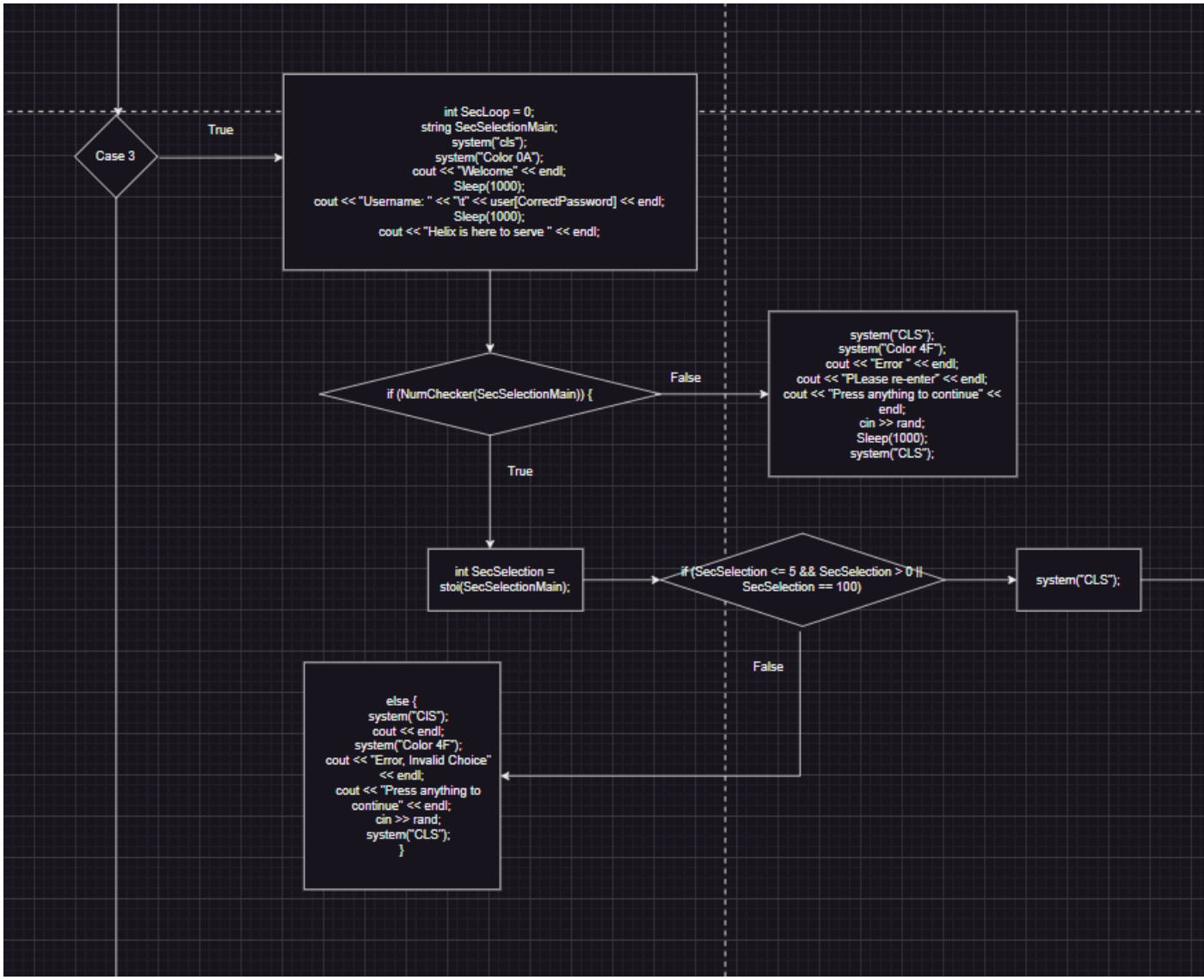

while (iss >> word)
False
True
user[t] = word;
t++;
istringstream
iss2(PassCode);
string word2;
t = 0;
while (iss2 >> word)
False
True
password[t]
= word2;
t++;
counter = t;
HelixV.close();


switch
(LoginSelection)
case 1
True
False
system("Color 0A");
cout << "Enter your
user name" << endl;
cin >> userin;
for (int selection = 0)
selection <= t
True
False
Break
if (user[selection] ==
userin)
True
False
LoginSelection = 2;
CorrectPassword =
selection;
break;
selection++
else if (selection == t)
True
system("CLS");
system("Color 4F");
cout << "Error : Wrong Username!!!!!!!!!!!!!!" << endl;
cout << "Please Reenter User ID!!!!!!!!!!!!!!" << endl;
cout << "Press anything to continue" << endl;
cin >> rand;
system("CLS");
LoginSelection = 1;
break;
Case 2
True
False
int lockscreen;
string passwordin;
system("Color 0A");
cout << "Please enter your password" << endl;
lockscreen = _getch();//suspend showing of
keyboard until further notice
while (lockscreen != 13)
True
False
passwordin.push_back(lockscreen);
cout << "*";
lockscreen = _getch();
if (passwordin == password[CorrectPassword])
True
cout << "

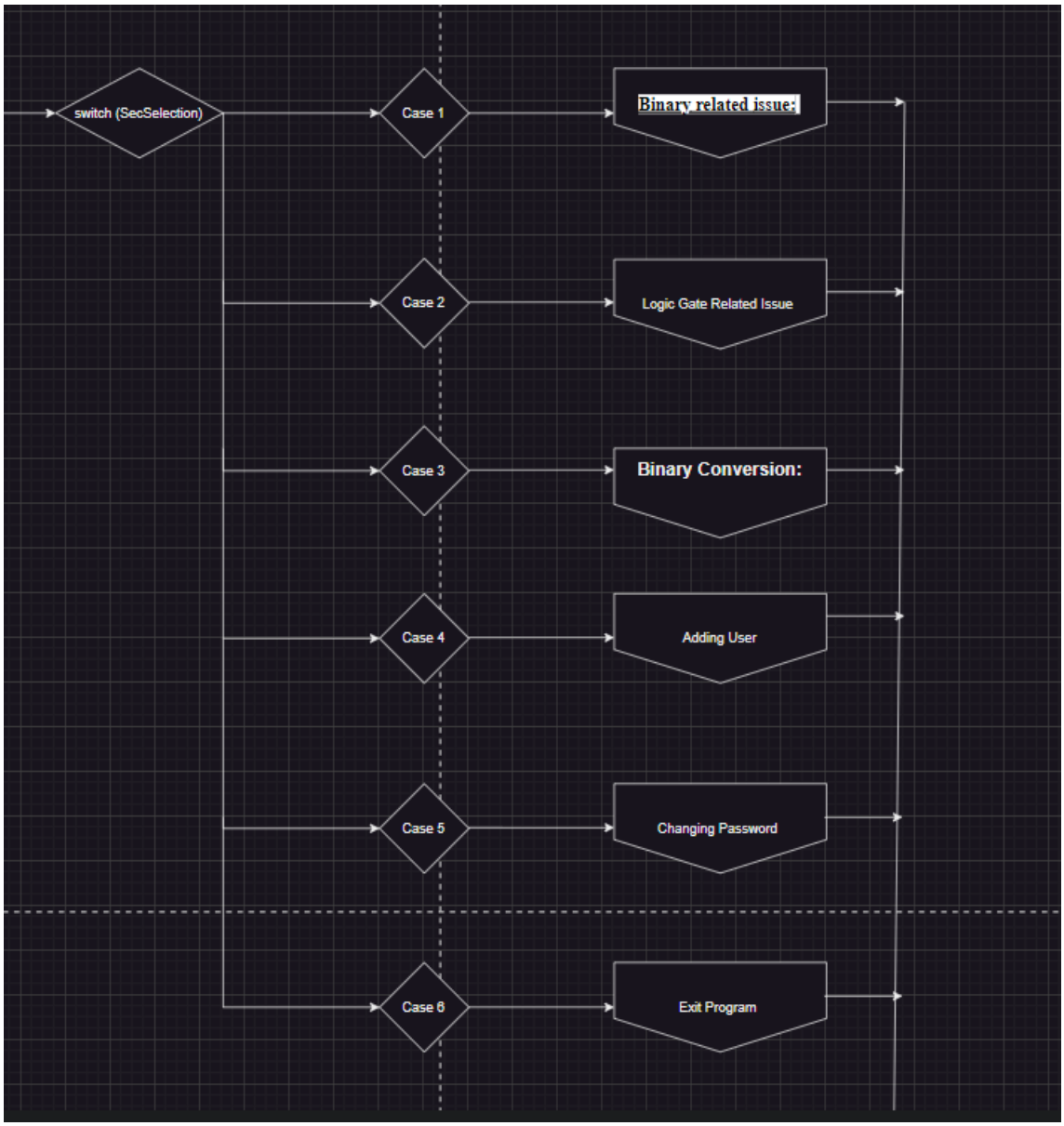
Main Framework Case 2- Continue



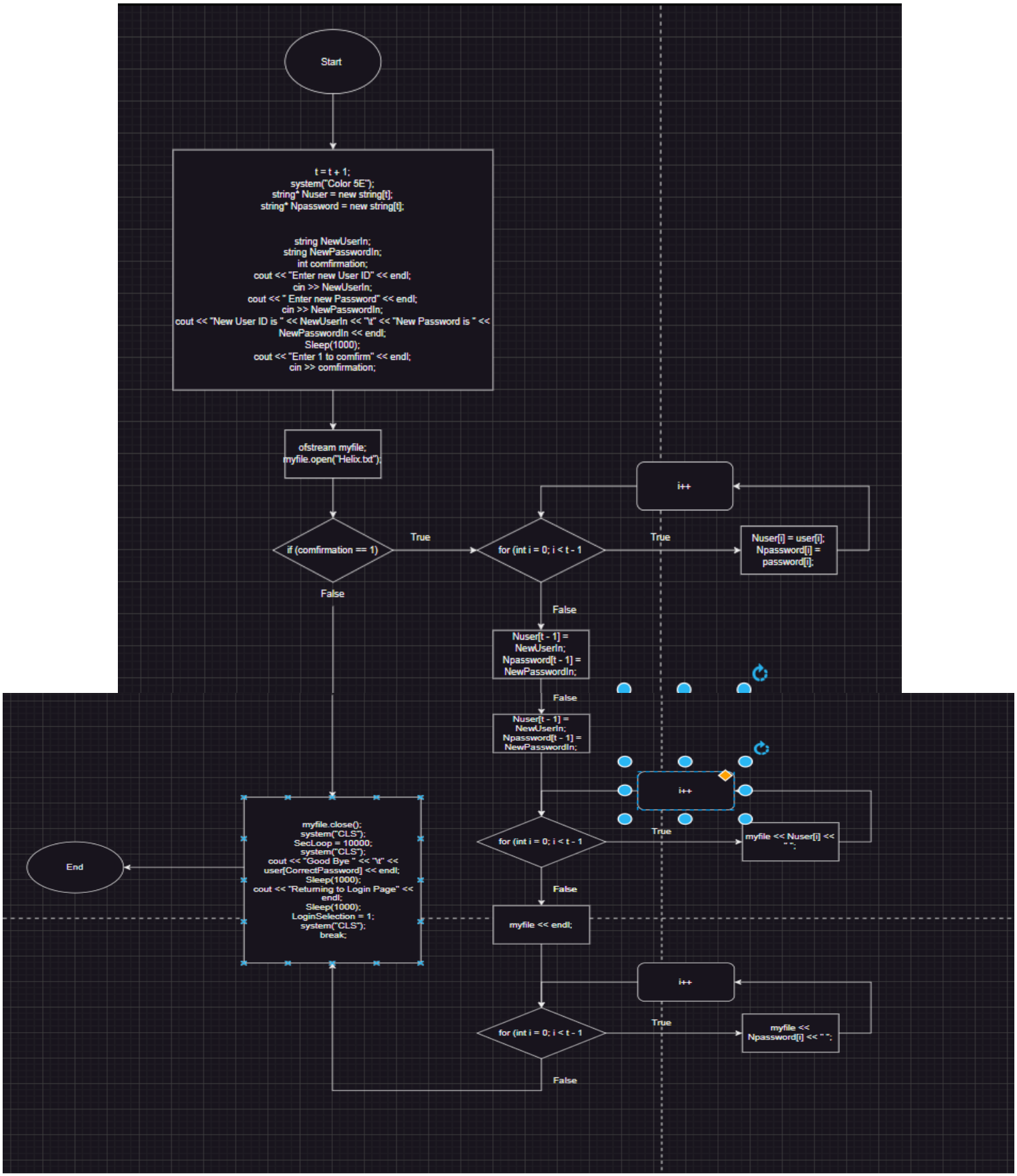
Main Framework Case 3



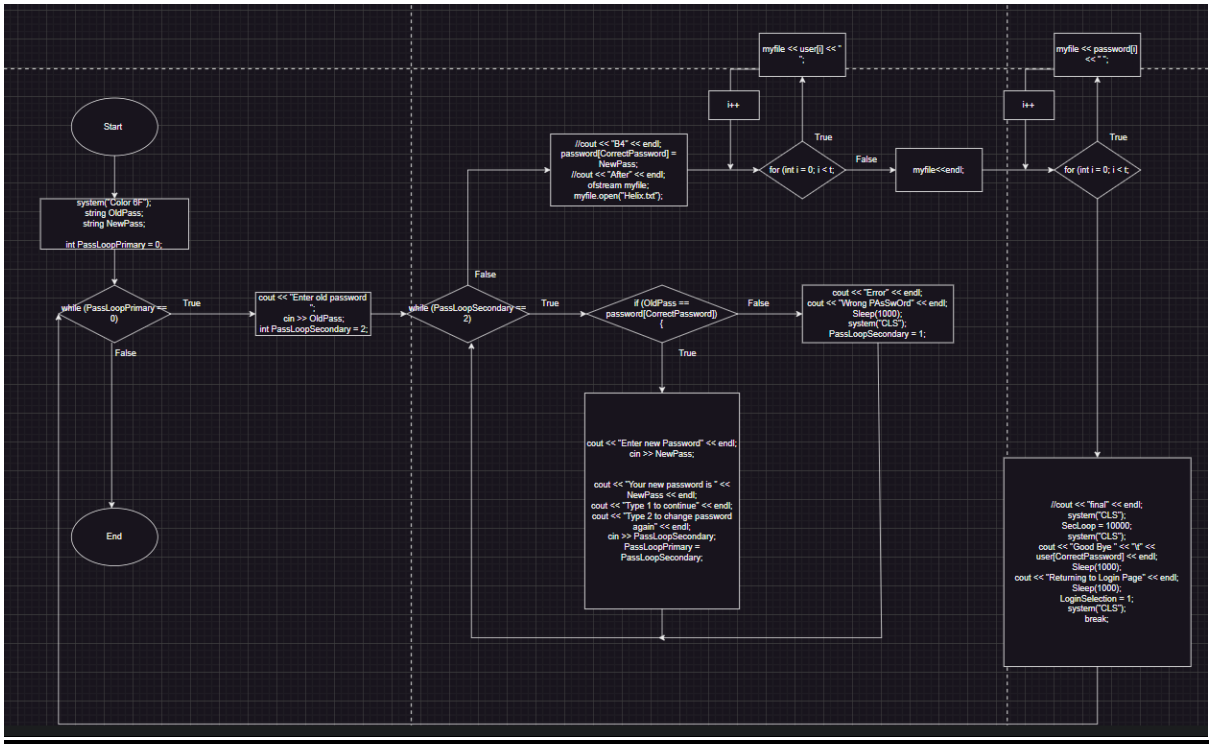
Main Framework Case 3 Continue



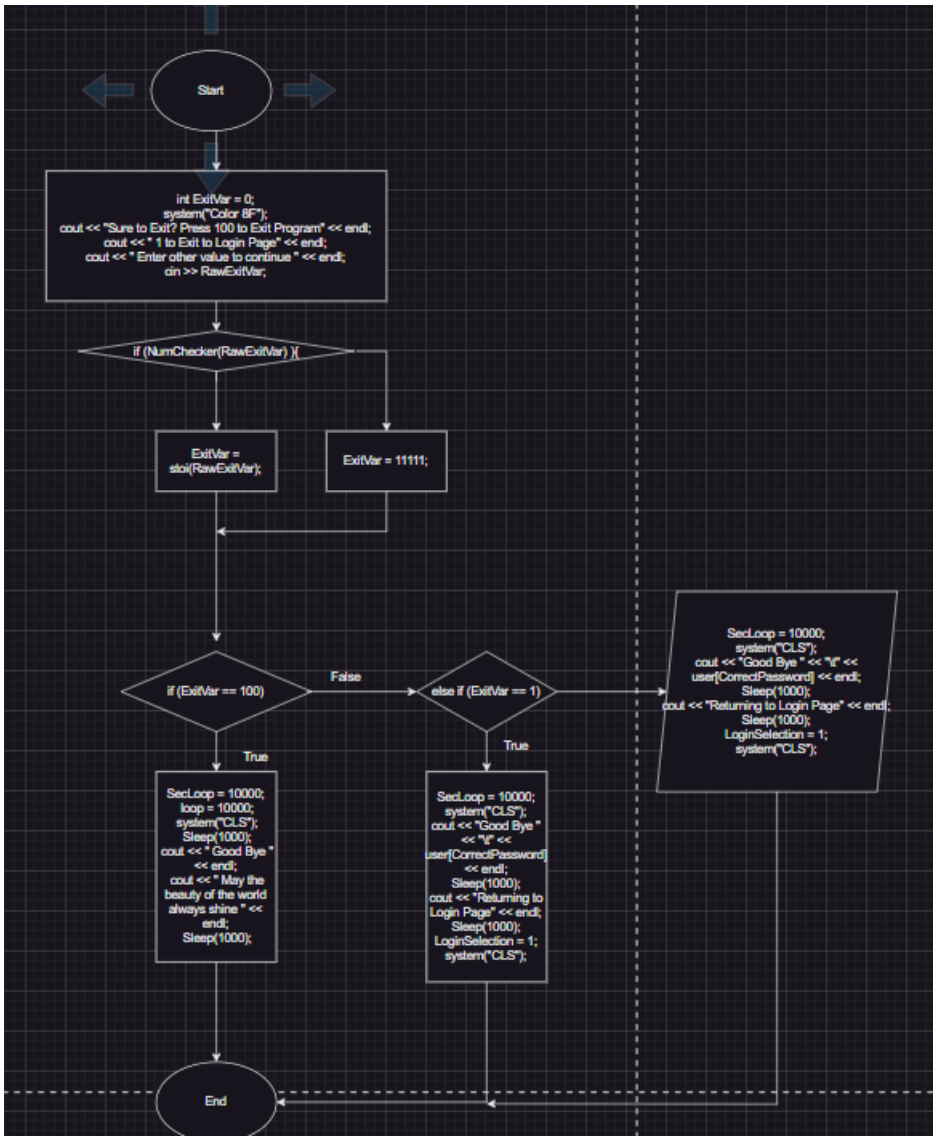
Function: Add User/Password



Function: Change Password



Function: Back to Menu



Code Screenshot

Figure 1.1 Welcome Prompt: Display Welcome Page

```
//////////////Initialization/////////////////////////////////////////////////////////////////////
string userin;
int CorrectPassword = 0;
int LoginSelection = 1;
system("Color 0A");

cout << "Welcome to" << endl;

cout << "HHHHHHHHH    HHHHHHHHH                lllllll  iiii" << endl;
cout << "H:::::::H    H:::::::H                l:::::l i::::i" << endl;
cout << "H:::::::H    H:::::::H                l:::::l  iiii" << endl;
cout << "HH::::::H    H::::::HH                l:::::l" << endl;
cout << "H:::::H      H:::::H     eeeeeeeeeeee   l::::l iiiiiiixxxxxxx      xxxxxxx" << endl;
cout << "H:::::H      H:::::H   ee::::::::::::ee l::::l i:::::i x:::::x    x:::::x" << endl;
cout << "H::::::HHHHH::::::H  e::::::eeeee:::::el::::l i::::i  x:::::x  x:::::x" << endl;
cout << "H:::::::::::::::::H e::::::e     e:::::l::::l i::::i   x:::::xx:::::x" << endl;
cout << "H:::::::::::::::::H e:::::::eeeee:::::l::::l i::::i    x::::::::::x" << endl;
cout << "H::::::HHHHH::::::H e:::::::::::::::::el::::l i::::i     x::::::::x" << endl;
cout << "H:::::H      H:::::H e::::::eeeeeeeeeee l::::l i::::i     x::::::::x" << endl;
cout << "H:::::H      H:::::H e:::::::e          l::::l i::::i    x::::::::::x" << endl;
cout << "HH::::::H    H::::::H e::::::::e        l::::::i::::::i  x::::xx::::x" << endl;
cout << "H:::::::H    H:::::::H e::::::::eeeeeeeel::::::i::::::i x::::x  x::::x" << endl;
cout << "H:::::::H    H:::::::H ee::::::::::::::el::::::i::::::ix::::x    x::::x" << endl;
cout << "HHHHHHHHH    HHHHHHHHH  eeeeeeeeeeeeeeellllllliiiiiiixxxxxx      xxxxxx" << endl;
int loop = 0;
```

Figure 1.2: Initialization of file

```
string rand;
//////////////////////////////////////Open File/////////////////////////////////////////////////////
fstream outFile;
outFile.open("Helix.txt");

ifstream HelixV("Helix.txt");

string PassCode;
string UserName;
int counter = 0;
string count;
while (getline(HelixV, PassCode))
{
    if (counter == 0) {
        UserName = PassCode;
        count = PassCode;
    }

    counter++;
```

Opens the file and store the data in the first line as a string named UserName and second line as PassCode, the data stored now is a very long string with only spaces in between, hence difficult to isolate the data out for further usage. Hence further processing of these two strings are necessary.

Figure 1.3 Extraction of data

```
////////////////////////////////////////Extraction of Data////////////////////////////////////////
int t = 0;
istringstream issCount(count);
string element;
/////Calculate Element number
while (issCount >> element)
{
    t++;
}
////////Non static array using pointer
string* user = new string[t];
string* password = new string[t];

istringstream iss(UserName);
string word;
t = 0;
while (iss >> word) {
    user[t] = word;
    //cout << user[t] << " ";
    t++;
}

istringstream iss2(PassCode);
string word2;
t = 0;
//cout << endl;
while (iss2 >> word2) {
    password[t] = word2;
                t++;
}

counter = t;
HelixV.close();
```

The first t++ allows the computer to tabulate the number of elements in the first line of the text file(which is the number of elements present in the username segment) this will allow the program to identify the number of usernames present, and as the number of username and the number of password will be the same, it also represents the number of passwords present in the text file. The number of element t is then used for the dynamic array, to declare the number of elements in the dynamic array. Lastly, the elements are being plugged into the dynamic array hence preparing themselves to be used later.

Figure 1.4: Login Selection Process

```
//////////////////////////Login Selection Process
switch (LoginSelection) {
    //User in
case 1: { ... }
     //Psssword In
case 2: { ... }
     //Function
case 3: { ... }
     break;
```

Pre-set value of LoginSelection is 1, hence it will automatically goes to 1 when the program first compiles, after that, the program will follows the latest declaration of LoginSelection. This will allow the program to direct itself to different parts of the selection for example password checking system or the main menu of the program , depending on the requirement of the user.

Figure 1.5: Input User Data

```
    //User in
case 1: {
    system("Color 0A");
    cout << "Enter your user name" << endl;
    cin >> userin;
```

Figure 1.6 Elimination of Error

```
    //Eliminate error
    for (int selection = 0; selection <= t; selection++)
    {
        if (user[selection] == userin) {
            LoginSelection = 2;
            CorrectPassword = selection;
            break;
        }

        else if (selection == t) {
            system("CLS");
            system("Color 4F");
            cout << "Error : Wrong Username!!!!!!!!!!!!!!!" << endl;
            cout << "Please Reenter User ID!!!!!!!!!!!!!!!" << endl;
            cout << "Press anything to continue" << endl;
            cin >> rand;
            system("CLS");
            LoginSelection = 1;
            break;
```

Figure 1.7: Accept Password as input

```
    }
        //Psssword In
    case 2: {
        int lockscreen;
        string passwordin;
        system("Color 0A");
        cout << "Please enter your password" << endl;;
        lockscreen = _getch();//suspend showing of keyboard until further notice

        while (lockscreen != 13)//convert typing into asterisk
        {
            passwordin.push_back(lockscreen);
            cout << "*";
            lockscreen = _getch();
        }
```

Lockscreen=getch_() is a command that instructs the computer to suspend the keyboard hence not showing any input value. The term pushback refers to another command that forcrs the display to be replaced with the word “*”

Figure 1.8: Password Checking

```
////Correct Password check
if (passwordin == password[CorrectPassword])
{
    system("Color 0A");
    cout << endl;
    cout << "Correct Password" << endl;
    Sleep(700);
    cout << "Proceeding" << endl;
    cout << "-----------------------------------------------------------------------------------------------------" << endl;
    Sleep(1000);
    system("CLS");
    LoginSelection = 3;
    break;
}
```

Figure 1.9: Error Prompt

```
    }
    ////////Wrong password warning
    else {
        cout << endl;
        system("CLS");
        system("Color 4F");
        cout << "Wrong Password!!!!!!!!!!!!!!!" << endl;
        cout << "Returning to Login page" << endl;
        LoginSelection = 1;
        Sleep(1000);
        system("CLS");

        break;
    }

    break;
}
```

Figure 1.10: Initialization of Main Menu

```
    //Function
case 3: {
    int SecLoop = 0;
    string SecSelectionMain;
    system("cls");
    system("Color 0A");
    cout << "Welcome" << endl;
    Sleep(1000);
    cout << "Username: " << "\t" << user[CorrectPassword] << endl;
    Sleep(1000);
    cout << "Helix is here to serve " << endl;
    while (SecLoop == 0) {
```

Figure 1.11 Main Menu Prompt

```
while (SecLoop == 0) {
    /////Main Menu
    system("Color 0A");
    cout << "1 for Binary related issue" << endl;
    cout << "2 for Logic Gate Related issue" << endl;
    cout << "3 for Binary Conversion" << endl;
    cout << "4 for new user and password" << endl;
    cout << "5 for changing password" << endl;
    cout << "   100 for exiting the program" << endl;
    cin >> SecSelectionMain;
```

Figure 1.12 Process Input

```
if (NumChecker(SecSelectionMain)) {
    int SecSelection = stoi(SecSelectionMain);
    if (SecSelection <= 5 && SecSelection > 0 || SecSelection == 100)
    {

        system("CLS");
        switch (SecSelection)

        {

        case 1: { ... }

        case 2: { ... }

        case 3: { ... }

        case 4: { ... }

        case 5: { ... }

        case 100: { ... }
        }
    }
```

Figure 1.13 Error Prompt

```
else {
    system("ClS");
    cout << endl;
    system("Color 4F");
    cout << "Error, Invalid Choice" << endl;
    cout << "Press anything to continue" << endl;
    cin >> rand;
    system("CLS");
}
```

Figure 1.14: Case 1: Binary Related Issue(Elaborated in Part 2 of the report)

```
case 1: { ... }
```

Figure 1.15: Case 2: Logic Gate Related Issue (Elaborated in Part 3 of the report)

```
case 2: { ... }
```

Figure 1.16 Case 3: Binary Conversion: (Elaborated in Part 4 of the report)

```
case 3: { ... }
```

Figure 1.17 Case 4: Adding New User/Password

```
case 4: { ... }
```

Figure 1.18: changing Password of current user

```
case 5: { ... }
```

Figure 1.19 Exit Selection

```
case 100: { ... }
```

Figure 1.20 Error Prompt

```
}
else {
    system("CLS");
    system("Color 4F");
    cout << "Error " << endl;
    cout << "PLease re-enter" << endl;
    cout << "Press anything to continue" << endl;
    cin >> rand;
    Sleep(1000);
    system("CLS");
}
```

Figure 1.21 Case 4: Initialization

```
t = t + 1;
system("Color 5E");
string* Nuser = new string[t];
string* Npassword = new string[t];

string NewUserIn;
string NewPasswordIn;
int comfirmation;
```

IN Figure 1.21 the program prepares itself to accept new inputs and store them into new array which can accept one more element in the array in comparison to the old array

Figure 1.22 Case 4: Accepting Input

```
cout << "Enter new User ID" << endl;
cin >> NewUserIn;
cout << " Enter new Password" << endl;
cin >> NewPasswordIn;
cout << "New User ID is " << NewUserIn << "\t" << "New Password is " << NewPasswordIn << endl;
Sleep(1000);
cout << "Enter 1 to comfirm" << endl;
cin >> comfirmation;
```

Figure 1.23 Case 4: Open File for writing

```
    ofstream myfile;
    myfile.open("Helix.txt");
```

Figure 1.24 Case 4: Writing New data

```
myfile.open( "Helix.txt" );
if (comfirmation == 1)

{
    for (int i = 0; i < t - 1; i++)
    {
        Nuser[i] = user[i];
        Npassword[i] = password[i];
    }

    Nuser[t - 1] = NewUserIn;
    Npassword[t - 1] = NewPasswordIn;

    for (int i = 0; i < t; i++)
    {

        myfile << Nuser[i] << " ";

    }

    myfile << endl;


    for (int i = 0; i < t; i++)
    {

        myfile << Npassword[i] << " ";

    }


}
myfile.close();
```

By creating a new set of arrays (Nuser and Npassword) the program stores the old arrays and include the latest set of user name and password into the new array. This allows the new array to have a +1 size of the original old array without having to encounter errors when trying to increase the size of a static array.

Figure 1.25: Closing text file and return to menu

```
myfile.close();
system("CLS");
SecLoop = 10000;
system("CLS");
cout << "Good Bye " << "\t" << user[CorrectPassword] << endl;
Sleep(1000);
cout << "Returning to Login Page" << endl;
Sleep(1000);
LoginSelection = 1;
system("CLS");
break;
```

It must return to main menu or else the program will crash due to an error in the dynamic array usage. Hence to prevent the bug and flaw, such action has been implemented to prevent crashing the program.

Figure 1.26 Case 5 Initialization

```
case 5: {
    system("Color 6F");
    string OldPass;
    string NewPass;


    int PassLoopPrimary = 0;
```

Figure 1.27 Case 5: Accepting old password

```


    int PassLoopPrimary = 0;

    while (PassLoopPrimary == 0) {
        cout << "Enter old password ";
        cin >> OldPass;
        int PassLoopSecondary = 2;
```

Figure 1.28 Case 5: Eliminating wrong old Password

```

    else {
        cout << "Error" << endl;
        cout << "Wrong PAsSwOrd" << endl;
        Sleep(1000);
        system("CLS");
        PassLoopSecondary = 1;
    }
```

Figure 1.29 Case 5: Replacing New Password

```
if (OldPass == password[CorrectPassword])
{
    cout << "Enter new Password" << endl;
    cin >> NewPass;

    cout << "Your new password is " << NewPass << endl;
    cout << "Type 1 to continue" << endl;
    cout << "Type 2 to change password again" << endl;
    cin >> PassLoopSecondary;
    PassLoopPrimary = PassLoopSecondary;


}
```

Figure 1.30:Case 5: Open Text File to write

```
//cout << "B4" << endl;
password[CorrectPassword] = NewPass;
//cout << "After" << endl;
ofstream myfile;
myfile.open("Helix.txt");
```

Figure 1.31: Case 5: Writing New data into text file

```
for (int i = 0; i < t; i++)
{

    myfile << user[i] << " ";

}

myfile << endl;


for (int i = 0; i < t; i++)
{

    myfile << password[i] << " ";

}
```

The data is written directly in to the text file, with the element at the respective location referring to the password in respect to the user being replaced as the new password.

Figure 1.32: Close text file an exit to Login Page

```
    //cout << "final" << endl;
    system("CLS");
    SecLoop = 10000;
    system("CLS");
    cout << "Good Bye " << "\t" << user[CorrectPassword] << endl;
    Sleep(1000);
    cout << "Returning to Login Page" << endl;
    Sleep(1000);
    LoginSelection = 1;
    system("CLS");
    break;
}
```

It must return to main menu or else the program will crash due to an error in the dynamic array usage. Hence to prevent the bug and flaw, such action has been implemented to prevent crashing the program.

Figure 1.33: Case 100 Initialization

```
string RawExitVar;
int ExitVar = 0;
```

Figure 1.34: Case 100: Accept User input

```
system("Color 8F");
cout << "Sure to Exit? Press 100 to Exit Program" << endl;
cout << " 1 to Exit to Login Page" << endl;
cout << " Enter other value to continue " << endl;
cin >> RawExitVar;
```

Figure 1.35: Case 100 Error Checking

```
if (NumChecker(RawExitVar) ){
    ExitVar = stoi(RawExitVar);
}

else {
    ExitVar = 11111;
}
```

Figure 1.36: Case 100 Exit Completely

```
}
if (ExitVar == 100)
{
    SecLoop = 10000;
    loop = 10000;
    system("CLS");
    Sleep(1000);
    cout << " Good Bye " << endl;
    cout << " May the beauty of the world always shine " << endl;
    Sleep(1000);
}
```

Figure 1.37: Case 100 Exit to Login Page

```
    }
    else if (ExitVar == 1)
    {
        SecLoop = 10000;
        system("CLS");
        cout << "Good Bye " << "\t" << user[CorrectPassword] << endl;
        Sleep(1000);
        cout << "Returning to Login Page" << endl;
        Sleep(1000);
        LoginSelection = 1;
        system("CLS");

    }
```

Figure 1.38: Exit to Menu

```
    }
    else {
        cout << " Program continues" << endl;
        Sleep(1000);
        system("CLS");
    }
    break;
}
```

Test plans and results

Figure 1.39: Prompt user to enter Username



Figure 1.40: Error Prompt if wrong username is being typed



Figure 1.41: Prompt the user to key in password

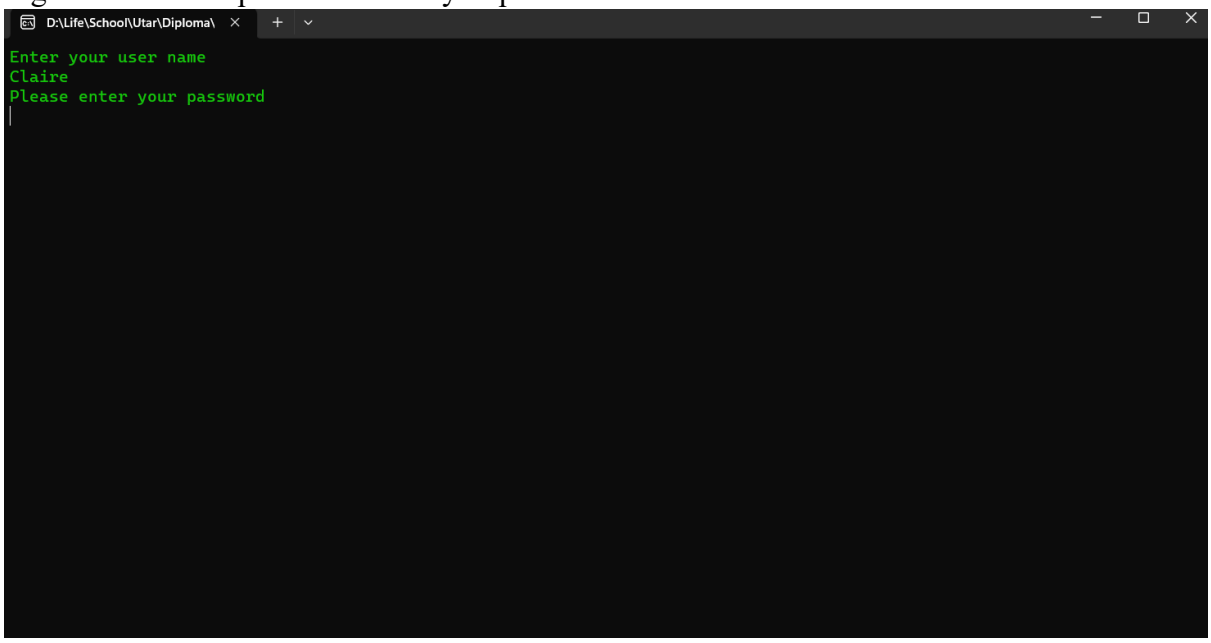


Figure 1.41: Error Prompt if wrong password is being keyed in



Figure 1.42: Asterisk is used to protect the privacy of the user



Figure 1.43: Menu Page to prompt the user regarding the available options

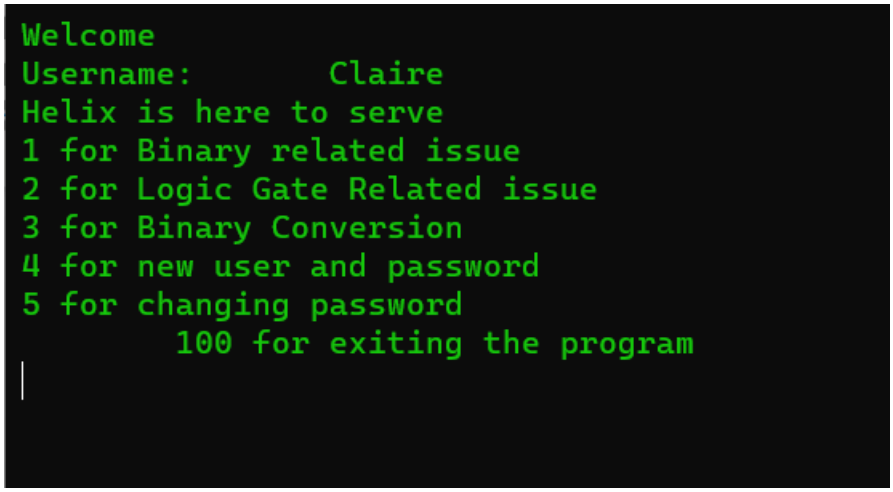


Figure 1.44: Binary related issue(will be further illustrated in part 2)

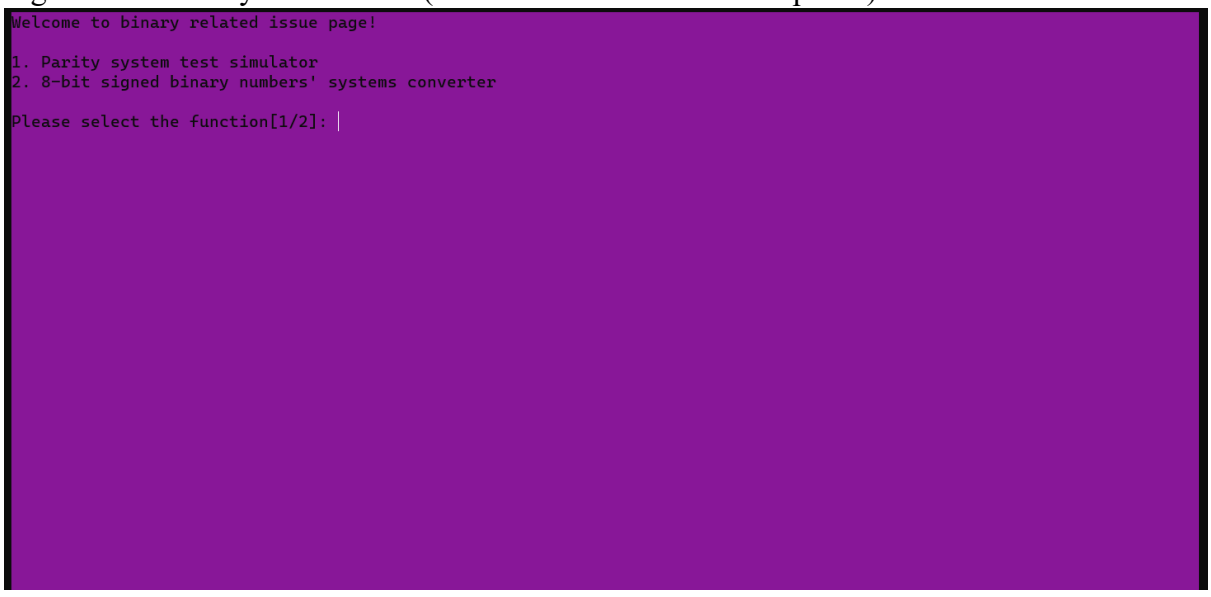


Figure 1.45 Logic gate related issue (will be illustrated in part 3)



Figure 1.46 Binary Conversion (will be further illustrated in part 4)

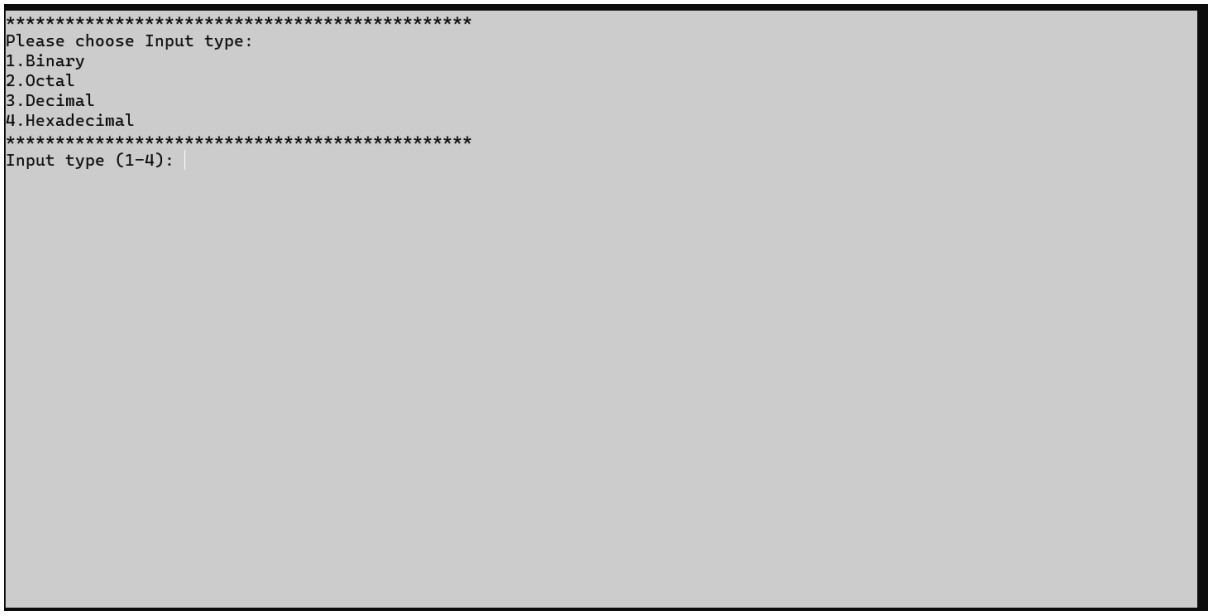


Figure 1.47: Prompt the user to enter new ID



Figure 1.48 Prompt the user to enter new Password



Figure 1.49: New User added

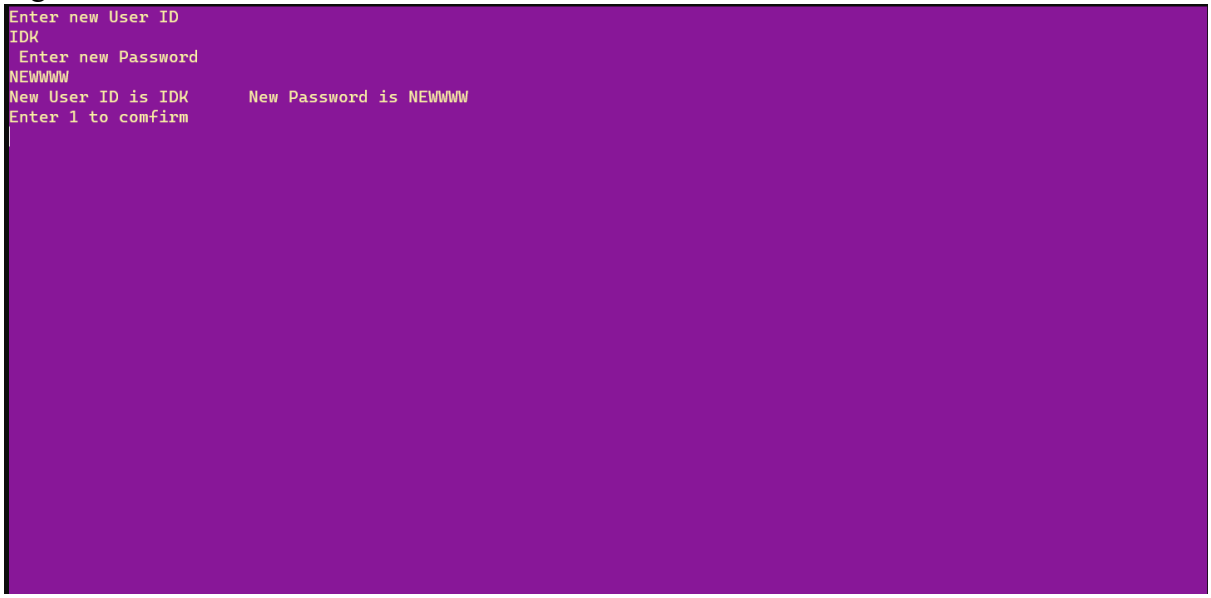


Figure 1.50: Changing Password



Figure 1.51 Prompt the user to enter old password



Figure 1.52 Wrong old password entered



Figure 1.53: Correct Password Entered



Figure 1.54: Prompt user to enter new password

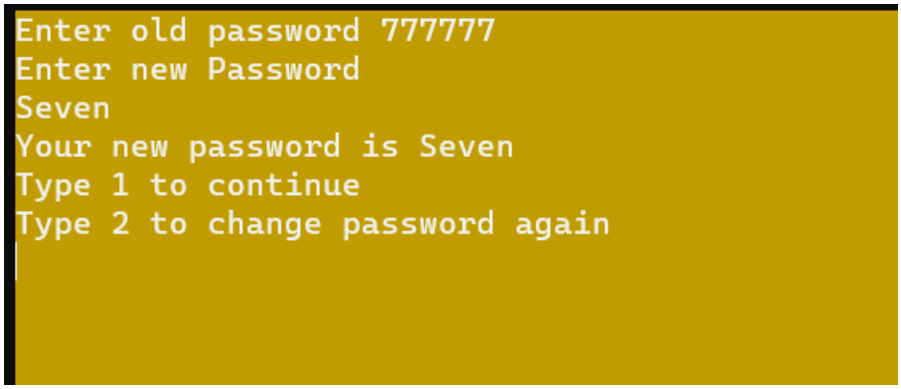


Figure 1.55 Prompt the user to continue or change the password again

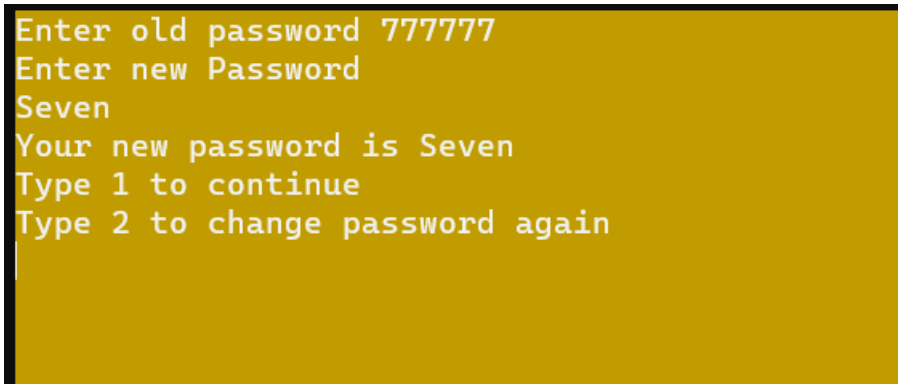


Figure 1.56: Exit to Login Page



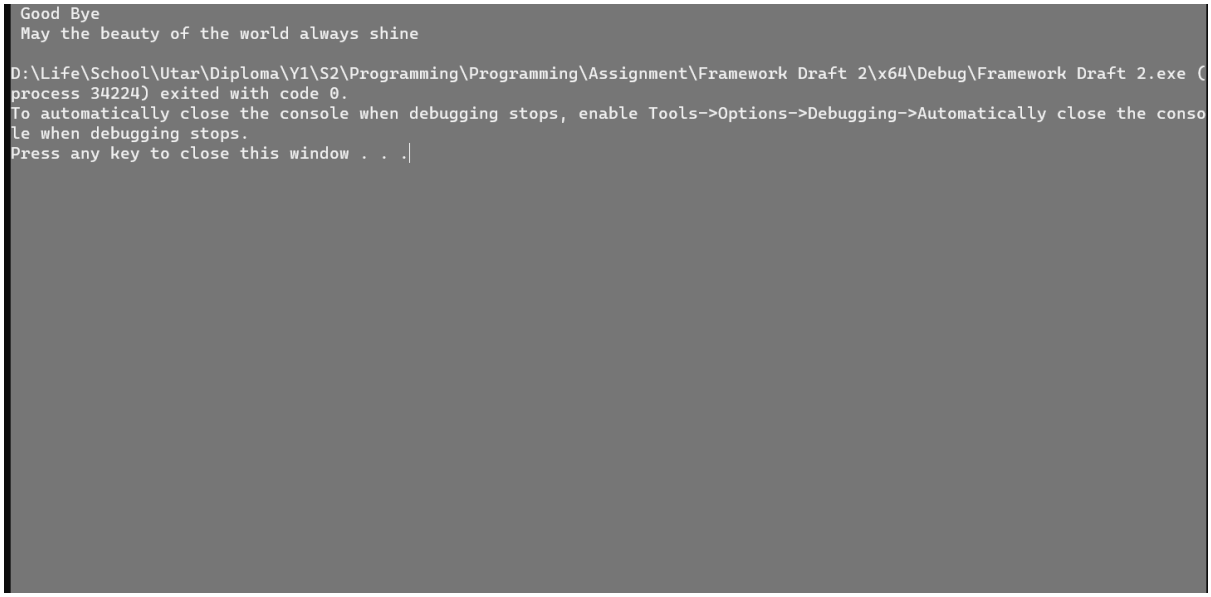
Figure 1.57 Prompt the user to exit



Figure 1.58 Program Continues



1.59 Exit the program completely



1.60 If non available selection is selected



Specialty and limitation of the program

The nature of this framework is simple: it provides the user with a basic UI. The first page which displays the word HELIX using ASCII Arts allows the user to have a cleaner understanding of what they are using. The ASCII word art is a large HELIX made out of each alphabet in the word HELIX. This will impress and attract the user more as an interesting user interface will always have the potential to attract the user to continue to use the program. Secondly is the Login In UI. This UI is clear and simple hence it will be extremely convenient for the user to proceed, when an unexpected input is entered, an error prompt will be displayed. This error prompting page is red in colour, hence can warn the user in the most effective and clear manner to indicate that the user had unfortunately keyed in the wrong username and password. Lastly is the menu login page, after a series of transition which makes the program more vivid and entertaining, the program will enter the login page which is green in word and black in background. This is the most classical way of display hence would have the most attractive design for the user. The menu login page consist of 6 selection which directs the user to 6 different function

The first second and third function will be elaborated later at part two three and four. The fourth function is the function which allows the user to add another user and their respective password. This will effectively allow the user to add in another user, hence allow more people to have the chance to use our program. The fifth function is a function which allows the current user to change its password into something else. This will allow the user to change their password hence protecting their privacy in the unfortunate circumstance that their password has been leaked. The last function requires the user to enter a number which is 100. This prevents the user from misclicking the exit button. This function allows the user to either log out of the current system or exit the program completely. This will allow the program to run more smoothly when the user wishes to just log out from the system and not exit the program completely.

There are some programs that are running behind the scene. The program will read the necessary information such as the username and their respective password from the text file. They will then be stored inside two different arrays with one username correspond to another password. This is done by recognizing the respective location in the array. For example, Name is at the 5th index of the array Username and hence the respective password will be the 5th index of the array password which is One 1234.

Other than that, for function 4, the program that runs behind the scene is that it will create two new list with one more element added into the new arrays. The new element added is the new username and new password. These two elements are added into their new respective arrays. This is done so by allowing the array to be a dynamic array. A dynamic array refers to an array that has no pre-set size value. Normally when an array is being declared, the number of elements inside the array, i.e. the size of the array must be indicated to prevent error. However, for dynamic array it uses pointer method to store the data, hence allowing the size of the dynamic array to be able to vary according to the variable given. In this case, the dynamic array is used to allow the infinite appending of the size of the array.

The system uses case function to ensure that the program can direct itself to different module of the program. This will form a loop within the system and ensure that the program can enter the required program point, hence ensuring the coherence and flow of the entire program. The result of such decision is that our program flow is extremely smooth ad every point of program can direct itself to another point that the user wants.

Some of the limitations that we have is that firstly, for function 4 and function 5, the program must exit itself to the log out page to prevent contradiction. Upon investigation we realize that the cause of this error is because the computer identifies the new array as a null pointer which is a pointer that does not contains any meaning. This resulted in when the computer tries to run the program and writes the data into the text file, the data written will be random value data that has no meaning. Which resulted in us to have to reset the text file. IN order to solve this problem, we added a compulsory exit at the end of function 4 and function 5.
Another limitation that we face is that our program is too bulky, up to 2010 lines in total. With nested loops and nested switch cases all over the place. Such complex program will hinder it's performance and make our program lagging. This can be shown in some of the testing when the program just hangs there and does not respond to any command until several seconds later.

Self-review and future improvement

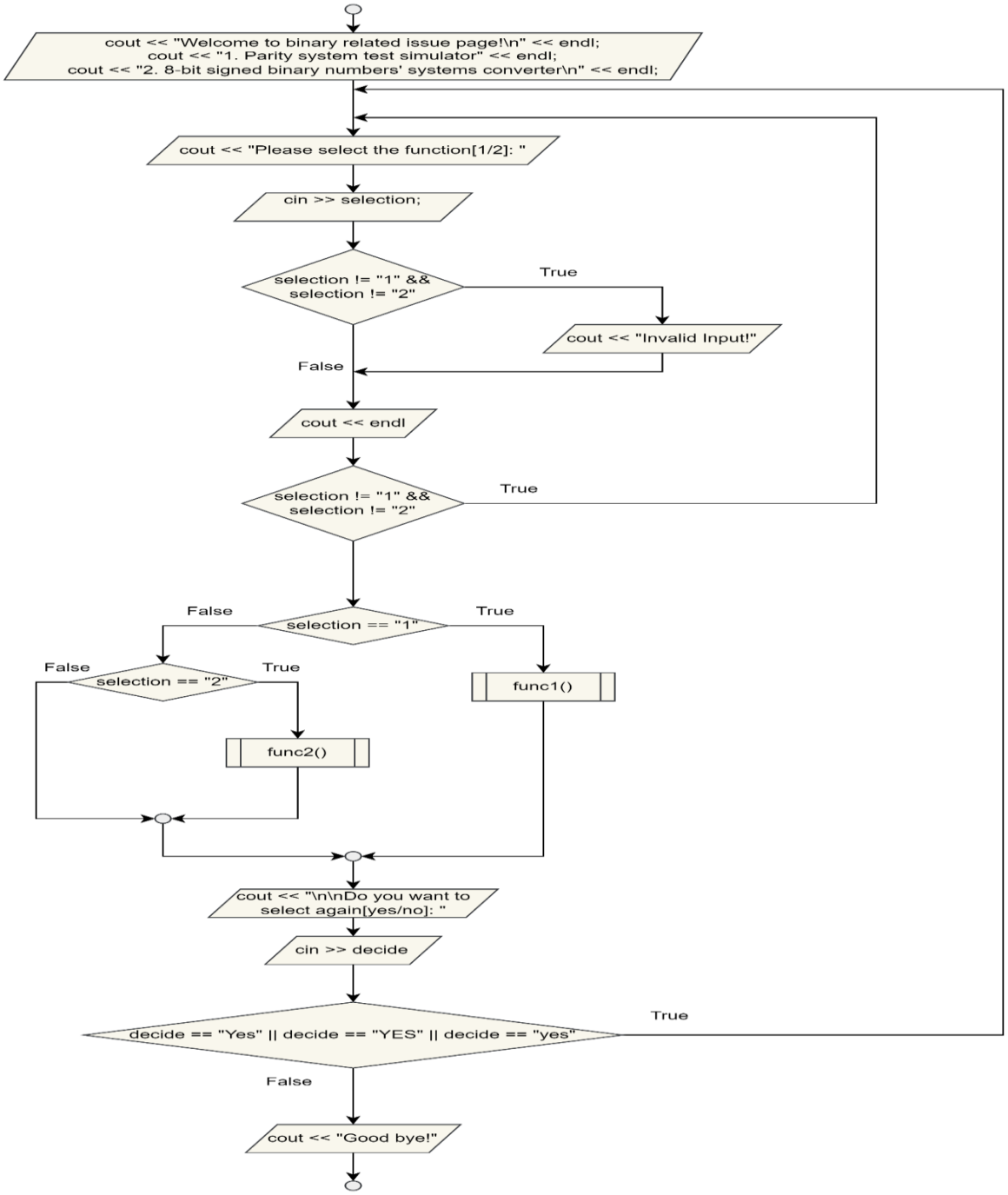
We believe that our program is a completed and well written program. But just like others it still have room for improvement.
We believe that our program has done well in several parts. Firstly is the potential to direct the user to any parts of the program they want without having to quit and restart the program all over again. This will minimize the trouble in having to restart the program or having to log in to HELIX all over again. Secondly is the usage of dynamic array. This allows us to add unlimited user and unlimited corresponding password into the system. This will provide great help to the user if he or she plans to implement HELIX in a large user count environment such as schools or factories. Lastly is the password and username system. This ensures that only the selected user will have the permission to use and edit the system.
Some future improvement for our program includes we will try to eliminate the null pointer problem. We plan to eliminate such problem by either creating a default pointer to prevent the null pointer problem from happening again. We will also try to use other methods such as fstream and direct the data directly into the login selection process rather than putting them into a string than into an array. This will decease our program size significantly hence ensuring that our program can run more smoothly. Lastly we will try to replace the most commonly used variable such as loop and loginselection into pointer. As pointer reads the address of the variable directly hence will be able to access the variable directly without having to pass through other processes. Hence by replacing the commonly used variable with pointer we can decrease the runtime of our program hence allow our program to run more smoothly. Other than that we plan to decrease the bulkiness of our program by replacing some of the parts with shorter phrases, This will allow our program to have a smaller size. Hence ensuring the effectives of our program as a whole .

Binary related issue:

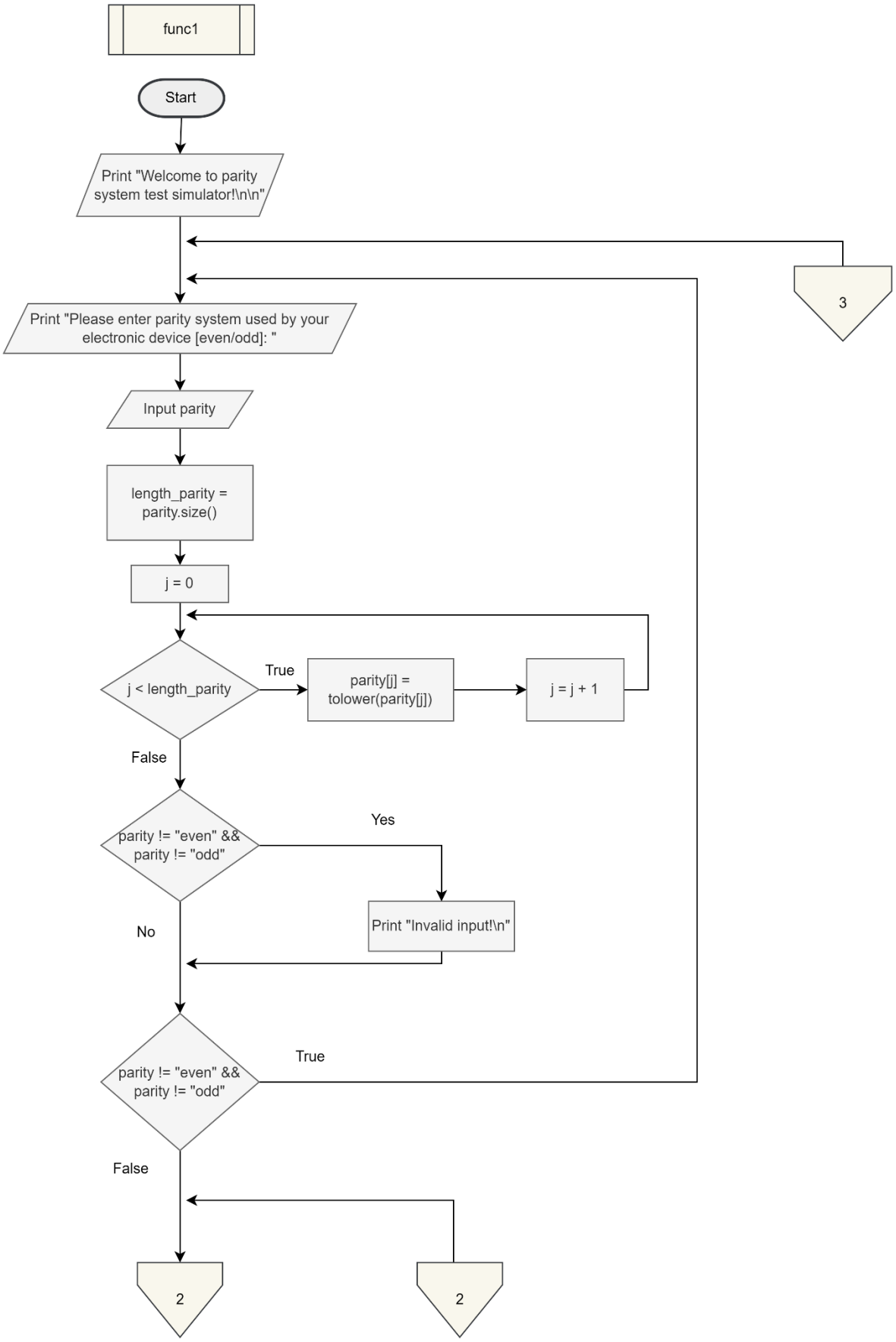
This program consists of two functionalities which are parity system test simulator and 8-bit signed binary numbers' systems converter. This program can help the user more understand the parity system and know that what kind of binary code is satisfied with it. Moreover, the user can use it to convert a decimal number to binary form in sign and magnitude system, 1's complement system and 2's complement system.

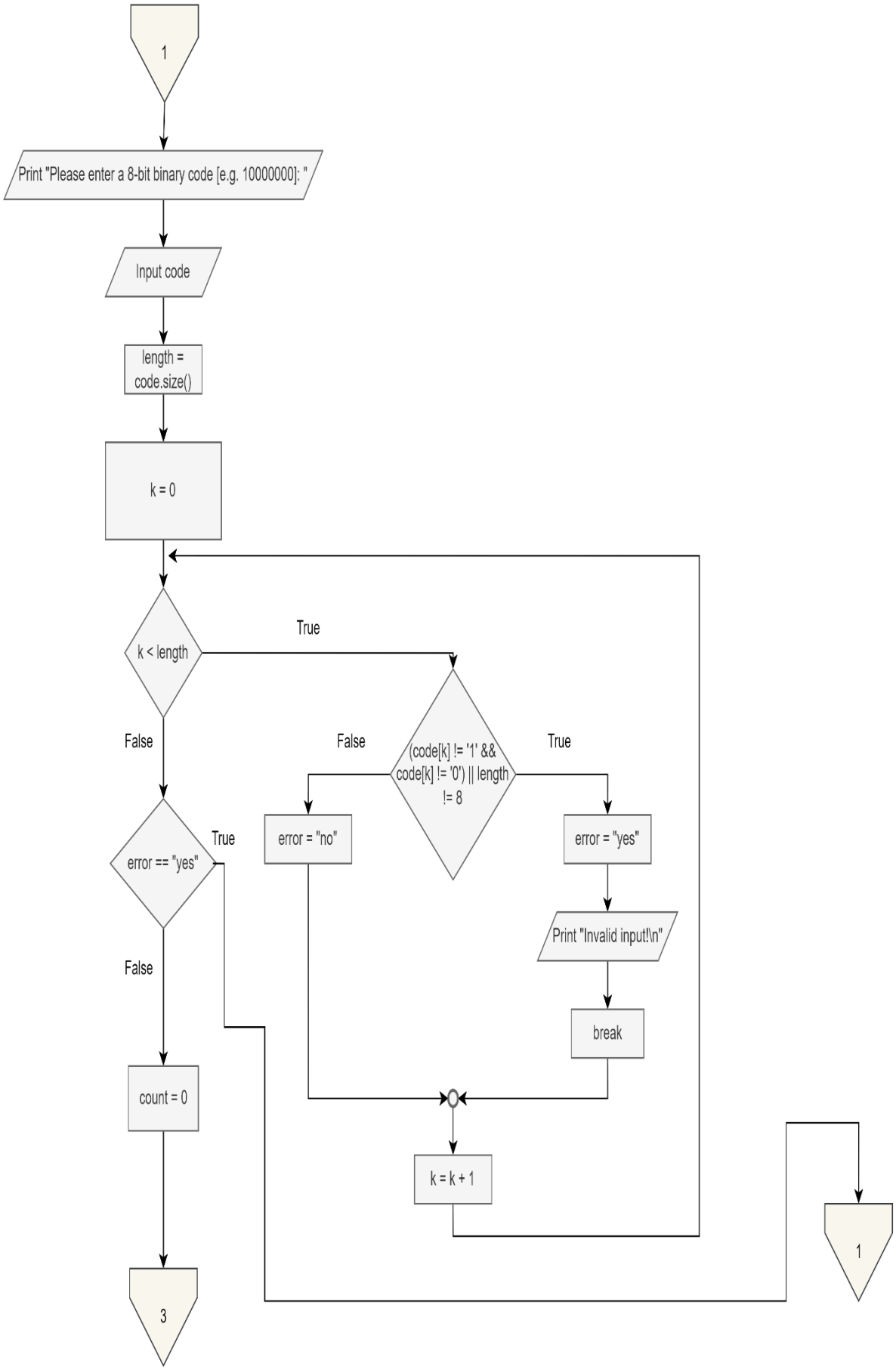
Structure chart

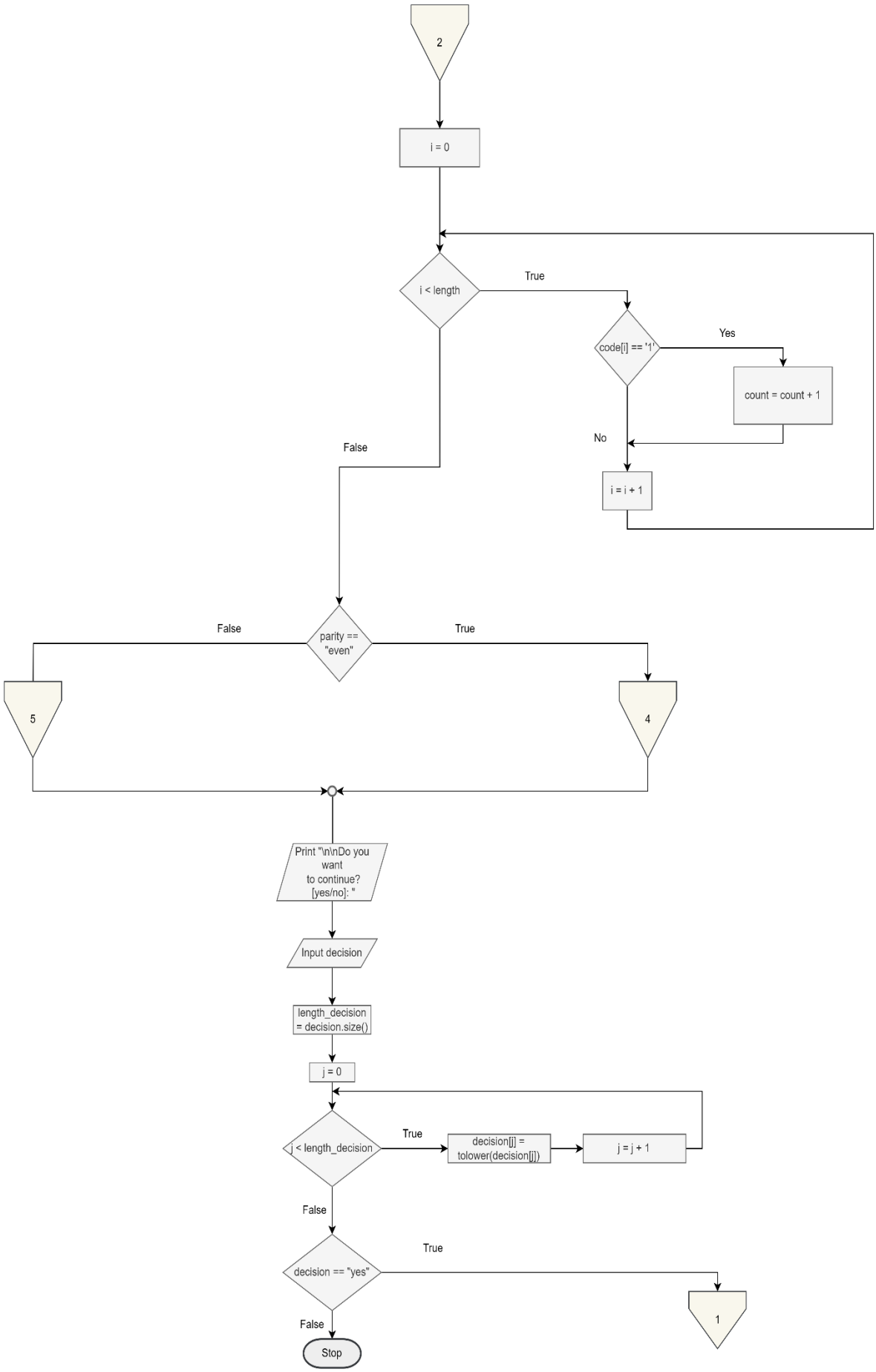
Main Function

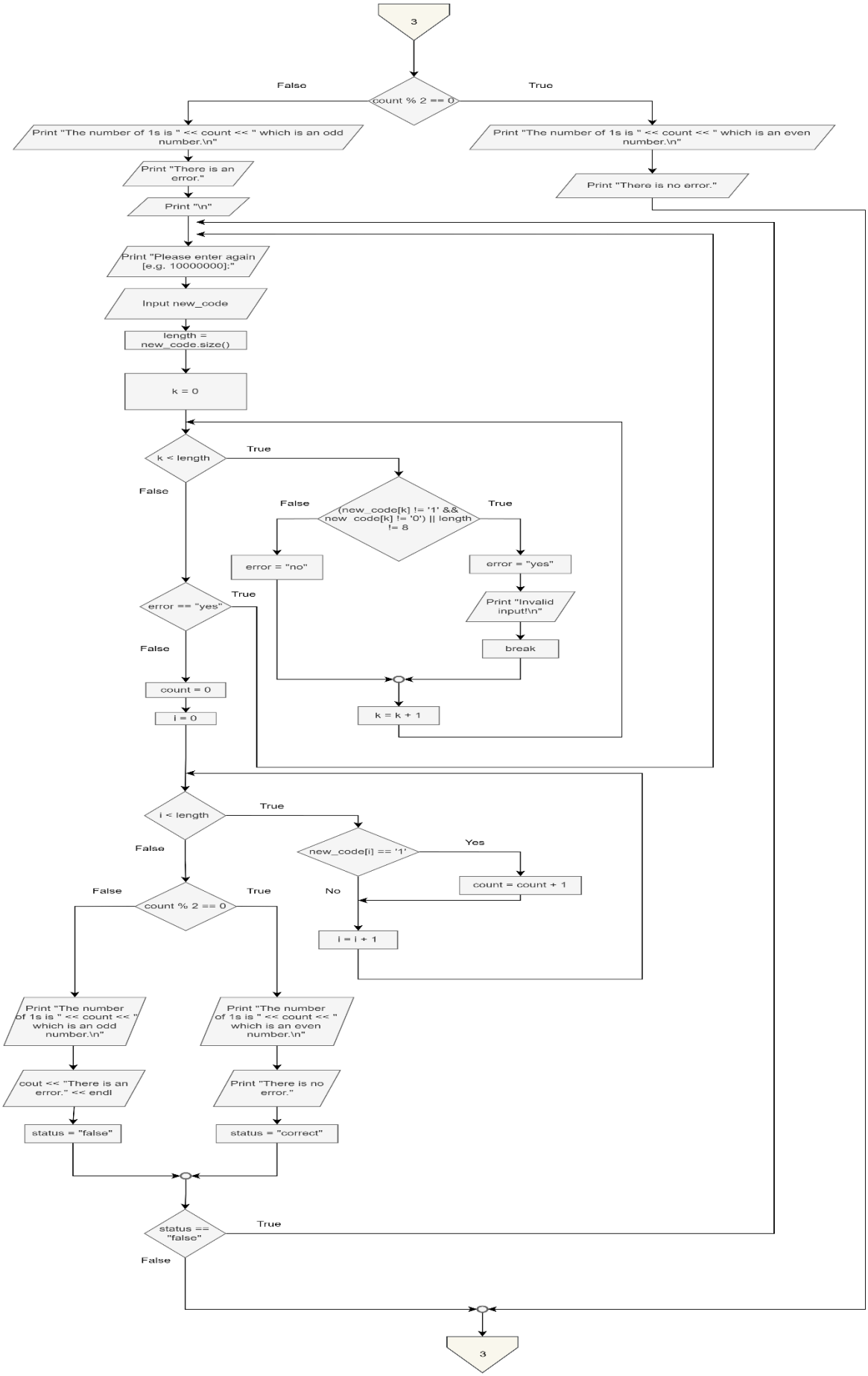


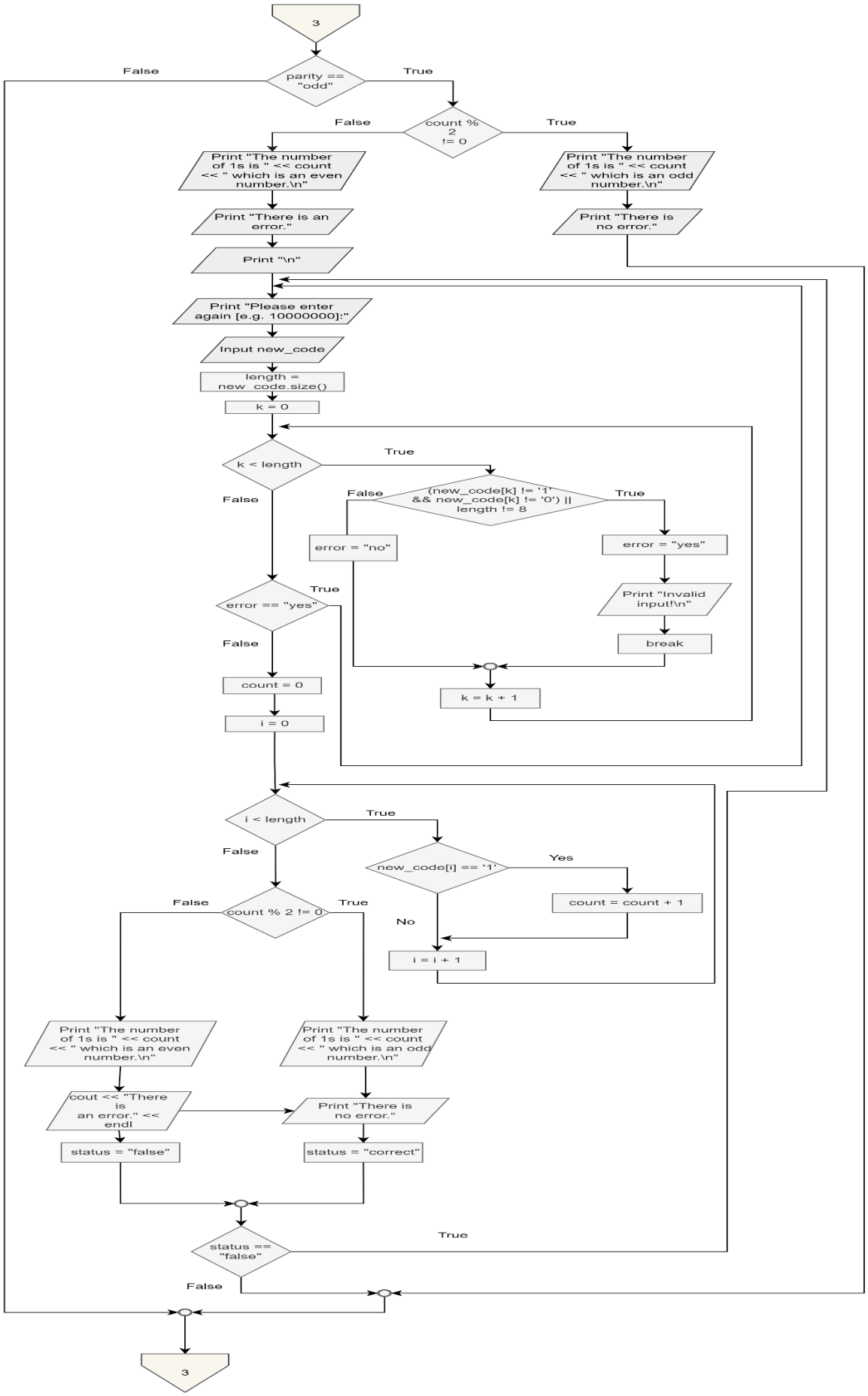
func1


func1
Start
Print "Welcome to parity system test simulator!\n\n"
3
Print "Please enter parity system used by your electronic device [even/odd]: "
Input parity
length_parity = parity.size()
j = 0
j < length_parity
True
parity[j] = tolower(parity[j])
j = j + 1
False
parity != "even" && parity != "odd"
Yes
Print "Invalid input!\n"
No
parity != "even" && parity != "odd"
True
False
2
2

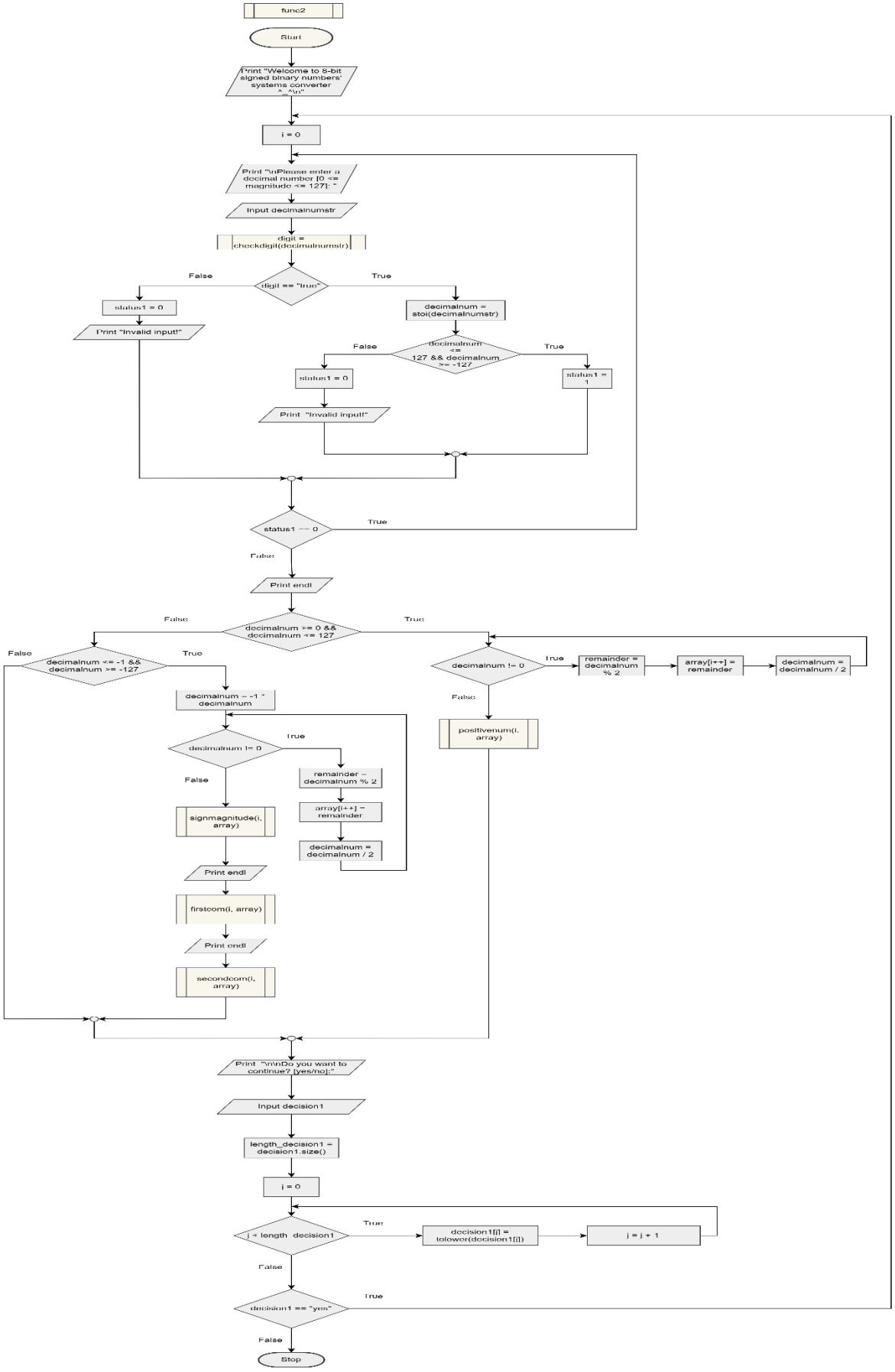

1
Print "Please enter a 8-bit binary code [e.g. 10000000]: "
Input code
length = code.size()
k = 0
k < length
True
False
(code[k] != '1' && code[k] != '0') || length != 8
False
True
error = "no"
error = "yes"
Print "Invalid input!\n"
break
error == "yes"
True
False
count = 0
k = k + 1
3
1


2
i = 0
i < length
True
code[i] == '1'
Yes
count = count + 1
No
i = i + 1
False
parity == "even"
False
True
5
4
Print "\n\nDo you want to continue? [yes/no]: "
Input decision
length_decision = decision.size()
j = 0
j < length_decision
True
decision[j] = tolower(decision[j])
j = j + 1
False
decision == "yes"
True
1
False
Stop

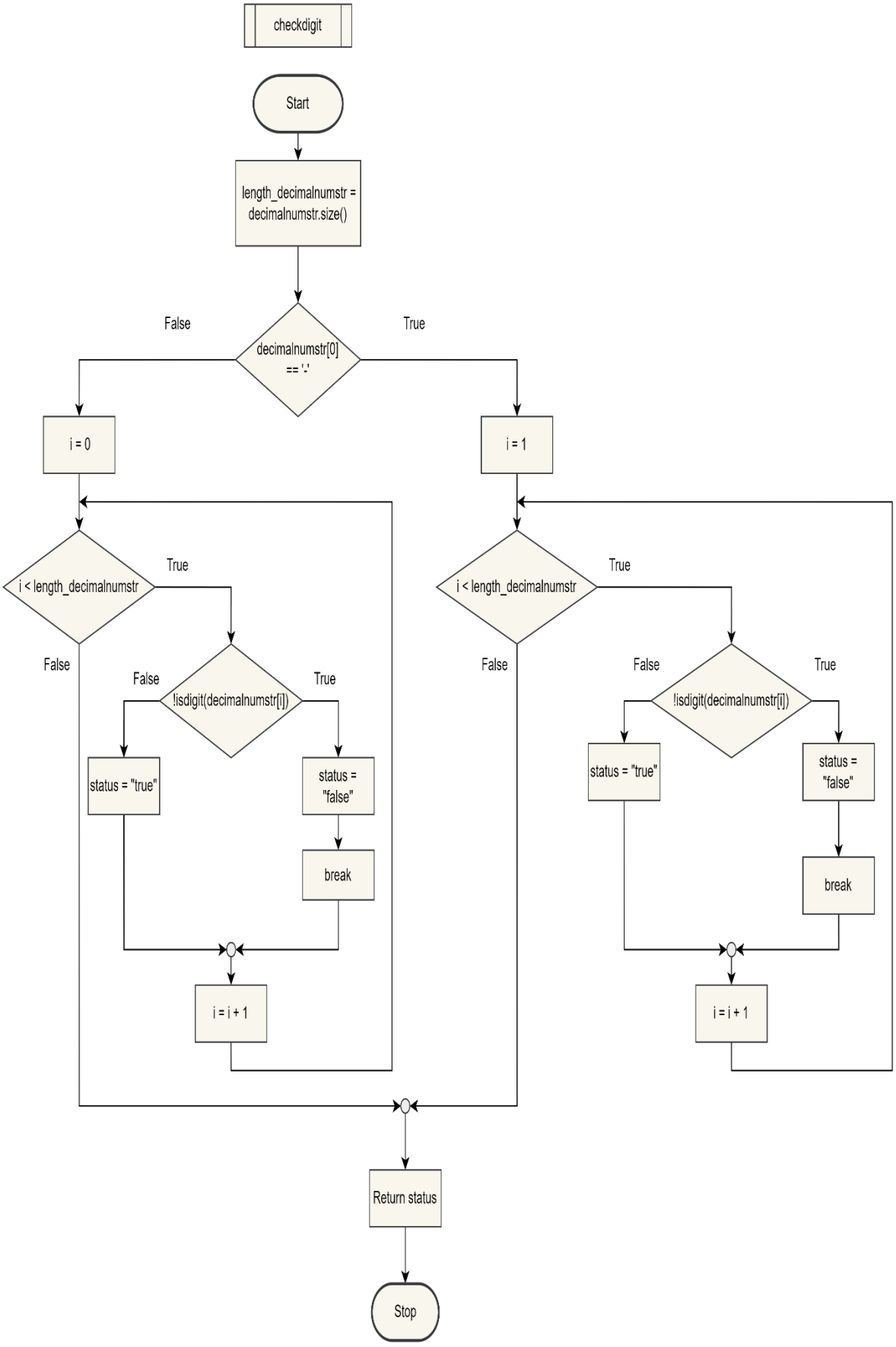

3
count % 2 == 0
False
True
Print "The number of 1s is " << count << " which is an odd number.\n"
Print "The number of 1s is " << count << " which is an even number.\n"
Print "There is an error."
Print "There is no error."
Print "\n"
Print "Please enter again [e.g. 10000000]:"
Input new_code
length = new_code.size()
k = 0
k < length
True
False
(new_code[k] != '1' && new_code[k] != '0') || length != 8
error = "no"
error = "yes"
Print "Invalid input!\n"
break
k = k + 1
error == "yes"
count = 0
i = 0
i < length
new_code[i] == '1'
Yes
No
count = count + 1
i = i + 1
Print "The number of 1s is " << count << " which is an odd number.\n"
Print "The number of 1s is " << count << " which is an even number.\n"
cout << "There is an error." << endl
Print "There is no error."
status = "false"
status = "correct"
status == "false"
3


3
parity == "odd"
False
True
count % 2 != 0
False
True
Print "The number of 1s is " << count << " which is an even number.\n"
Print "The number of 1s is " << count << " which is an odd number.\n"
Print "There is an error."
Print "There is no error."
Print "\n"
Print "Please enter again [e.g. 10000000]:"
Input new_code
length = new_code.size()
k = 0
k < length
True
False
(new_code[k] != '1' && new_code[k] != '0') || length != 8
False
True
error = "no"
error = "yes"
Print "Invalid input!\n"
break
k = k + 1
error == "yes"
True
False
count = 0
i = 0
i < length
True
False
new_code[i] == '1'
Yes
No
count = count + 1
i = i + 1
count % 2 != 0
False
True
Print "The number of 1s is " << count << " which is an even number.\n"
Print "The number of 1s is " << count << " which is an odd number.\n"
cout << "There is an error." << endl
Print "There is no error."
status = "false"
status = "correct"
status == "false"
True
False
3

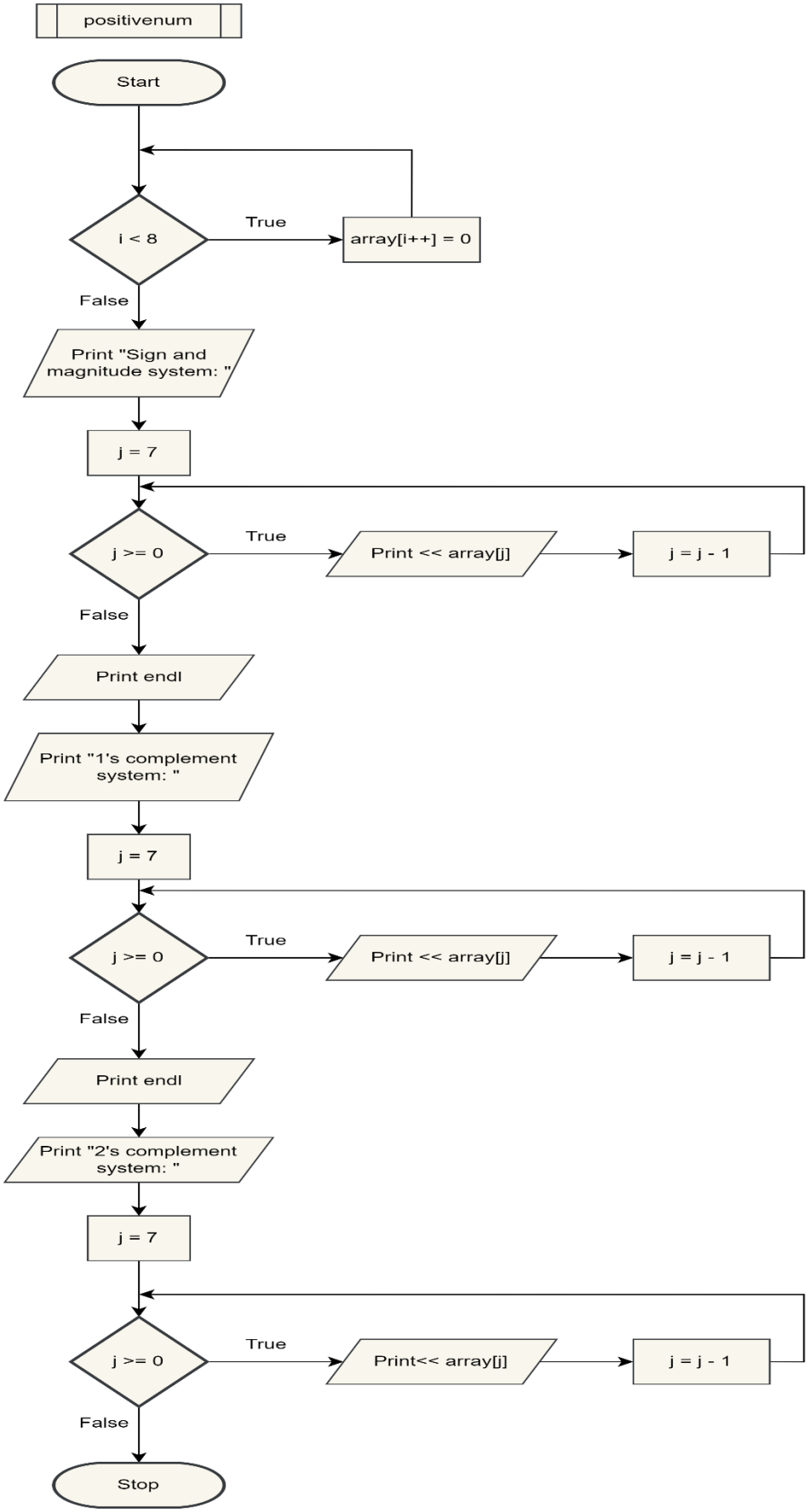
func2


func2
Start
Print "Welcome to 8-bit signed binary numbers' systems converter ^_^\n"
i = 0
Print "\nPlease enter a decimal number [0 <= magnitude <= 127]: "
Input decimalnumstr
digit = checkdigit(decimalnumstr)
digit == "true"
False
True
status1 = 0
Print "Invalid input!"
decimalnum = stoi(decimalnumstr)
decimalnum <= 127 && decimalnum >= -127
False
True
status1 = 0
Print "Invalid input!"
status1 = 1
status1 == 0
True
False
Print endl
decimalnum >= 0 && decimalnum <= 127
False
True
decimalnum <= -1 && decimalnum >= -127
False
True
decimalnum = -1 * decimalnum
decimalnum != 0
True
False
signmagnitude(i, array)
remainder = decimalnum % 2
array[i++] = remainder
decimalnum = decimalnum / 2
Print endl
firstcom(i, array)
Print endl
secondcom(i, array)
decimalnum != 0
True
remainder = decimalnum % 2
array[i++] = remainder
decimalnum = decimalnum / 2
False
positivenum(i, array)
Print "\n\nDo you want to continue? [yes/no]:"
Input decision1
length_decision1 = decision1.size()
j = 0
j < length_decision1
True
decision1[j] = tolower(decision1[j])
j = j + 1
False
decision1 == "yes"
True
False
Stop

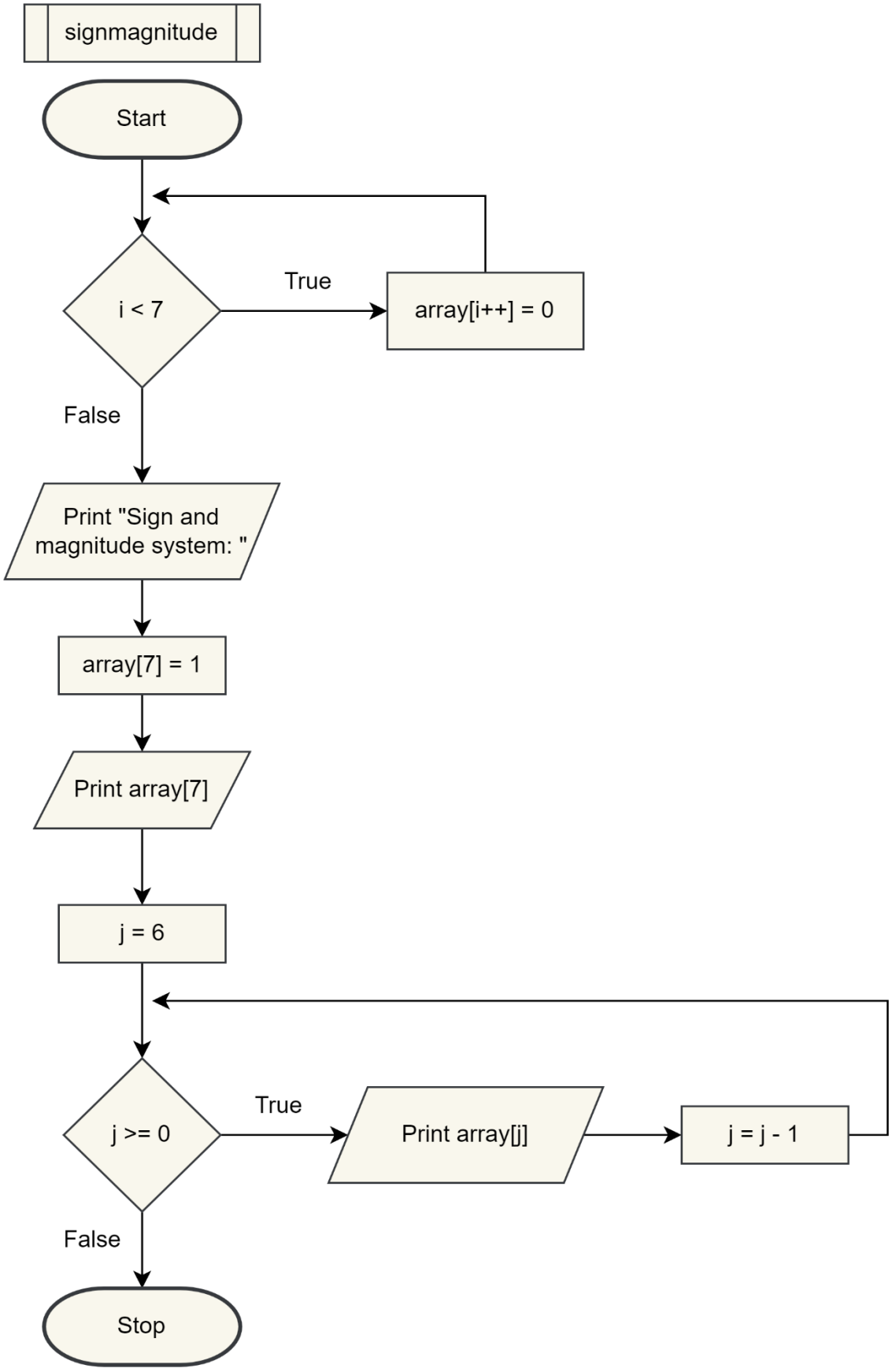
checkdigit


checkdigit
Start
length_decimalnumstr = decimalnumstr.size()
False
decimalnumstr[0] == '.'
True
i = 0
i = 1
i < length_decimalnumstr
True
False
!isdigit(decimalnumstr[i])
False
True
status = "true"
status = "false"
break
i = i + 1
Return status
Stop

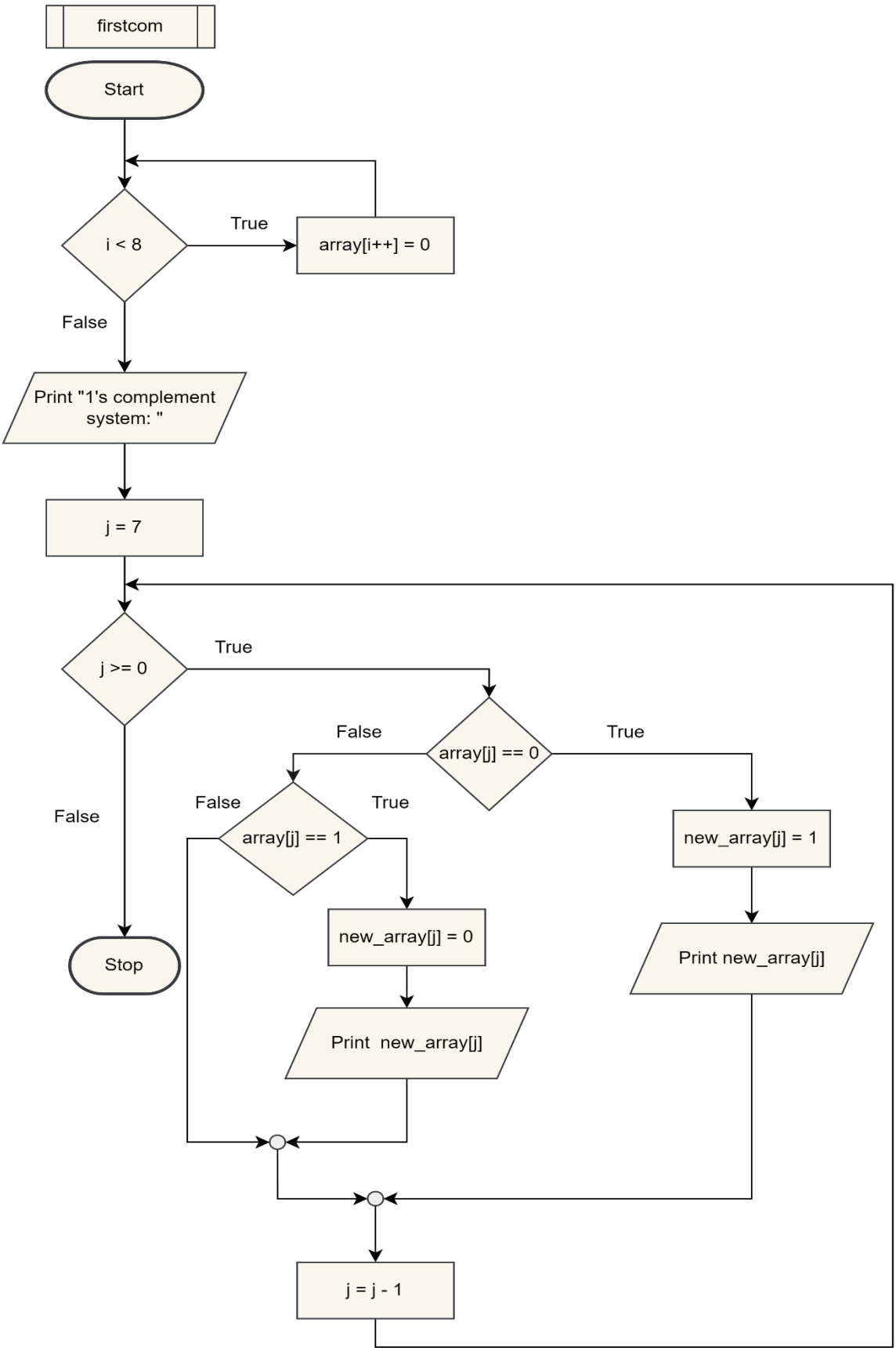
positivenum



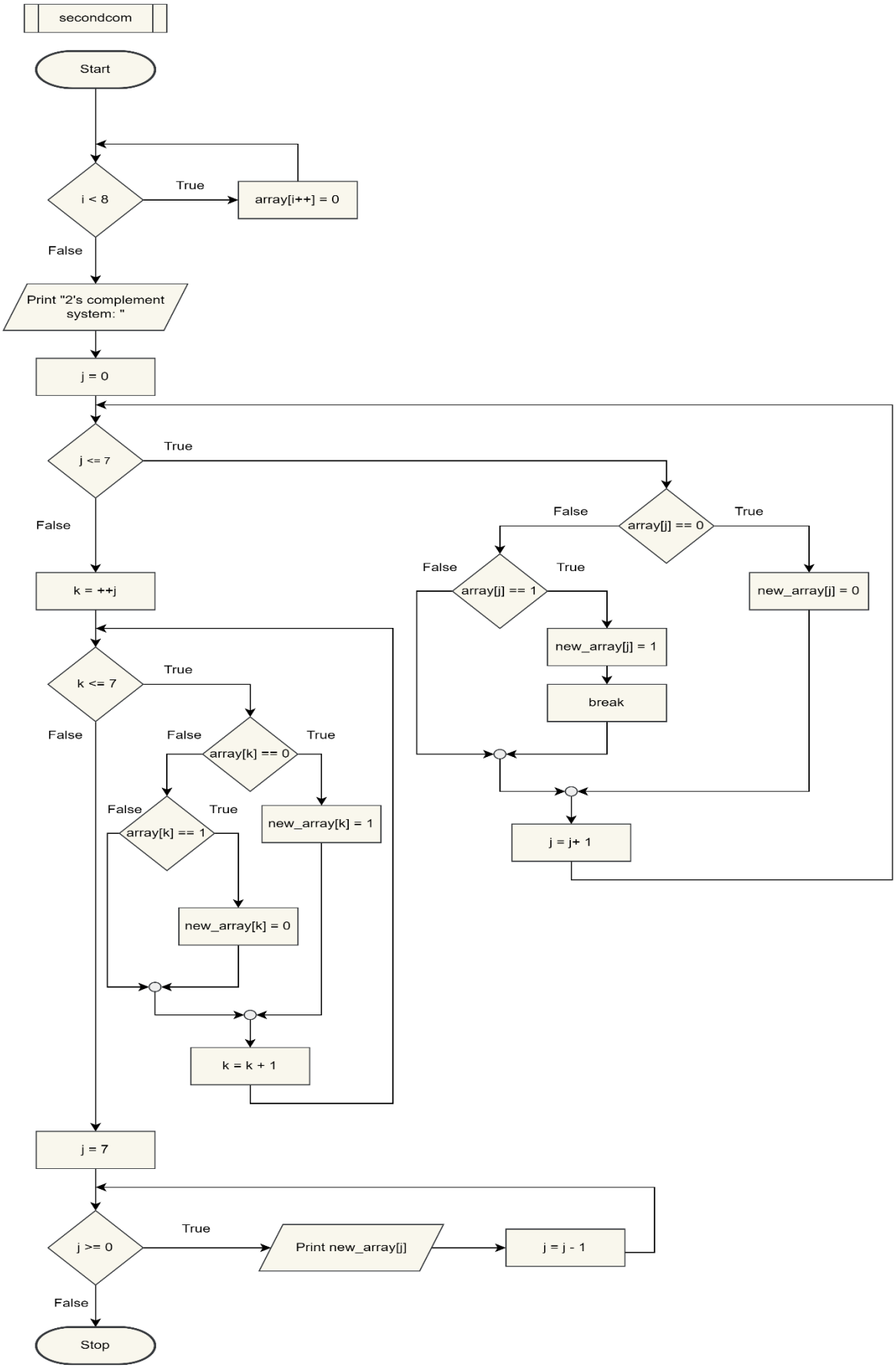
signmagnitude



firstcom


firstcom
Start
i < 8
True
array[i++] = 0
False
Print "1's complement system: "
j = 7
j >= 0
True
False
Stop
array[j] == 0
False
True
array[j] == 1
False
True
new_array[j] = 0
Print new_array[j]
new_array[j] = 0
Print new_array[j]
j = j - 1

secondcom


secondcom
Start
i < 8
True
array[i++] = 0
False
Print "2's complement system: "
j = 0
j <= 7
True
False
array[j] == 0
False
True
array[j] == 1
False
True
new_array[j] = 0
new_array[j] = 1
break
k = ++j
k <= 7
True
False
array[k] == 0
False
True
array[k] == 1
False
True
new_array[k] = 1
new_array[k] = 0
k = k + 1
j = j+ 1
j = 7
j >= 0
True
Print new_array[j]
j = j - 1
False
Stop

Screen shots of program

```
#include <iostream>
#include <cctype>
#include <string>
using namespace std;

void positivenum(int, int[]);
void signmagnitude(int, int[]);
void firstcom(int, int[]);
void secondcom(int, int[]);
string checkdigit(string);
```

Figure 2.1 shows the header files that included and function prototypes.

```
int main()
{
    string selection;
    string decide;

    cout << "Welcome to binary related issue page!\n" << endl;
    cout << "1. Parity system test simulator" << endl;
    cout << "2. 8-bit signed binary numbers' systems converter\n" << endl;

    do
    {
        do
        {
            cout << "Please select the function[1/2]: ";
            cin >> selection;
            if (selection != "1" && selection != "2")
                cout << "Invalid Input!";
            cout << endl;
        } while (selection != "1" && selection != "2");

        if (selection == "1")
        {
            func1();
        }

        else if (selection == "2")
        {
            func2();
        }

        cout << "\n\nDo you want to select again[yes/no]: ";
        cin >> decide;
    } while (decide == "Yes" || decide == "YES" || decide == "yes");

    cout << "Good bye!";
    return 0;
}
```

Figure 2.2: The main function of the program.

```
void func1()
{
    string code;
    string new_code;
    string parity;
    string status;
    string decision;
    string error;
    int length;
    int length_parity;
    int length_decision;

    cout << "Welcome to parity system test simulator!\n\n";

    do
    {
        do
        {
            cout << "Please enter parity system used by your electronic device [even/odd]: "; // prompt the guest to enter the parity system used
            cin >> parity; // enter parity system used
            length_parity = parity.size();
            for (int j = 0; j < length_parity; j++) //To change each element in parity to lowercase
            {
                parity[j] = tolower(parity[j]);
            }
            if (parity != "even" && parity != "odd")
                cout << "Invalid input!\n";
        } while (parity != "even" && parity != "odd"); //Ask the guest enter again if input is wrong
```

Figure 2.3: func1 function which is the parity system test simulator (Part1)

```
        do
        {
            cout << "Please enter a 8-bit binary code [e.g. 10000000]: "; //prompt the guest to enter 8-bit binary code
            cin >> code; // enter the code
            length = code.size(); //To get the size or length of code
            for (int k = 0; k < length; k++) //if the element in the code is not 1 and 0, there is an error
            {                                //if the binary code entered is not 8-bit, there is an error
                if ((code[k] != '1' && code[k] != '0') || length != 8)
                {
                    error = "yes";
                    cout << "Invalid input!\n";
                    break;
                }
                else
                    error = "no";
            }
        } while (error == "yes"); //if error == yes, means there is invalid input, the program will ask the guest to enter again


        int count = 0;
        for (int i = 0; i < length; i++) //check each element inside code if equals to '1', if yes, the program record down the number of one
        {

            if (code[i] == '1')
            {
                count = count + 1;
            }
        }
```

Figure 2.4: func1 function which is the parity system test simulator (Part 2)

```
if (parity == "even")
{
    if (count % 2 == 0) // To show the count is an even number
    {
        cout << "The number of 1s is " << count << " which is an even number.\n";
        cout << "There is no error.";
    }

    else
    {
        cout << "The number of 1s is " << count << " which is an odd number.\n";
        cout << "There is an error."; //error occurs
        cout << "\n";

        do
        {
            do
            {
                cout << "Please enter again [e.g. 10000000]:"; //ask the guest to enter again
                cin >> new_code;
                length = new_code.size();
                for (int k = 0; k < length; k++) //if the element in the new_code is not 1 and 0, there is an error
                {                                //if the binary code entered is not 8-bit, there is an error
                    if ((new_code[k] != '1' && new_code[k] != '0') || length != 8)
                    {
                        error = "yes";
                        cout << "Invalid input!\n";
                        break;
```

Figure 2.5: func1 function which is the parity system test simulator (Part 3)

```
                        break;
                    }
                    else
                        error = "no";
                }
            } while (error == "yes"); //if error == yes, means there is invalid input, the program will ask the guest to enter again


            int count = 0;
            for (int i = 0; i < length; i++) //check each element inside new entered code if equals to '1', if yes, the program record down the number of one
            {

                if (new_code[i] == '1')
                {
                    count = count + 1;
                }
            }

            if (count % 2 == 0) // To show the count is an even number
            {
                cout << "The number of 1s is " << count << " which is an even number.\n";
                cout << "There is no error.";
                status = "correct";
            }
            else
            {
                cout << "The number of 1s is " << count << " which is an odd number.\n";
                cout << "There is an error." << endl;
                status = "false";
            }
```

Figure 2.6: func1 function which is the parity system test simulator (Part 4)

```
            }
        } while (status == "false"); //Error still occurs. The program will ask the guest to enter again until the status is correct.
    }
}

else if (parity == "odd")
{
    if (count % 2 != 0) // To show the count is an odd number
    {
        cout << "The number of 1s is " << count << " which is an odd number.\n";
        cout << "There is no error.";
    }

    else
    {
        cout << "The number of 1s is " << count << " which is an even number.\n";
        cout << "There is an error."; //error occurs
        cout << "\n";

        do
        {
            do
            {
                cout << "Please enter again [e.g. 10000000]:"; //ask the guest to enter again
                cin >> new_code;
                length = new_code.size();
                for (int k = 0; k < length; k++) //if the element in the new_code is not 1 and 0, there is an error
                {                                //if the binary code entered is not 8-bit, there is an error
                    if ((new_code[k] != '1' && new_code[k] != '0') || length != 8)
                    {
                        error = "yes";
                        cout << "Invalid input!\n";
```

Figure 2.7: func1 function which is the parity system test simulator (Part 5)

```
                        error = "yes";
                        cout << "Invalid input!\n";
                        break;
                    }
                    else
                        error = "no";
                }
            } while (error == "yes"); //if error == yes, means there is invalid input, the program will ask the guest to enter again


            int count = 0;
            for (int i = 0; i < length; i++) //check each element inside new entered code if equals to '1', if yes, the program record down the number of one
            {

                if (new_code[i] == '1')
                {
                    count = count + 1;
                }
            }

            if (count % 2 != 0) // To show the count is an odd number
            {
                cout << "The number of 1s is " << count << " which is an odd number.\n";
                cout << "There is no error.";
                status = "correct";
            }

            else
            {
                cout << "The number of 1s is " << count << " which is an even number.\n";
                cout << "There is an error." << endl;
                status = "false";
```

Figure 2.8: func1 function which is the parity system test simulator (Part 6)

```
            }

            if (count % 2 != 0) // To show the count is an odd number
            {
                cout << "The number of 1s is " << count << " which is an odd number.\n";
                cout << "There is no error.";
                status = "correct";
            }

            else
            {
                cout << "The number of 1s is " << count << " which is an even number.\n";
                cout << "There is an error." << endl;
                status = "false";
            }
          } while (status == "false"); //Error still occurs. The program will ask the guest to enter again until the status is correct.
        }
    }
    cout << "\n\nDo you want to continue? [yes/no]: "; //Ask the guest if want to continue
    cin >> decision;
    length_decision = decision.size();
    for (int j = 0; j < length_decision; j++) //To change each element in decision to lowercase
    {
        decision[j] = tolower(decision[j]);
    }
} while (decision == "yes"); // The program will run again if the decision is yes.
}
```

Figure 2.9: func1 function which is the parity system test simulator (Part 7)

```
void func2()
{
    string decision1;
    string decimalnumstr;
    string digit;
    int remainder;
    int decimalnum;
    int array[8]; //set array's size to 8 for 8-bit binary number
    int status1;
    int length_decision1;

    cout << "Welcome to 8-bit signed binary numbers' systems converter ^_^\n";

    do
    {
        int i = 0;
        do
        {
            cout << "\nPlease enter a decimal number [0 <= magnitude <= 127]: "; // ask guest to enter a decimal number
            cin >> decimalnumstr; // enter decimal number in string data type

            digit = checkdigit(decimalnumstr); //call function of checkdigit to check if each character in decimalnumstr
            // is digit. If all the characters are digit, the function returns "true"
            // if got one character is not digit, the function returns "false"
            if (digit == "true")
            {
                decimalnum = stoi(decimalnumstr); //Convert string decimalnumstr to int decimalnum by stoi (use <string> header file)

                if (decimalnum <= 127 && decimalnum >= -127) //if decimalnum is between -127 to 127, the status is 1
                    status1 = 1;
```

Figure 2.10: The func2 function which is the 8-bit signed binary numbers' systems converter (Part 1)

```
            status1 = 1;
        else                                    //else the status is 0 and Invalid input! is printed out.
        {
            status1 = 0;
            cout << "Invalid input!";
        }
    }

    else //else the status is 0 and Invalid input! is printed out.
    {
        status1 = 0;
        cout << "Invalid input!";
    }

} while (status1 == 0); //if decimal number is not between 127 and -127 or the input is not digit, the program ask the guest to enter the decimal value again

cout << endl;

if (decimalnum >= 0 && decimalnum <= 127) // for positive decimal number case
{
    while (decimalnum != 0)  // when decimalnum == 0, no more division is required
    {

        remainder = decimalnum % 2; // To calculate remainder by % 2 because of binary system
        array[i++] = remainder;  // current remainder value store in array[i]
        decimalnum = decimalnum / 2;  // i = i + 1 , To calculate new decimalnum

    }

    positivenum(i, array); // To convert positive decimal number to signed binary numbers' systems
```

Figure 2.11: The func2 function which is the 8-bit signed binary numbers' systems converter (Part 2)

```
    }

    else if (decimalnum <= -1 && decimalnum >= -127) //for negative decimal number case
    {
        decimalnum = -1 * decimalnum; //to get the magnitude of decimal number

        while (decimalnum != 0)  // when decimalnum == 0, no more division is required
        {

            remainder = decimalnum % 2; // To calculate remainder by % 2 because of binary system
            array[i++] = remainder;  // current remainder value store in array[i]
            decimalnum = decimalnum / 2;  // i = i + 1 , To calculate new decimalnum

        }

        signmagnitude(i, array); //Convert to sign and magnitude system
        cout << endl;

        firstcom(i, array); //Convert to 1's complement system
        cout << endl;

        secondcom(i, array); //Convert to 2's complement system
    }

    cout << "\n\nDo you want to continue? [yes/no]:";
    cin >> decision1;
    length_decision1 = decision1.size();
    for (int j = 0; j < length_decision1; j++) //To change each element in decision to lowercase
    {
        decision1[j] = tolower(decision1[j]);
```

Figure 2.12: The func2 function which is the 8-bit signed binary numbers' systems converter (Part 3)

```
            signmagnitude(i, array); //Convert to sign and magnitude system
            cout << endl;

            firstcom(i, array); //Convert to 1's complement system
            cout << endl;

            secondcom(i, array); //Convert to 2's complement system
        }

        cout << "\n\nDo you want to continue? [yes/no]:";
        cin >> decision1;
        length_decision1 = decision1.size();
        for (int j = 0; j < length_decision1; j++) //To change each element in decision to lowercase
        {
            decision1[j] = tolower(decision1[j]);
        }
    } while (decision1 == "yes"); //if the guest enter yes as decision, the program will run again
}
```

Figure 2.13: The func2 function which is the 8-bit signed binary numbers' systems converter (Part 4)

```
string checkdigit(string decimalnumstr)
{
    int length_decimalnumstr = decimalnumstr.size(); //To get size or length of decimalnumstr.
    string status;

    if (decimalnumstr[0] == '-') //if the first character is '-', the program will ignore it and check from next character.
    {
        for (int i = 1; i < length_decimalnumstr; i++) //check from the character after '-'.
        {
            if (!isdigit(decimalnumstr[i])) //if the charater is not a digit, the status = "false" and the for loop break.
            {
                status = "false";
                break;
            }
            else
                status = "true"; //if the character is a digit, the status = "true"
        }
    }

    else
    {
        for (int i = 0; i < length_decimalnumstr; i++) //check from first value to the end.
        {
            if (!isdigit(decimalnumstr[i])) //if the charater is not a digit, the status = "false" and the for loop break.
            {
                status = "false";
                break;
            }
            else
                status = "true"; //if the character is a digit, the status = "true".
        }
    }
    return status; //return status either "true" or "false".
}
```

Figure 2.14 shows the checkdigit function which check if each character in decimalnumstr is digit.

```
void positivenum(int i, int array[])
{
    while (i < 8)  // when number of bit less than 8, add zero until 8 bit
    {
        array[i++] = 0;
    }

    cout << "Sign and magnitude system: ";
    for (int j = 7; j >= 0; j--) //print out the binary number in an correct order by starting from the highest index of the array
    {
        cout << array[j];
    }

    cout << endl;

    cout << "1's complement system: ";
    for (int j = 7; j >= 0; j--) //print out the binary number in an correct order by starting from the highest index of the array
    {
        cout << array[j];
    }

    cout << endl;

    cout << "2's complement system: ";
    for (int j = 7; j >= 0; j--) //print out the binary number in an correct order by starting from the highest index of the array
    {
        cout << array[j];
    }
}
```

Figure 2.15 shows the positivenum function which will print out the binary form in three binary system of a positive decimal number.

```
void signmagnitude(int i, int array[])
{
    while (i < 7)  //when number of bit less than 7, add zero until 7 bit
    {
        array[i++] = 0;
    }


    cout << "Sign and magnitude system: ";
    array[7] = 1; //for this system the most significant bit will always be 1 to show the negative sign
    cout << array[7];
    for (int j = 6; j >= 0; j--) //print out magnitude from the second highest index
    {
        cout << array[j];
    }
}
```

Figure 2.16 shows the signmagnitude function which will print out the binary form of a negative decimal number in sign and magnitude system.

```
void firstcom(int i, int array[])
{
    int new_array[8];

    while (i < 8)  // when number of bit less than 8, add zero until 8 bit
    {
        array[i++] = 0;
    }

    cout << "1's complement system: ";

    for (int j = 7; j >= 0; j--)
    {
        if (array[j] == 0)  // to change 0 to 1
        {
            new_array[j] = 1;
            cout << new_array[j];
        }
        else if (array[j] == 1) // to change 1 to 0
        {
            new_array[j] = 0;
            cout << new_array[j];
        }
    }
}
```

Figure 2.17 shows the firstcom function which will print out the binary form of a negative decimal number in 1's complement system.

```
void secondcom(int i, int array[])
{
    int new_array[8];
    int j;

    while (i < 8) // when number of bit less than 8, add zero until 8 bit
    {
        array[i++] = 0;
    }

    cout << "2's complement system: ";
    for (j = 0; j <= 7; j++) //check from lowest index of array to determine the location of 1
    {
        if (array[j] == 0)  //if array[j] = 0, the value remains at 0
        {
            new_array[j] = 0;
        }

        else if (array[j] == 1) //if array[j] = 1, the value remains at 1, the for loop stop.
        {
            new_array[j] = 1;
            break;
        }

    }

    for (int k = ++j; k <= 7; k++) // the bit value after 1 will change
    {
        if (array[k] == 0) // 0 change to 1
        {
            new_array[k] = 1;
        }

        else if (array[k] == 1) // 1 change to 0
        {
            new_array[k] = 0;
        }
    }

    for (int j = 7; j >= 0; j--) //print out the binary number in an correct order by starting from the highest index of the array
    {
        cout << new_array[j];
    }
}
```

Figure 2.18 shows the second com function which will print out the binary form of a negative decimal number in 2's complement system.

Test plans and results

```
Welcome to binary related issue page!

1. Parity system test simulator
2. 8-bit signed binary numbers' systems converter

Please select the function[1/2]:
```

Figure 2.19: The program will display 2 functions and prompt the user to select the function that they want.

```
Welcome to binary related issue page!

1. Parity system test simulator
2. 8-bit signed binary numbers' systems converter

Please select the function[1/2]: 123
Invalid Input!
Please select the function[1/2]: a1d
Invalid Input!
Please select the function[1/2]:
```

Figure 2.20: If the user key in an input which is not 1 or 2, the program will display invalid input and will prompt the user to key in again.

```
Welcome to binary related issue page!

1. Parity system test simulator
2. 8-bit signed binary numbers' systems converter

Please select the function[1/2]: 123
Invalid Input!
Please select the function[1/2]: a1d
Invalid Input!
Please select the function[1/2]: 1

Welcome to parity system test simulator!

Please enter parity system used by your electronic device [even/odd]:
```

Figure 2.21: After the user key in 1, the program will enter the parity system test simulator and prompt the user to enter parity system used by the electronic device.

```
Welcome to parity system test simulator!

Please enter parity system used by your electronic device [even/odd]: 123
Invalid input!
Please enter parity system used by your electronic device [even/odd]: dws1
Invalid input!
Please enter parity system used by your electronic device [even/odd]:
```

Figure 2.22: If the input is unsatisfied, the program will show invalid input and will prompt the user to key in again.

```
Please enter parity system used by your electronic device [even/odd]: even
Please enter a 8-bit binary code [e.g. 10000000]: 1111111
Invalid input!
Please enter a 8-bit binary code [e.g. 10000000]: 111111111
Invalid input!
Please enter a 8-bit binary code [e.g. 10000000]: dfgd12
Invalid input!
Please enter a 8-bit binary code [e.g. 10000000]:
```

Figure 2.23: The program will prompt the user to enter an 8-bit binary code. If the input is less than or more than 8-bit or the input is unsatisfied, the program will show invalid input and prompt the user to key in again.

```
Please enter a 8-bit binary code [e.g. 10000000]: 11110000
The number of 1s is 4 which is an even number.
There is no error.

Do you want to continue? [yes/no]:
```

Figure 2.24: If the user key in a correct input, the program will display number of 1 inside the input and tell the user whether it is an even number. If the input fulfills the parity system entered, the program will display no error and ask the user if want to continue.

```
Do you want to continue? [yes/no]: yes
Please enter parity system used by your electronic device [even/odd]: even
Please enter a 8-bit binary code [e.g. 10000000]: 11100000
The number of 1s is 3 which is an odd number.
There is an error.
Please enter again [e.g. 10000000]:11000000
The number of 1s is 2 which is an even number.
There is no error.

Do you want to continue? [yes/no]: no


Do you want to select again[yes/no]:
```

Figure 2.25: If the user enters yes, they will run this function again. If the binary code is not fulfilled with the parity system, the program will display error and ask the user to key in again until it is correct. The operation is the same when odd parity system is selected. After that, the program will ask the user to select the function again if they don't want to continue with this function.

```
Do you want to select again[yes/no]: yes
Please select the function[1/2]: 2

Welcome to 8-bit signed binary numbers' systems converter ^_^

Please enter a decimal number [0 <= magnitude <= 127]:
```

Figure 2.26: If it is yes and the selected function is 2, the program will enter to 8-bit signed binary numbers' systems converter and prompt the user to key in a decimal number.

```
Welcome to 8-bit signed binary numbers' systems converter ^_^

Please enter a decimal number [0 <= magnitude <= 127]: 128
Invalid input!
Please enter a decimal number [0 <= magnitude <= 127]: 1df
Invalid input!
Please enter a decimal number [0 <= magnitude <= 127]:
```

Figure 2.27: If the magnitude of the number is out of range or not satisfied, the program will show invalid input and prompt user to key in again.

```
Please enter a decimal number [0 <= magnitude <= 127]: 1

Sign and magnitude system: 00000001
1's complement system: 00000001
2's complement system: 00000001

Do you want to continue? [yes/no]:yes

Please enter a decimal number [0 <= magnitude <= 127]: -1

Sign and magnitude system: 10000001
1's complement system: 11111110
2's complement system: 11111111

Do you want to continue? [yes/no]:
```

Figure 2.28: If the user enters a decimal number, the program will show its binary form in sign and magnitude system, 1's complement system and 2's complement system. After that, the program will ask the user if want to continue. If yes, it will be run again.

```
Do you want to continue? [yes/no]:no


Do you want to select again[yes/no]: no
Good bye!
```

Figure 2.29: If the user does not want to continue, the program will ask the user again if want to select the function again. If it is no, the program displays "Good bye".

Specialty and limitation of the program

For the specialty of the program, the program provides two functions that related to binary issues. The first functionality is parity system test simulator. When this function is running, the user is required to enter parity system used by the electronic device and an 8-bit binary code. If the 8-bit binary code is fulfilled with the parity system entered, the program will display the number of 1 inside the binary code and tell the user about the number type as well as informing the user there is no error. For example, if the user enters “even” as the parity system and 11110000 as the binary code, the program will display “The number of 1s is 4 which is an even number.” and “There is no error.” which telling the user the number of 1 inside the binary code and the number type as well as the occurrence of the error. If the 8-bit binary code is not fulfilled with the parity system entered, the program will display the number of 1 inside the binary code and tell the user about the number type as well as informing the user there is an error. For instance, if the user enters “even” as the parity system and 11100000 as the binary code, the program will display “The number of 1s is 3 which is an odd number.” and “There is an error.” which telling the user the number of 1 inside the binary code and the number type as well as the occurrence of the error. After that, the program will prompt the user to enter the binary code again until the binary code is fulfilled with the parity system entered. Thus, this trial-and-error program will help the user more understand the parity system and the satisfied binary code. The second functionality is 8-bit signed binary numbers' systems converter. When this function is running, the user is required to enter a decimal number that is between -127 to 127. After that, the program will show the decimal number in binary form for three system, which are sign and magnitude system, 1's complement system and 2's complement system. Therefore, the user can convert the decimal number to three binary system through this program without doing calculation themselves. This will reduce time usage and increase the efficiency when dealing with this conversion. The user also can use this program to verify their answer when they doing conversion themselves by using calculation. Besides, the program provides clear instructions to the users to guide them to use the program. Every statement of the program is clear so that the users know what to do and would not key in invalid inputs. Moreover, the users can understand the purpose of the program as well as make good use of it. Other than that, the program consists of a complete error checking system. For every invalid input entered, the program will display invalid input and prompt the user to key in again. The program also allows the users to choose if they want to continue a function. If the user wants to continue, they only need to input “yes” and the particular function will be run again. The program also allows the user to select the function again, so that they can try two functions before the program end. Another specialty of the program is the program apply functions to modularize the code so that the code can be more readable. For each function has its own purpose and task to do so that the program will be tidier and can be easily understanded.

For the limitation of the program, one of it is the program do not specify the reason of why an input is invalid. For instance, if the user enters an 8-bit binary code that is more than 8-bit, the program will just show invalid input and never tell the user the code is out of range which is more than 8-bit. Besides, there is no error checking for the continue of a particular function and select again the function features. The two features will only be executed if the user input “yes”. For other inputs as well as “no”, the features will not be

executed. Therefore, if a user wants to continue or select again the function but he accidentally keys in a wrong input like “yes/” which is a typo error, the features would not be executed. Furthermore, the 8-bit signed binary numbers' systems converter can only convert a decimal number between -127 to 127. Therefore, the converter cannot be used to convert the decimal number which is out of the range.

Self-review and future improvement

The overall performance of the program is user-friendly and well-structured and the objectives of the program has been achieved. For the code organisation and readability, the code is organised into functions. This is good for maintainability and readability. Besides, the variable names and descriptive function in the code make it easier to be understanded. Moreover, the consistent indentation of the code provides a good readability. For the user interface part, the program provides clear instructions to the users. The program also provides clear prompts to the user. Thus, the program is user-friendly. The program also handles invalid inputs by prompting them to key in a valid input. Other than that, there is input validation in the program in order to make sure the users give valid inputs for the two functionalities. The reusability of the program is good because functions are used in the program. Thus, the code will be more modular and reusable. Code documentation is good since the code comments is added to explain the task of a particular section.

To improve the program, more informative error messages need to be added to specify the reason of why an input is invalid. Besides, error checking system need to be added for the continue of a particular function and select again the function features. Furthermore, the 8-bit signed binary numbers' systems converter can be improved by increasing the range of a decimal number that can be converted so that the program can convert more decimal number. On the other hand, the code needs to be reconstructed to reduce redundancy and verbosity.

Logic Gate Related issue:

(i) Truth Table generation

Truth Table generator:

Truth table generator is a versatile and user-friendly program that empowers users to generate truth tables for seven essential logic gates: AND, OR, NOT, NAND, NOR, XOR, and XNOR. The program allows users to define the number of input variables, and it automatically generates the corresponding truth table, along with a single output column. Whether you're a student learning about digital logic or a professional designing complex circuit, truth table generator simplifies the process of understanding and working with logic gates.

Flowchart/Structure chart

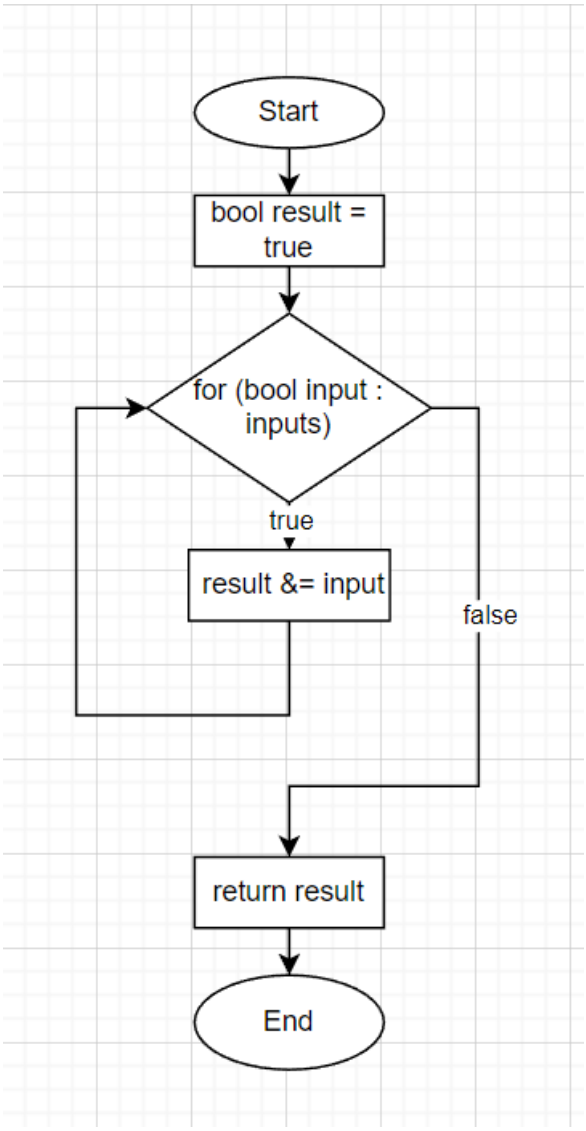


Figure 3.1: Flowchart for function to perform logical AND operation.

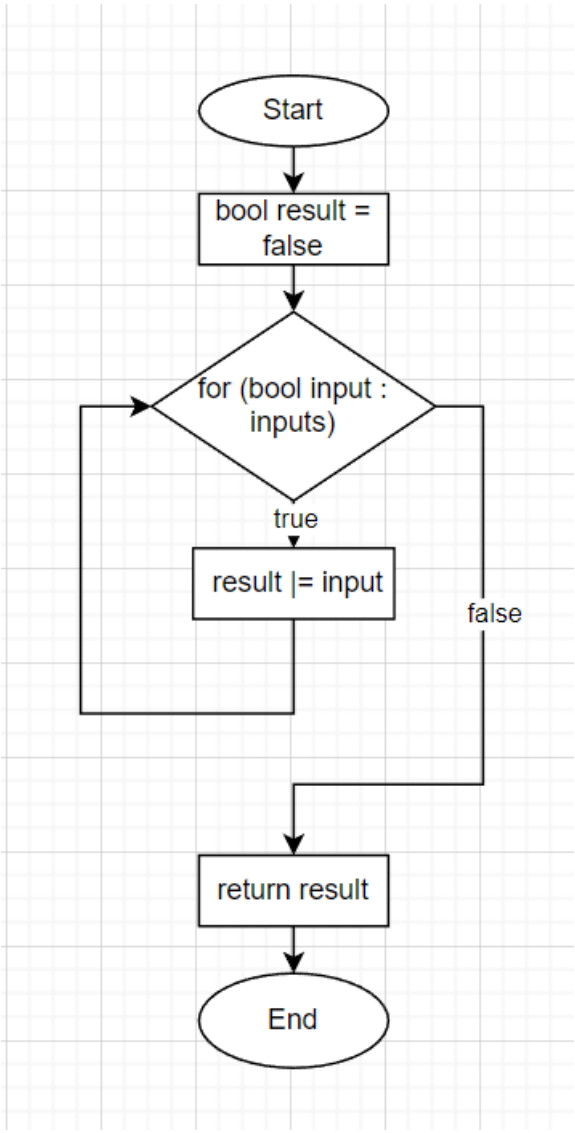


Figure 3.2: Flowchart for function to perform logical OR operation.

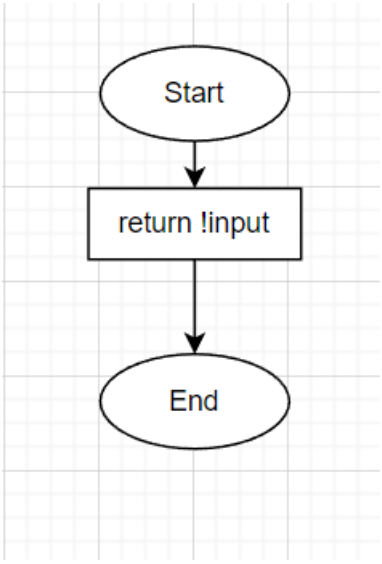


Figure 3.3: Flowchart for function to perform logical NOT operation.

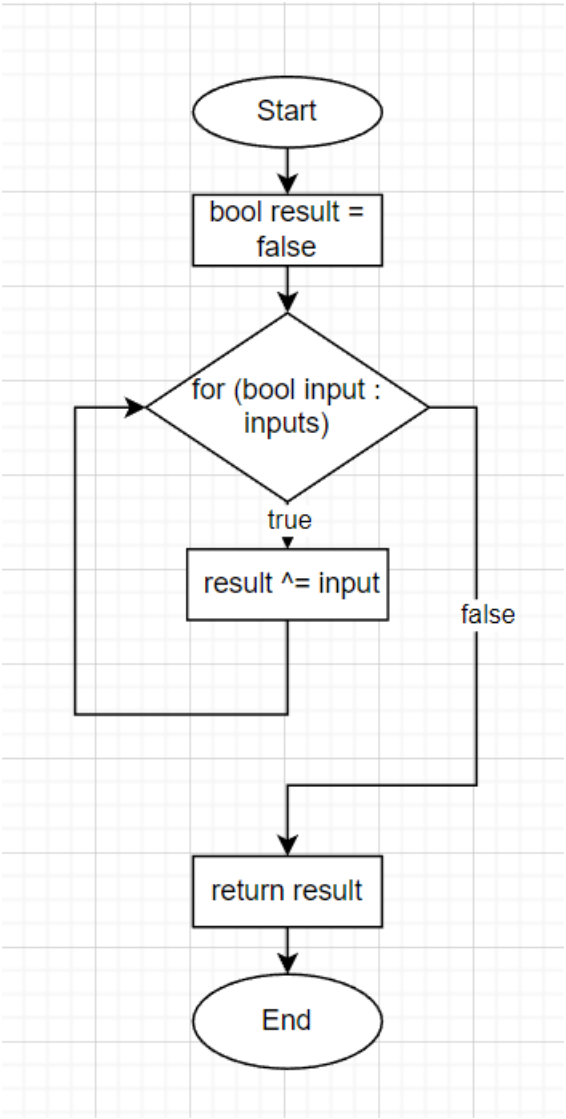


Figure 3.4: Flowchart for function to perform logical XOR operation.

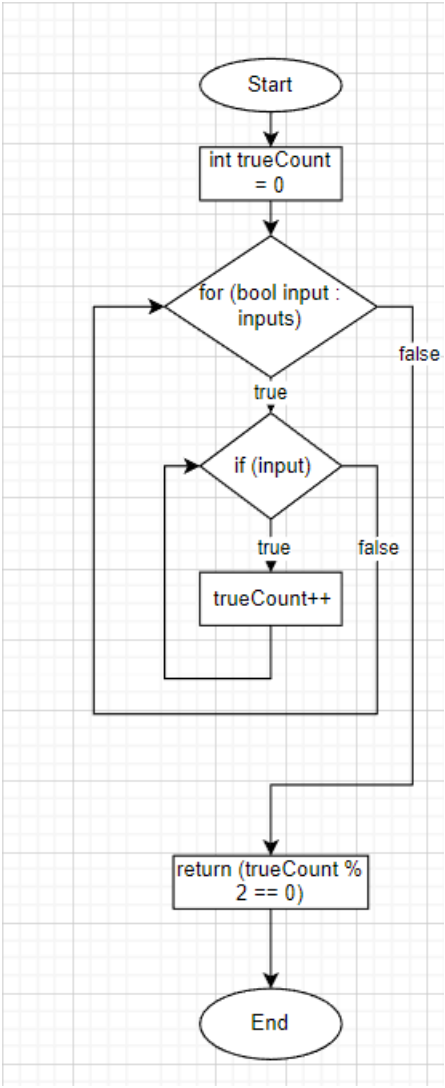


Figure 3.5: Flowchart for function to perform logical XNOR operation.

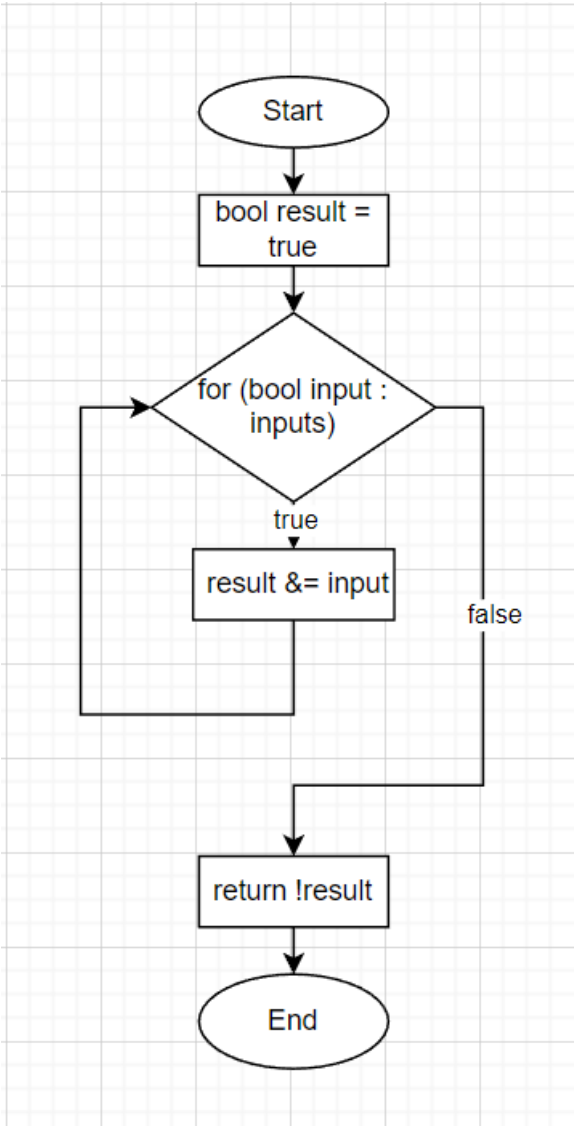


Figure 3.6: Flowchart for function to perform logical NAND operation.

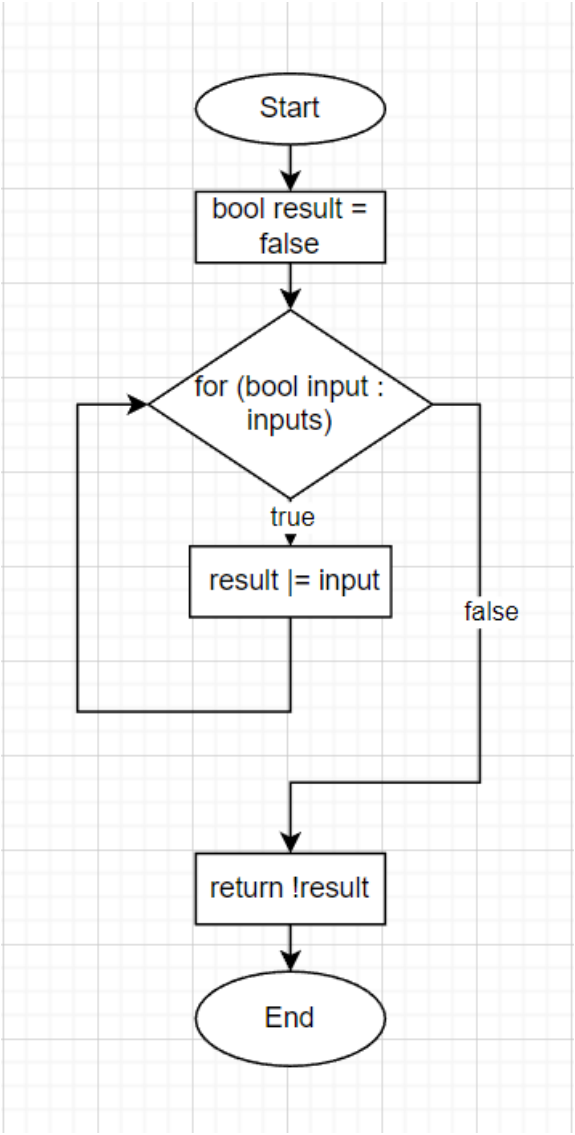


Figure 3.7: Flowchart for function to perform logical NOR operation.

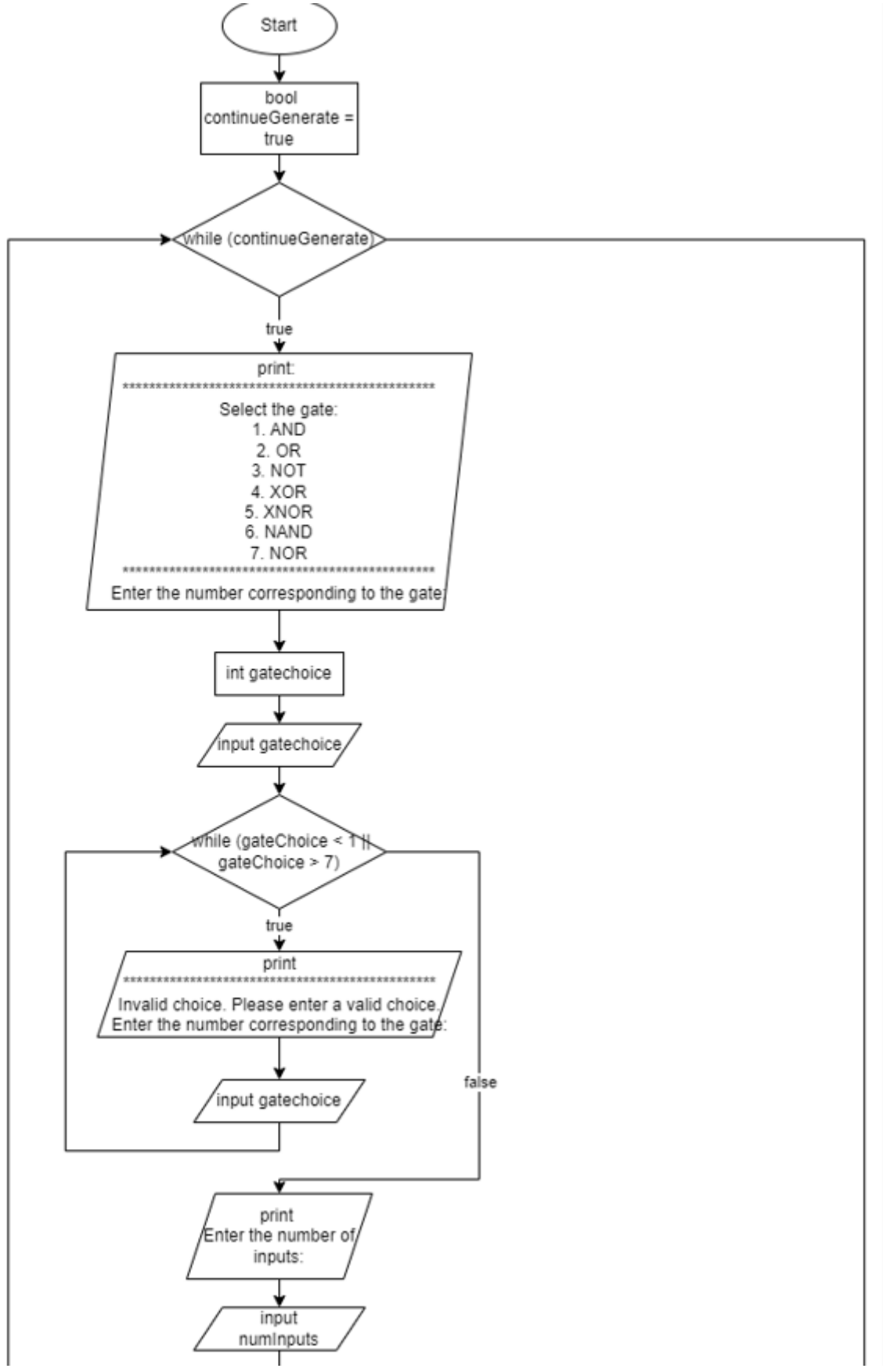


Figure 3.8: Flowchart for Main function of truth table generator. (Part1)

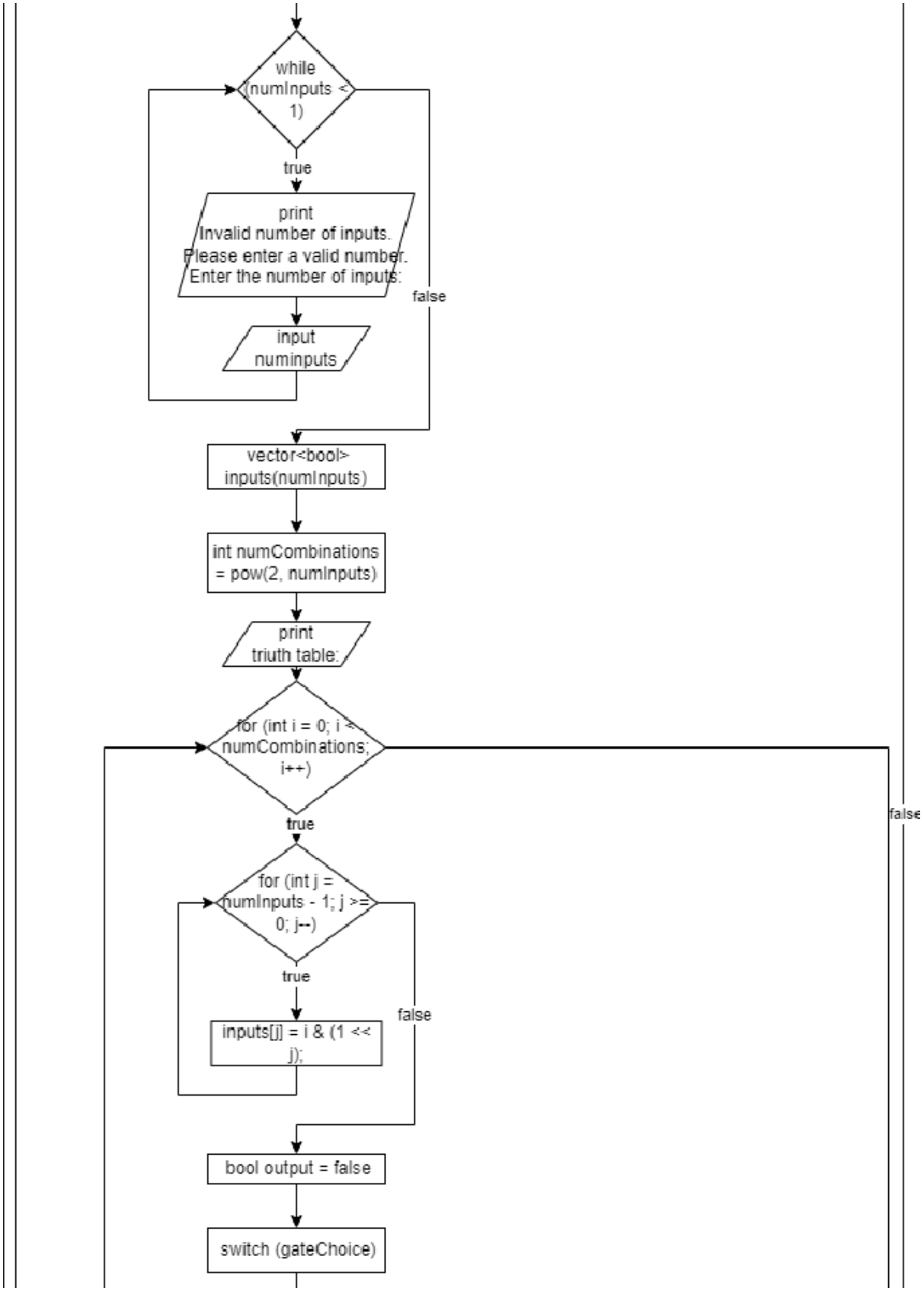


Figure 3.8(b): Flowchart for Main function of truth table generator. (Part2)

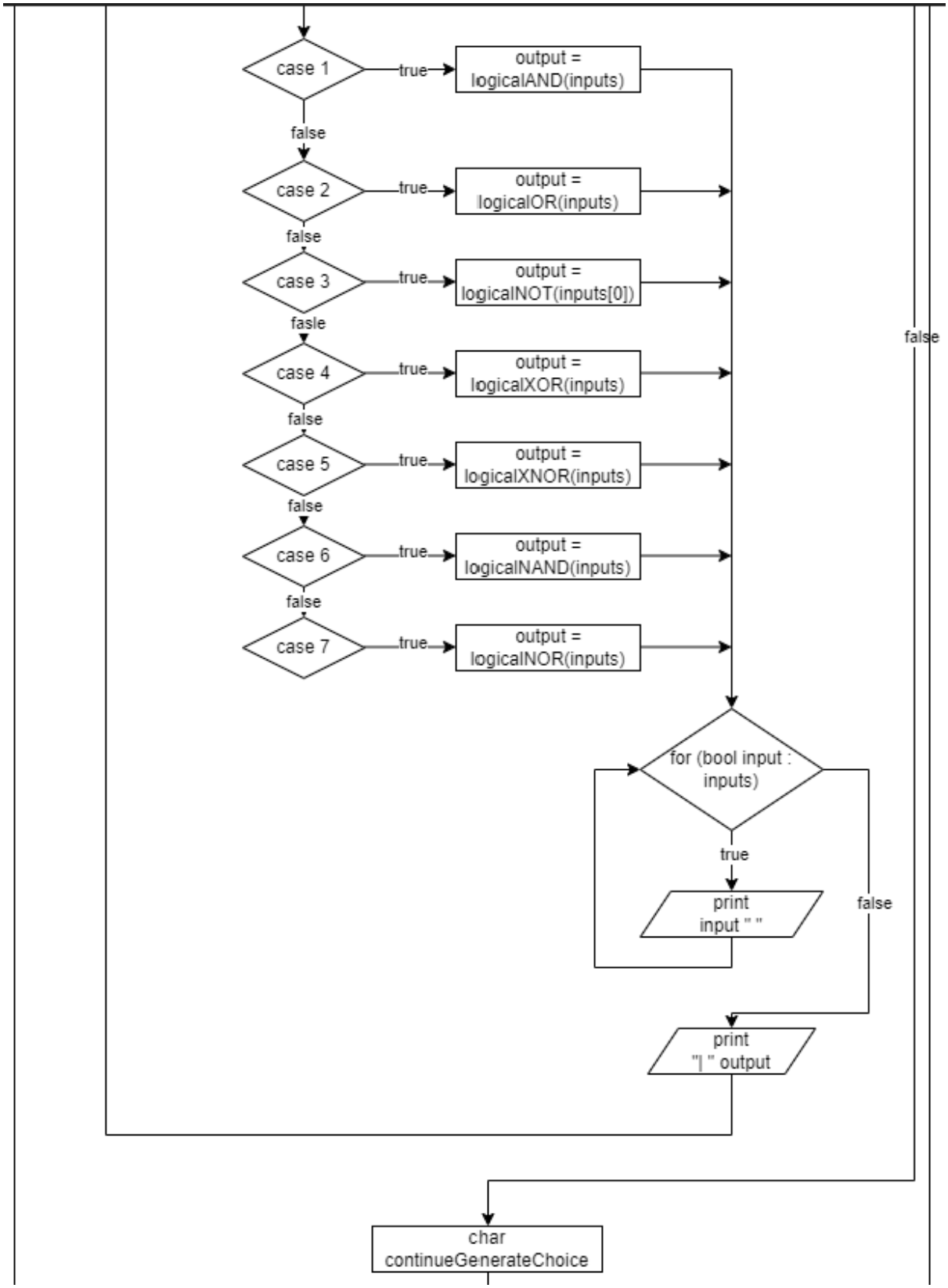


Figure 3.8(c): Flowchart for Main function of truth table generator. (Part3)

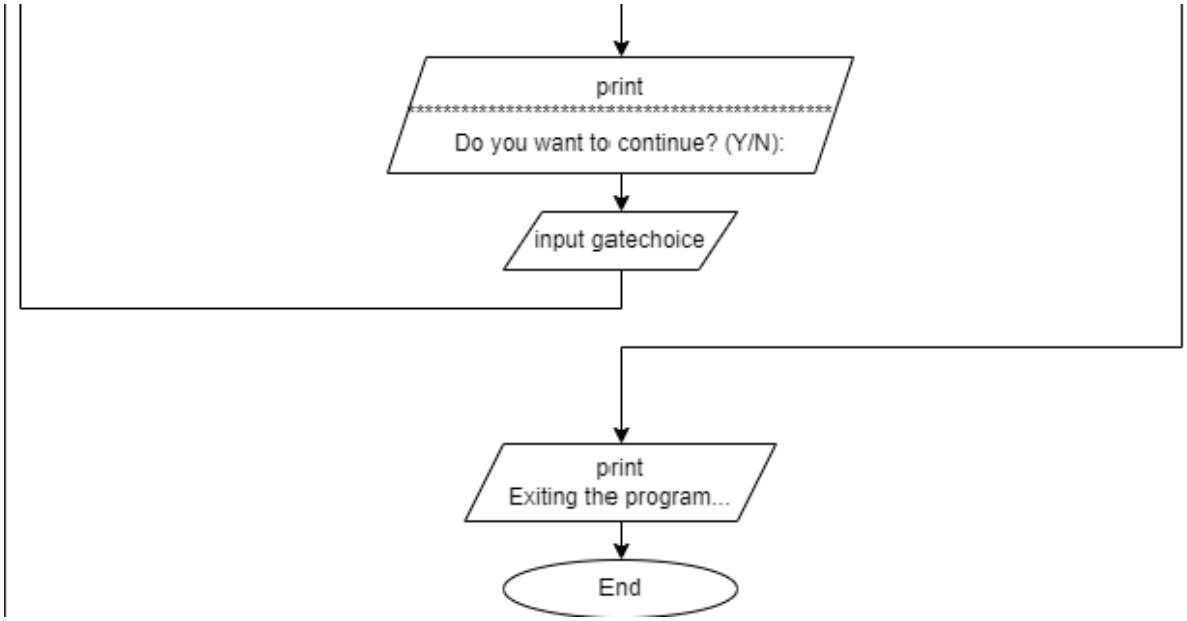


Figure 3.8(d): Flowchart for Main function of truth table generator. (Part4)

Screen shots of the program

```
// Function to perform logical AND operation
bool logicalAND(const vector<bool>& inputs)
{
    bool result = true;
    for (bool input : inputs)
    {
        result &= input;
    }
    return result;
}
```

Figure 3.9: Function to perform logical And operation.

```
// Function to perform logical OR operation
bool logicalOR(const vector<bool>& inputs)
{
    bool result = false;
    for (bool input : inputs)
    {
        result |= input;
    }
    return result;
}
```

Figure 3.10: Function to perform logical OR operation.

```cpp
// Function to perform logical NOT operation
bool logicalNOT(bool input)
{
    return !input;
}
```

Figure 3.11: Function to perform logical NOT operation.

```cpp
// Function to perform logical XOR operation
bool logicalXOR(const vector<bool>& inputs)
{
    bool result = false;
    for (bool input : inputs)
    {
        result ^= input;
    }
    return result;
}
```

Figure 3.12: Function to perform logical XOR operation.

```cpp
// Function to perform logical XNOR operation
bool logicalXNOR(const vector<bool>& inputs)
{
    int trueCount = 0;
    for (bool input : inputs)
    {
        if (input)
            trueCount++;
    }
    return (trueCount % 2 == 0);
}
```

Figure 3.13: Function to perform logical XNOR operation.

```cpp
// Function to perform logical NAND operation
bool logicalNAND(const vector<bool>& inputs)
{
    bool result = true;
    for (bool input : inputs)
    {
        result &= input;
    }
    return !result;
}
```

Figure 3.14: Function to perform logical NAND operation.

```cpp
// Function to perform logical NOR operation
bool logicalNOR(const vector<bool>& inputs)
{
    bool result = false;
    for (bool input : inputs)
    {
        result |= input;
    }
    return !result;
}
```

Figure 3.15: Function to perform logical NOR operation.

```cpp
int main()
{
    bool continueGenerate = true;
    while (continueGenerate)
    {
        cout << "*********************************************" << endl;
        cout << "Select the gate: " << endl;
        cout << "1. AND" << endl;
        cout << "2. OR" << endl;
        cout << "3. NOT" << endl;
        cout << "4. XOR" << endl;
        cout << "5. XNOR" << endl;
        cout << "6. NAND" << endl;
        cout << "7. NOR" << endl;
        cout << "*********************************************" << endl;
        cout << "Enter the number corresponding to the gate: ";
        int gateChoice;
        cin >> gateChoice;
        cout << "*********************************************" << endl;

        while (gateChoice < 1 || gateChoice > 7)
        {
            cout << "Invalid choice. Please enter a valid choice." << endl;
            cout<<"Enter the number corresponding to the gate: ";
            cin >> gateChoice;
            cout << "*********************************************" << endl;
        }

        cout << "Enter the number of inputs: ";
        int numInputs;
        cin >> numInputs;

        while (numInputs < 1)
        {
            cout << "Invalid number of inputs. Please enter a valid number." << endl;
            cout << "Enter the number of inputs: ";
            cin >> numInputs;
```

Figure 3.16: Main function for truth table generator (Part1).

```
        int numOutputs = 1;
        vector<bool> inputs(numInputs);
        int numCombinations = pow(2, numInputs);

        cout << "Truth Table:" << endl;
        for (int i = 0; i < numCombinations; i++)
        {
            // Generate all possible input combinations
            for (int j = numInputs - 1; j >= 0; j--)
            {
                inputs[j] = i & (1 << j);
            }

            bool output = false;
            switch (gateChoice)
            {
            case 1:
                output = logicalAND(inputs);
                break;
            case 2:
                output = logicalOR(inputs);
                break;
            case 3:
                output = logicalNOT(inputs[0]);
                break;
            case 4:
                output = logicalXOR(inputs);
                break;
            case 5:
                output = logicalXNOR(inputs);
                break;
            case 6:
                output = logicalNAND(inputs);
                break;
            case 7:
                output = logicalNOR(inputs);
                break;
```

Figure 3.17: Main function for truth table generator (Part 2)

```
            }

            // Print input and output values for this combination
            for (bool input : inputs)
            {
                cout << input << " ";
            }
            cout << "| " << output << endl;
        }

        char continueGenerateChoice;
        cout << "*****************************************" << endl;
        cout << "Do you want to continue? (Y/N): ";
        cin >> continueGenerateChoice;
        if (toupper(continueGenerateChoice) != 'Y')
        {
            break; // Exit the loop if the user does not want to continue
        }
    }
    cout << "Exiting the program.......";
    return 0;
}
```

Figure 3.18: Main function for truth table generator (Part 3)

Test plans and results

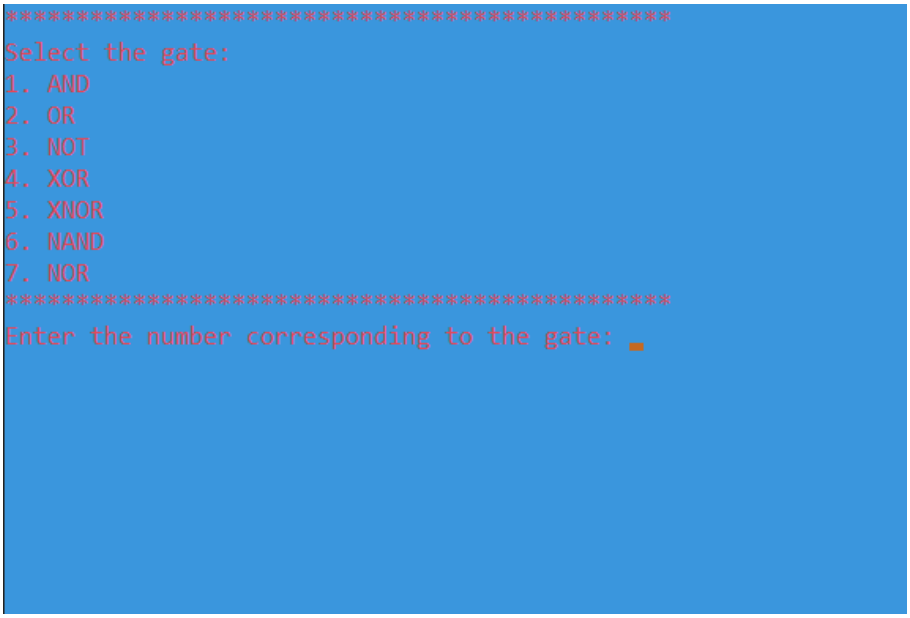


Figure 3.19: User interface for truth table generator.

First the program will show which logic gate is including in the program. The program will require the user to enter the type of logic gate they choose to generate truth table. In this case, the program only accepts 1 – 7 as input, 1 stands for AND gate, 2 stands for OR gate, 3 stands for NOT gate, 4 stands for XOR gate, 5 stands for XNOR gate. 6 stands for NAND gate and 7 stands for NOR gate. If the input is out of range from 1 – 7, the program will show error message and require the user to re-enter the valid input.

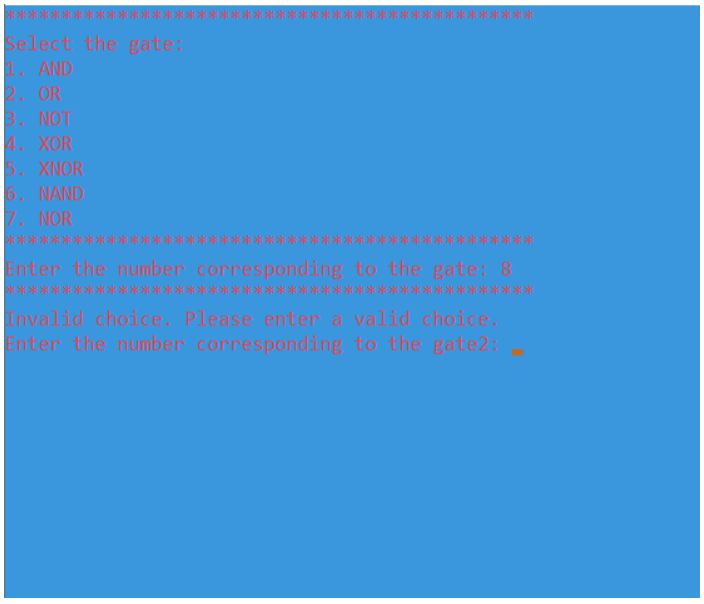


Figure 3.20: Error message for invalid input

Once users enter a correct input, the program will request users to enter the total number of input that user would like use to generate one input.

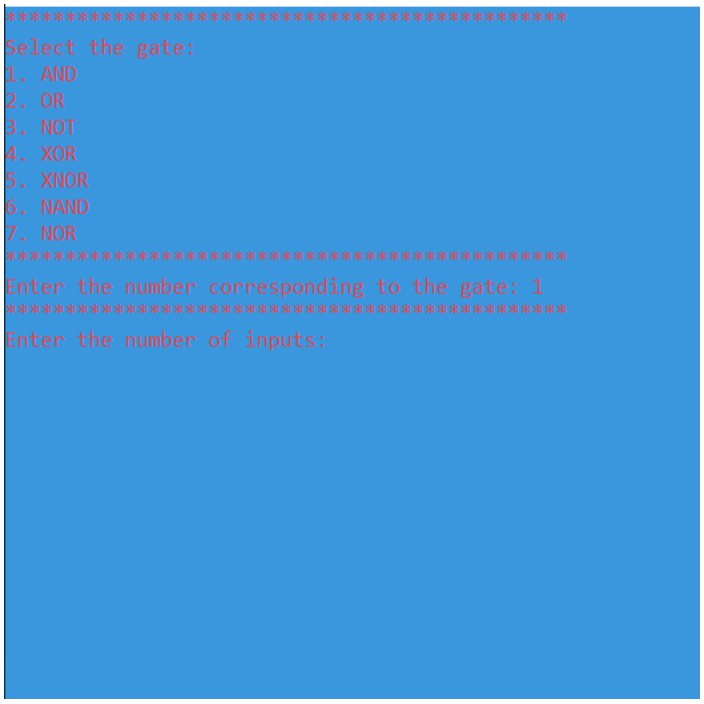


Figure 3.21: Input number requirement

As user enter an input number. The program will generate a truth table with different row and column. The number of rows is decided by the input number. The number of rows is followed by the equation of input number + 1. The number of columns is decided by the equation of 2 in the power of input number (2*input number). For example, if the user entered number of inputs as 4, the program would generate a truth table of 5 rows x 16 columns.

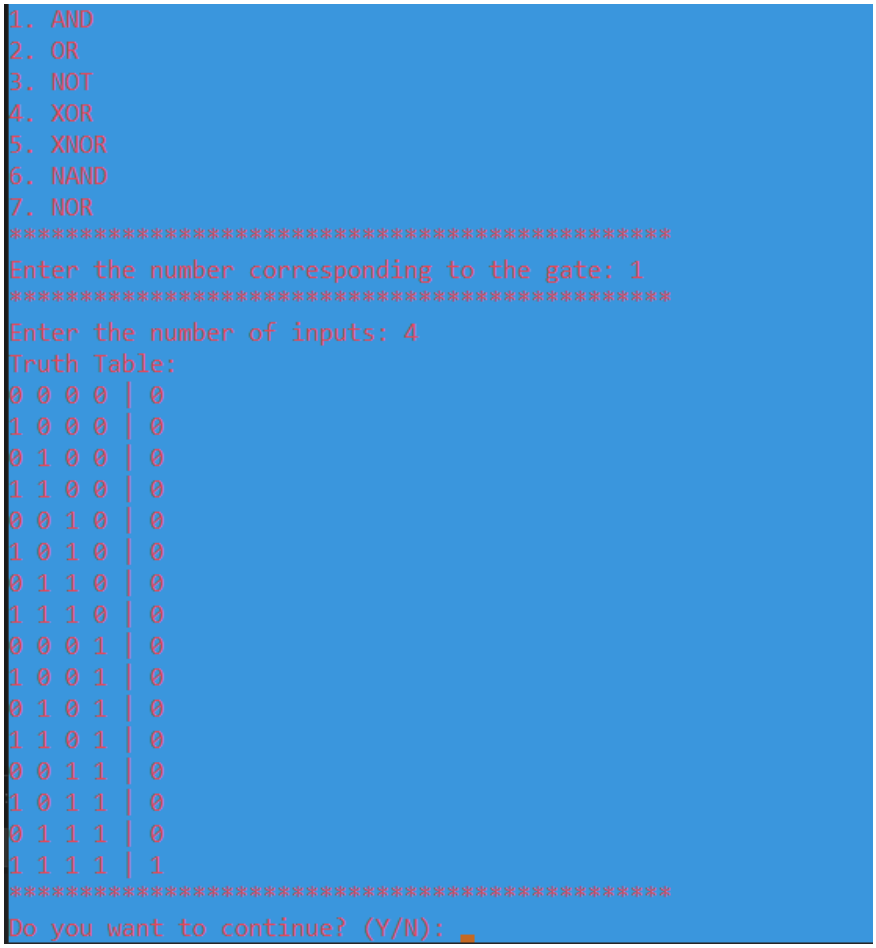


Figure 3.22: Truth table generated.

The program will then ask the user would they like to continue using truth table generator program. The program will bring user back to Input logic gate requirements (Figure 4.1) if user choose yes for continue, else the program will return user back to main menu if user choose no.

Specialty and limitation of the program

Truth table generator offers comprehensive support for seven fundamental logic gates, each serving a distinct logical function. The AND gate, for instance, produces a TRUE output (1) only when all input signals are TRUE (1), performing a logical "AND" operation. In contrast, the OR gate generates a TRUE output (1) when at least one input signal is TRUE (1), representing a logical "OR" operation. The NOT gate reverses its input's logic state, transforming TRUE (1) into FALSE (0) and vice versa. The NAND gate is the opposite of AND, producing TRUE (1) unless all inputs are TRUE (1), essentially performing a "NOT AND" operation. On the other hand, the NOR gate is the inverse of OR, yielding TRUE (1) only when none of its inputs are TRUE (1), executing a "NOT OR" operation. The XOR gate produces TRUE (1) when an odd number of inputs are TRUE (1), executing an exclusive OR operation. Lastly, the XNOR gate, also known as the equivalence gate, produces TRUE (1) when an even number of inputs are TRUE (1), effectively performing a "XOR followed by NOT" operation.

Customizability is another hallmark of Truth table generator. The program allows users to define the number of input variables for each gate. This adaptability ensures that the tool can

cater to a broad spectrum of applications, whether users are dealing with simple binary logic or more complex scenarios involving multiple inputs.

Furthermore, Truth table generator most notable features is its automated truth table generation. Once users specify the number of inputs and select a logic gate, the program calculates the corresponding truth table. It achieves this by applying the logical function associated with each gate to the input values, ensuring that the output column is generated accurately and reliably. This automation streamlines the process of understanding and working with logic gates.

As for the limitation, while Truth table generator excels in its support for fundamental logic gates, it has certain limitations. Primarily, it focuses on these basic gates, excluding more advanced or specialized logic gates that may be required for complex digital circuit design. Users with specific requirements beyond the seven fundamental gates may need to seek alternative software solutions. The program also offers a simplified representation of logic gates, considering only binary inputs (0 and 1). This simplicity means it does not support variable inputs or complex logical expressions, which can be limiting for users working on advanced applications that require more nuanced logic operations. Additionally, Truth table generator generates truth tables but lacks real-time circuit simulation or visualization of logic circuits. Users seeking comprehensive circuit design and simulation capabilities may need to complement with other software tools designed for such purposes.

Self-review and future improvement

Reflecting on the development of Truth table generator, I can see its value in simplifying logic gate operations and truth table generation. However, it's essential to acknowledge both its strengths and areas for improvement. One of its notable strengths lies in its comprehensive support for seven fundamental logic gates, accurately calculating outputs based on user-defined inputs. Additionally, the user-friendly interface makes it accessible to a broad range of users, regardless of their technical background. The program's automated truth table generation stands out as a valuable feature, enabling users to quickly understand logical outcomes for various input combinations, making it an excellent educational and practical tool.

To enhance Truth table generator further, I plan to expand its logic gate repertoire to include more advanced and specialized gates, catering to complex digital circuit design scenarios. Enabling users to define custom logic expressions beyond binary inputs will be a significant improvement, accommodating more advanced applications requiring sophisticated logical operations. Additionally, adding real-time circuit simulation and visualization features will make the program even more valuable for users designing and testing digital circuits. Developing comprehensive educational resources, including tutorials, explanations, and examples within the program, will assist users, especially beginners, in grasping digital logic concepts more effectively. Moreover, actively seeking user feedback and implementing user-requested features or addressing issues will be a priority, fostering community involvement and ensuring continuous improvement based on user experiences. Providing more customization options for user-defined inputs, such as specifying logic levels beyond 0 and 1 or using variables, will increase the program's flexibility and adaptability.

ii. Logic Gate Output

Logic Gates

Flowchart

Figure 3.33 Main Frame

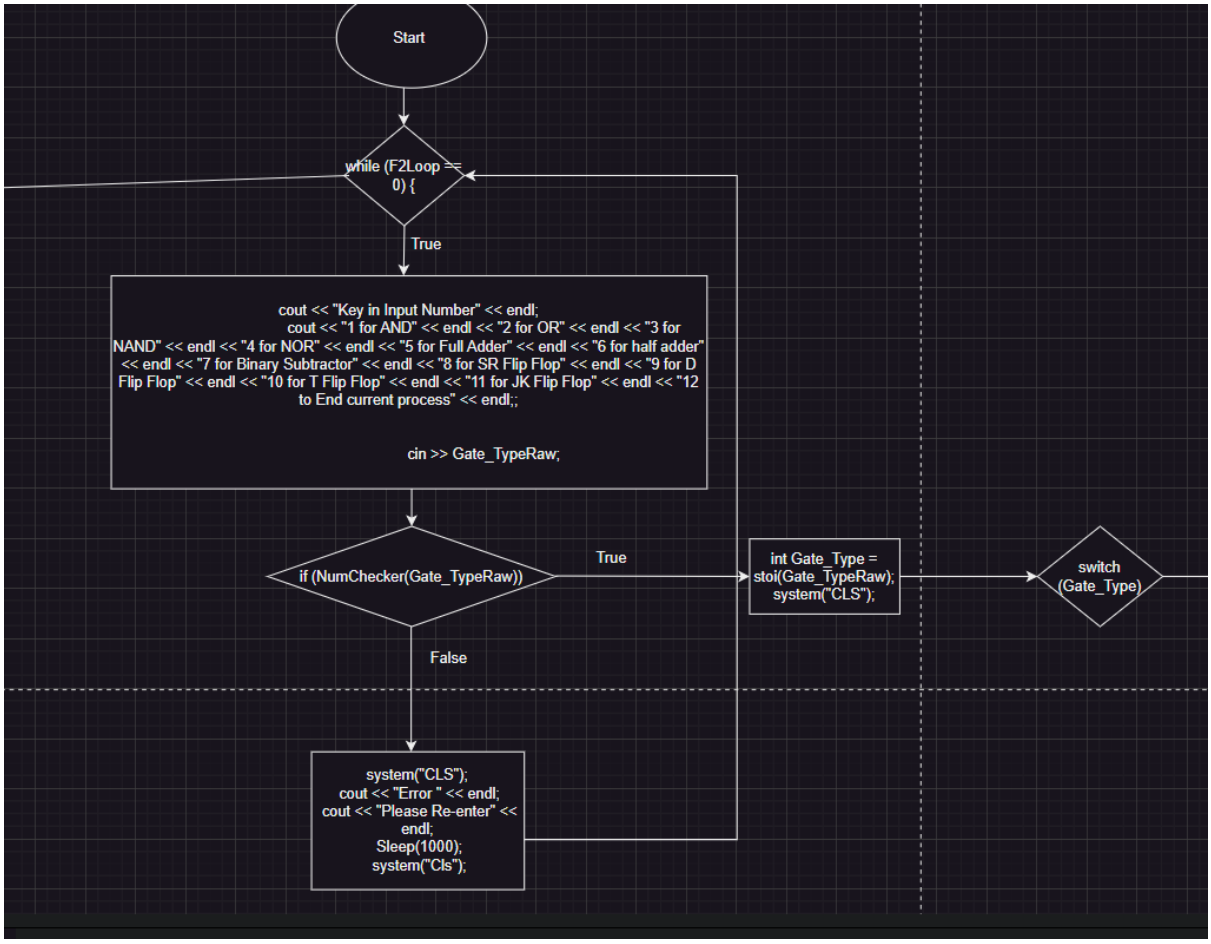


Figure 3.34 AND Gate

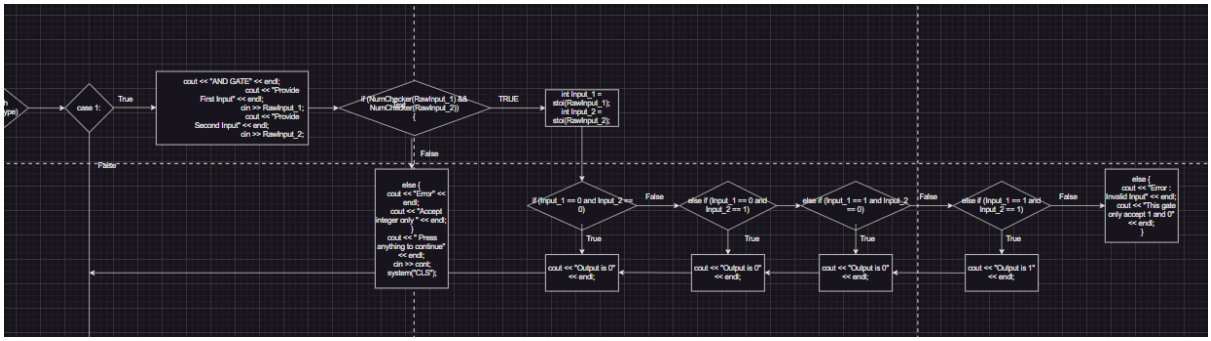


Figure 3.34 OR Gate Part 1

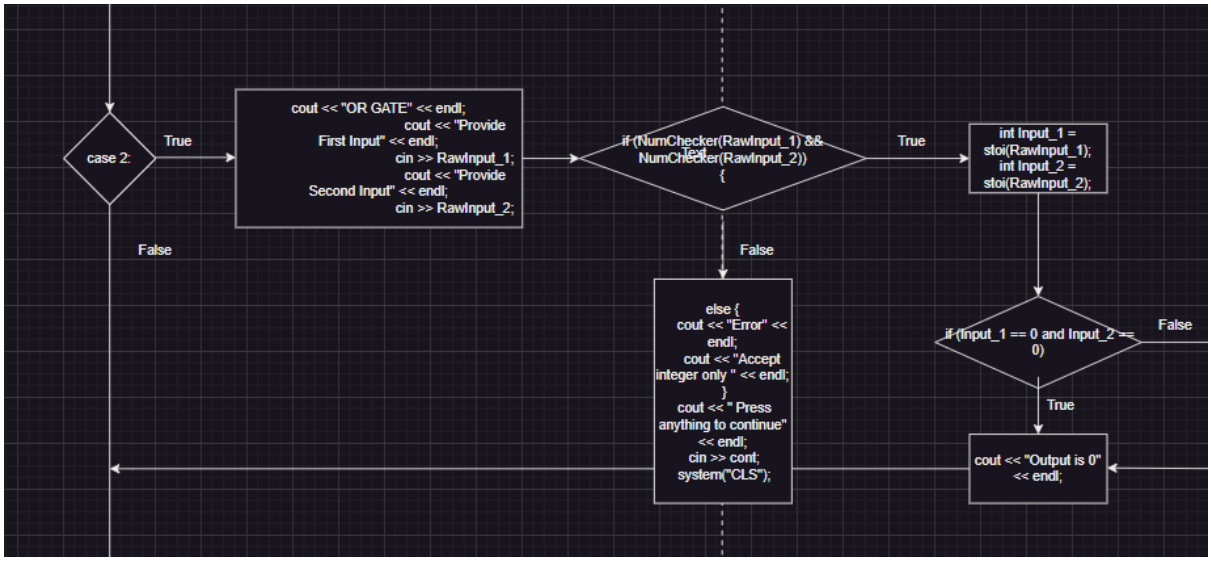


Figure 3.35 OR Gate Part 2

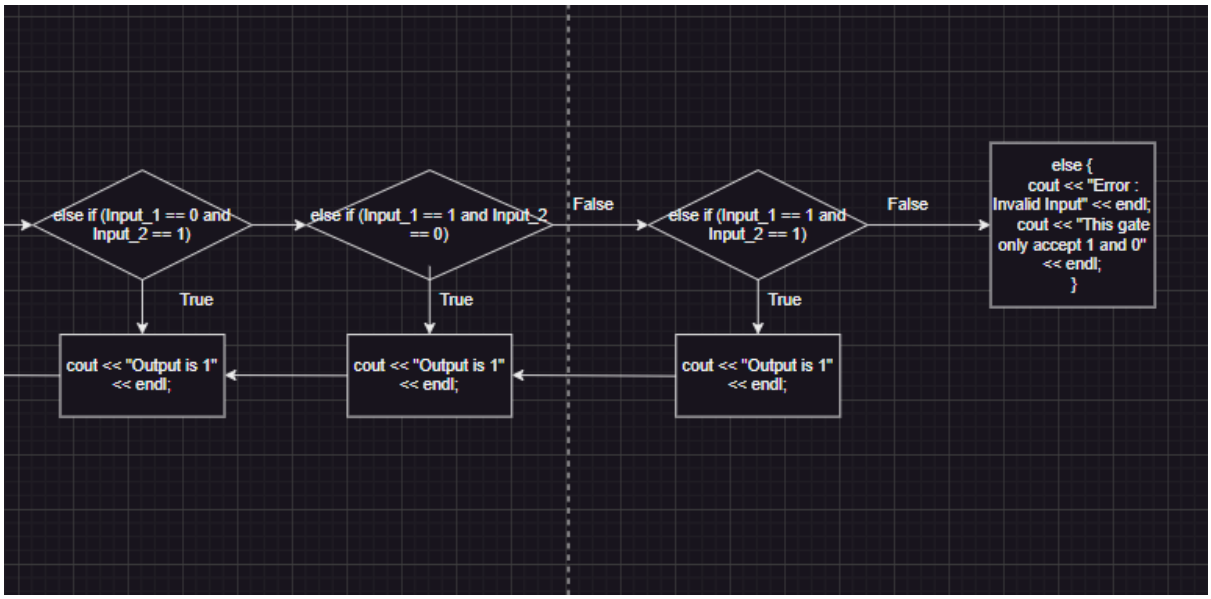


Figure 3.36 NAND Gate Part 1

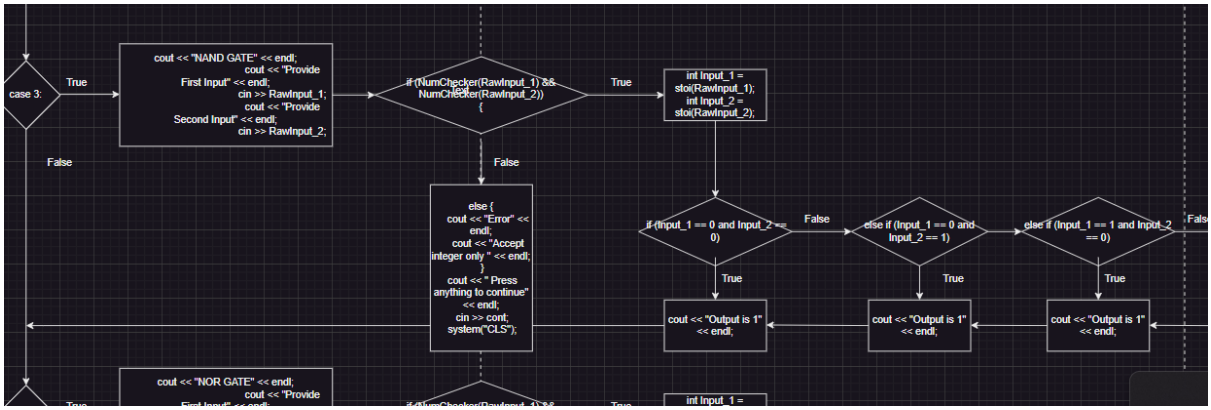


Figure 3.37 NAND Gate Part 2

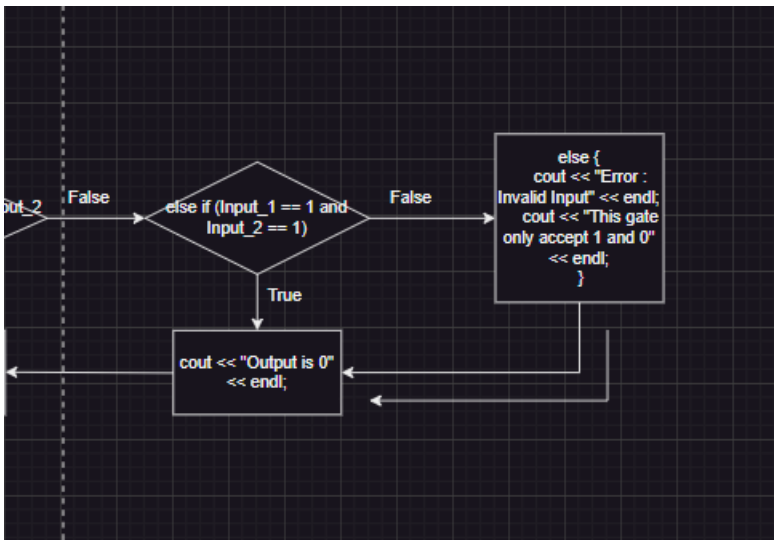


Figure 3.38 NOR Gate Part 1

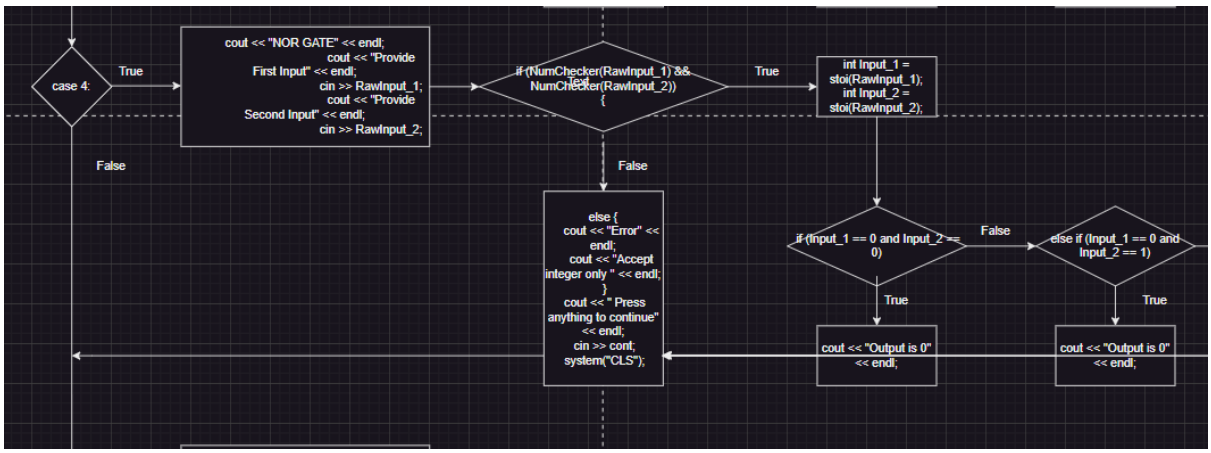


Figure 3.39 NOR Gate Part 2

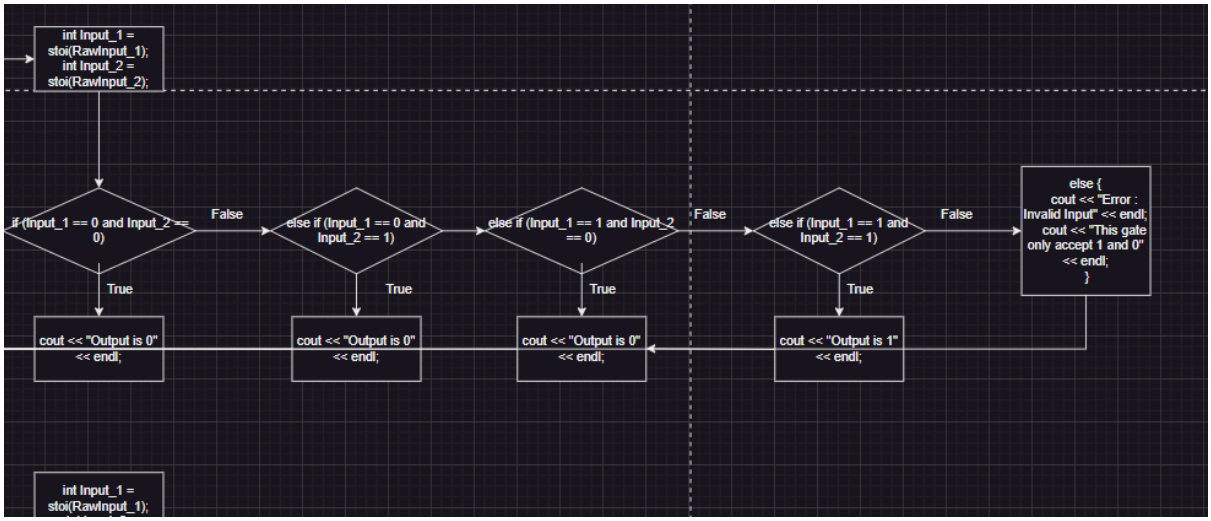


Figure 3.40 Full Adder Part 1

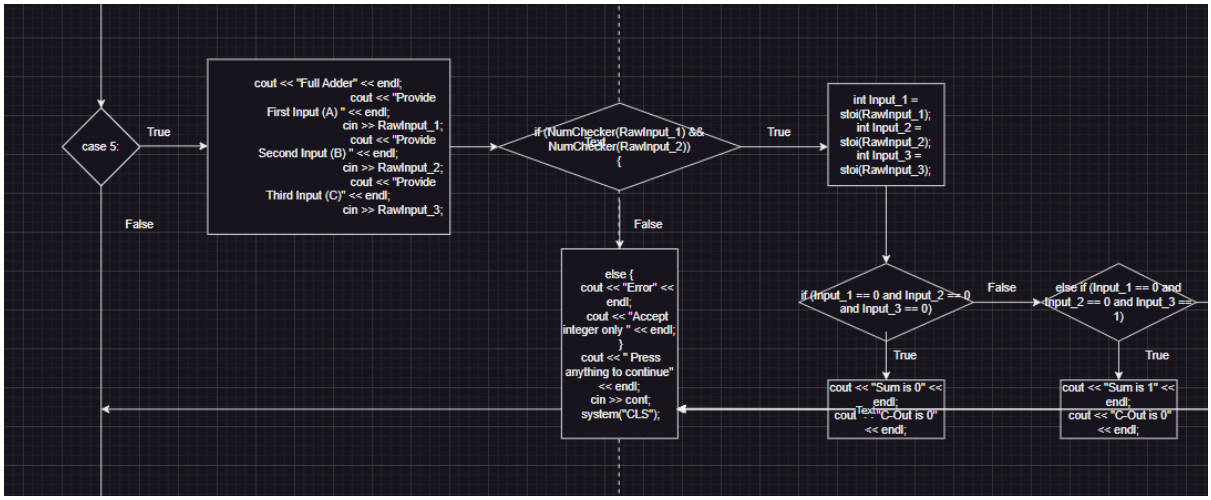


Figure 3.41 Full Adder Part 2

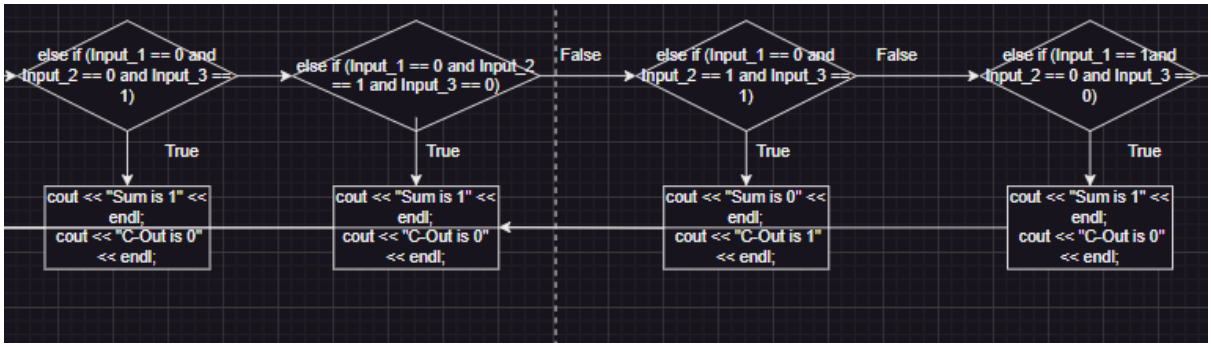


Figure 3.42 Full Adder Part 3

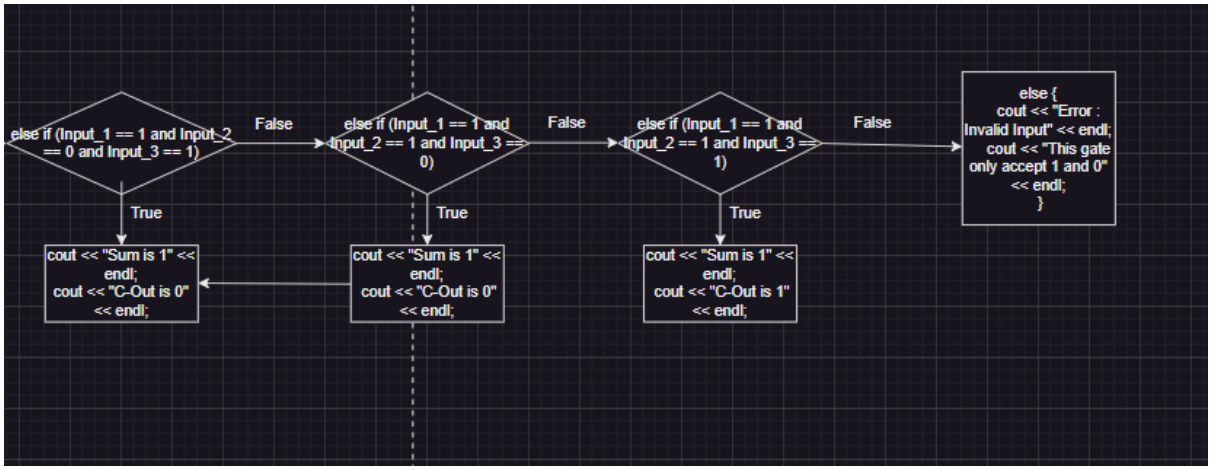


Figure 3.43 Half Adder Part 1

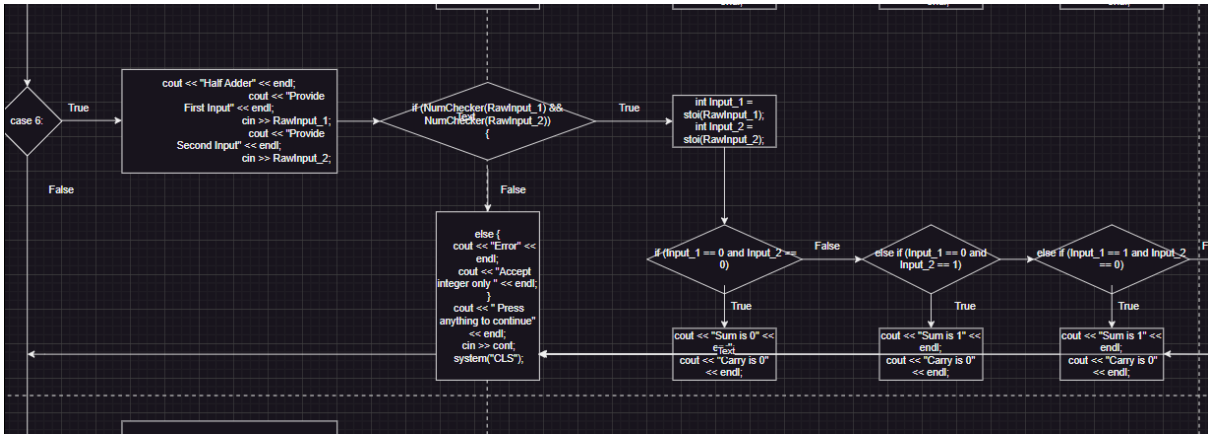


Figure 3.44 Half Adder Part 2

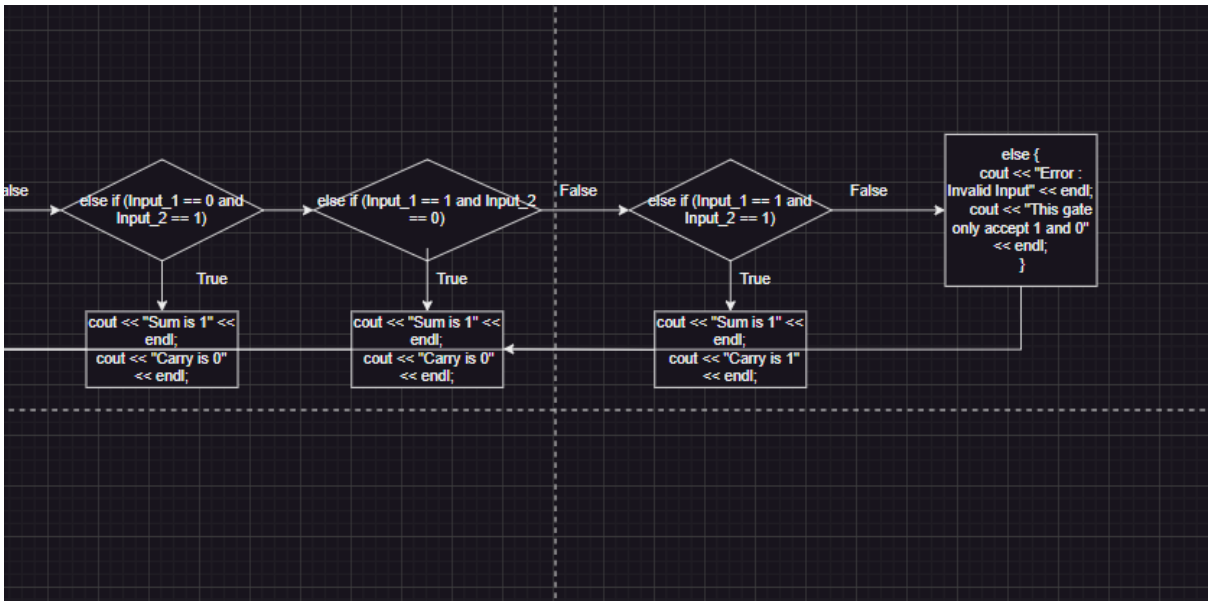


Figure 3.45 Binary Subtractor Part 1

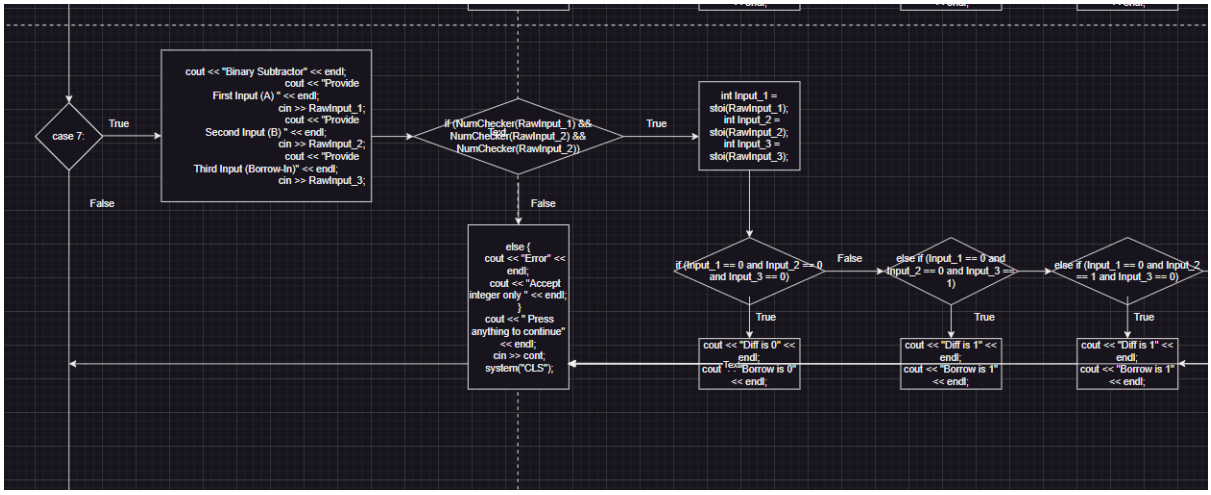


Figure 3.46 Binary Subtractor Part 2

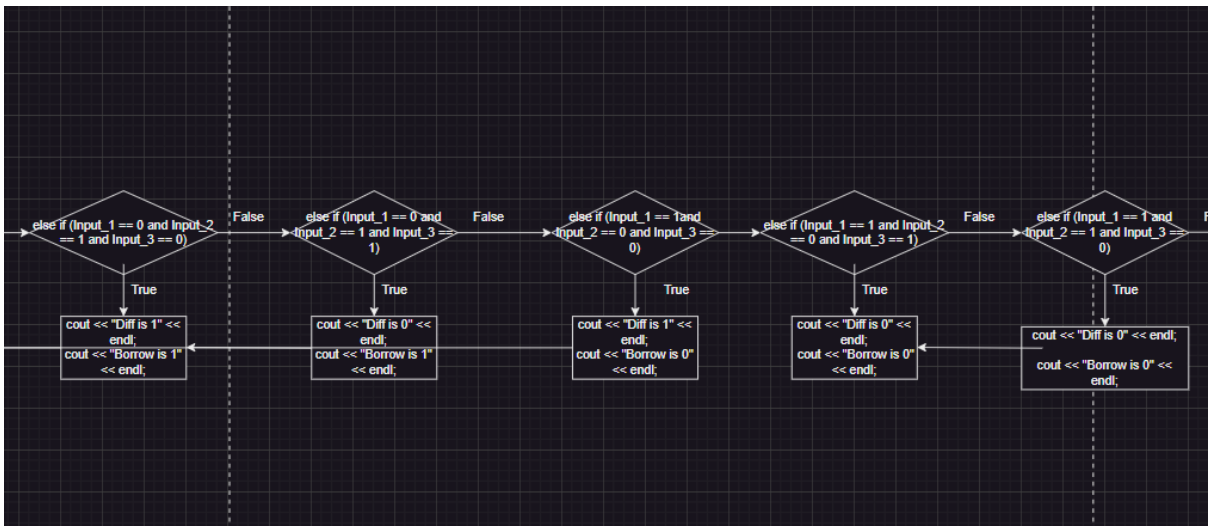


Figure 3.47 Binary Subtractor Part 3

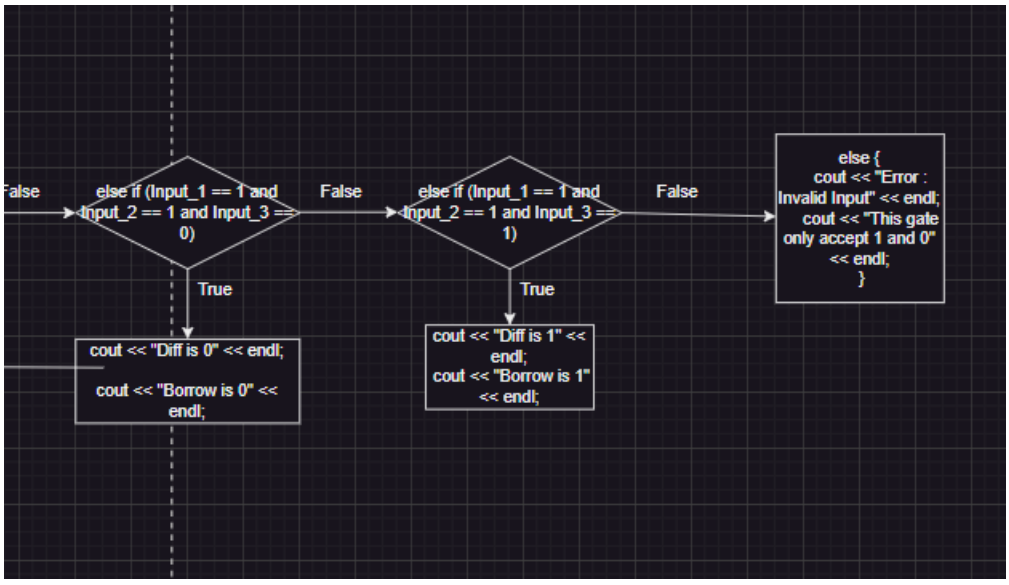


Figure 3.48 SR Flip Flop Part 1

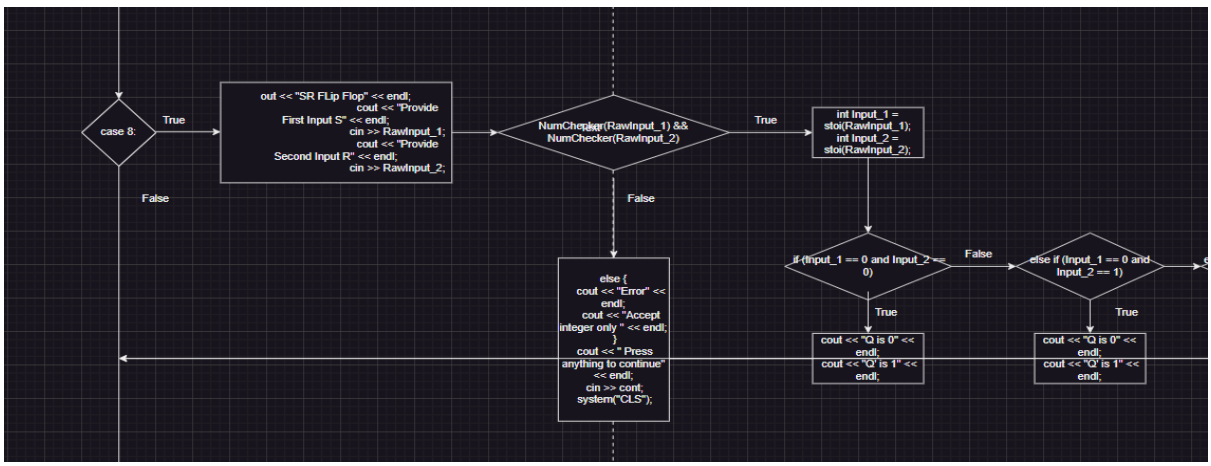


Figure 3.49 SR Flip Flop Part 2

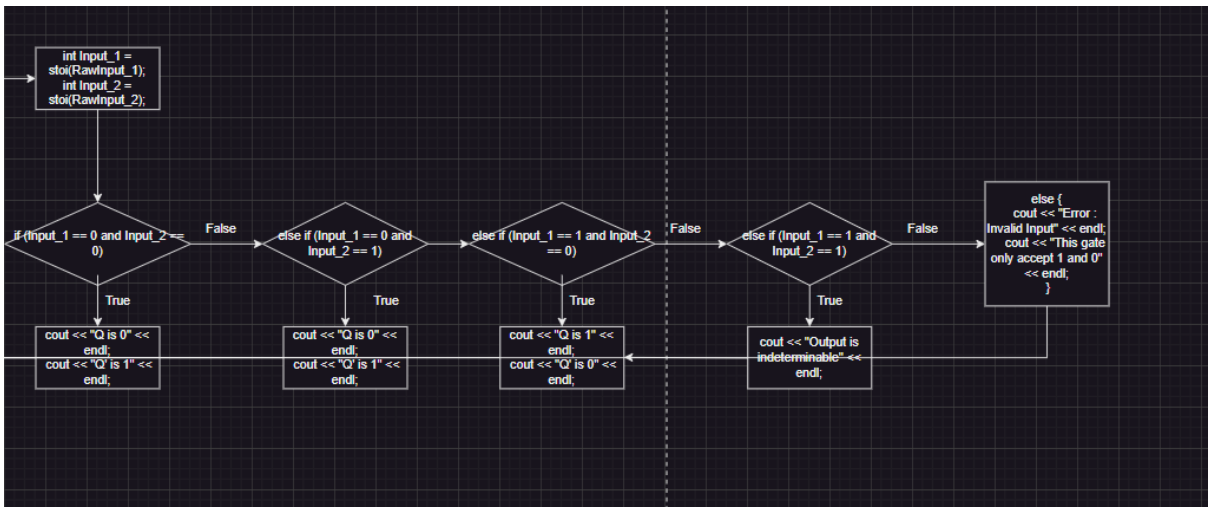


Figure 3.50 T-Flip Flop Part 1

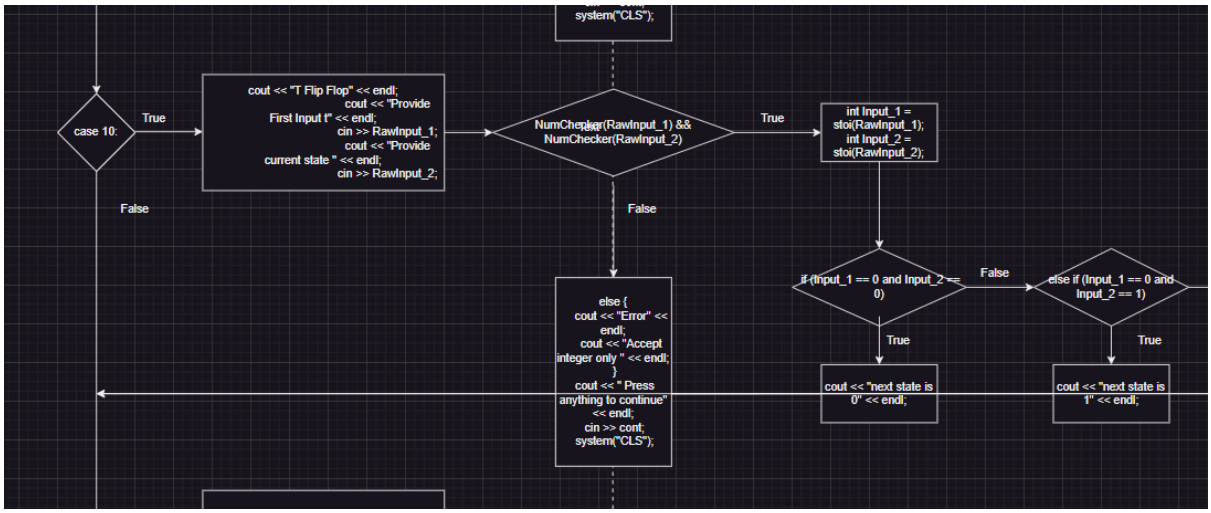


Figure 3.51 T Flip Flop Part 2

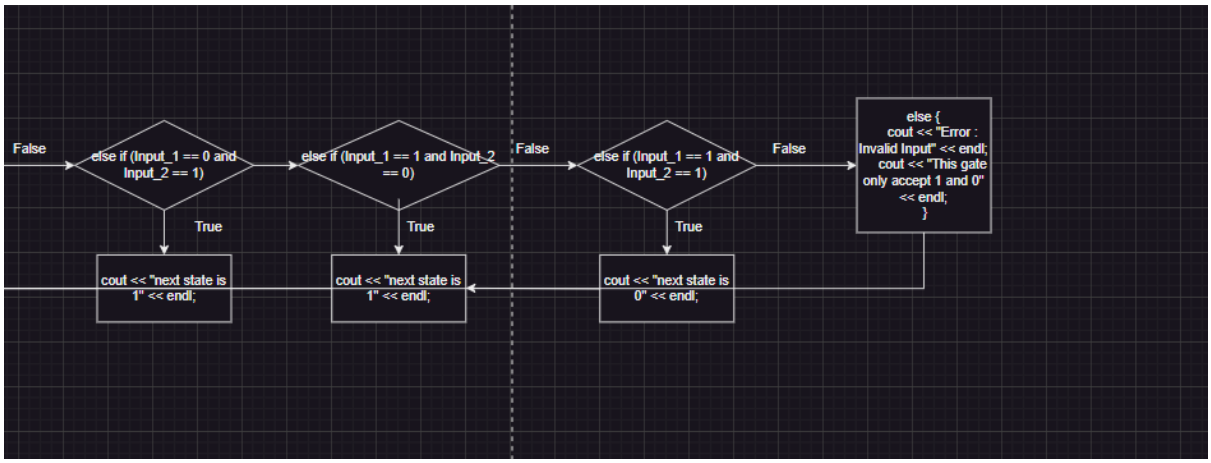


Figure 3.52 JK Flip Flop Part 1

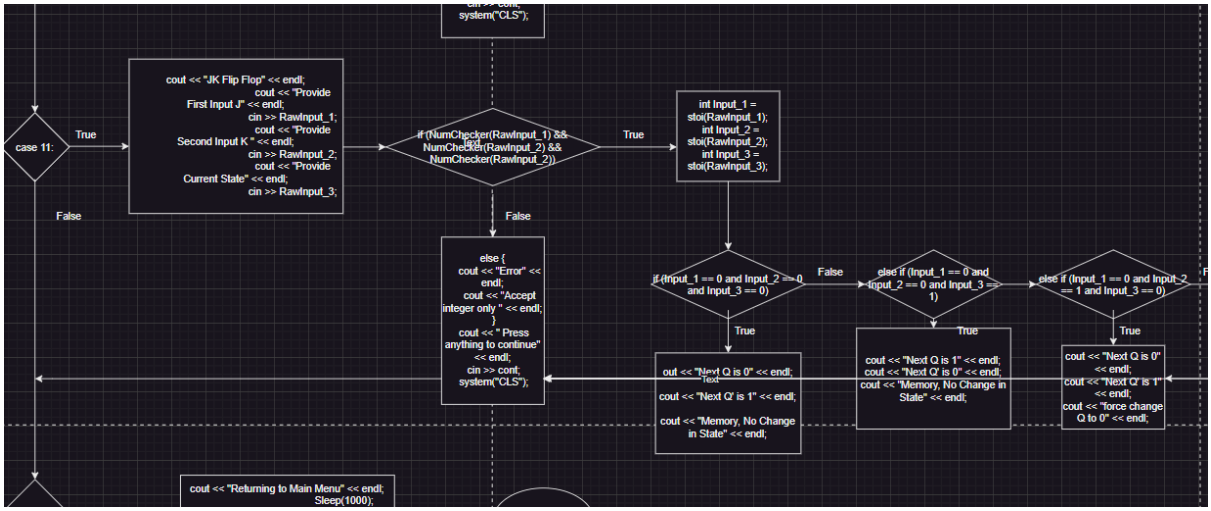


Figure 3.53 JK Flip Flop Part 2

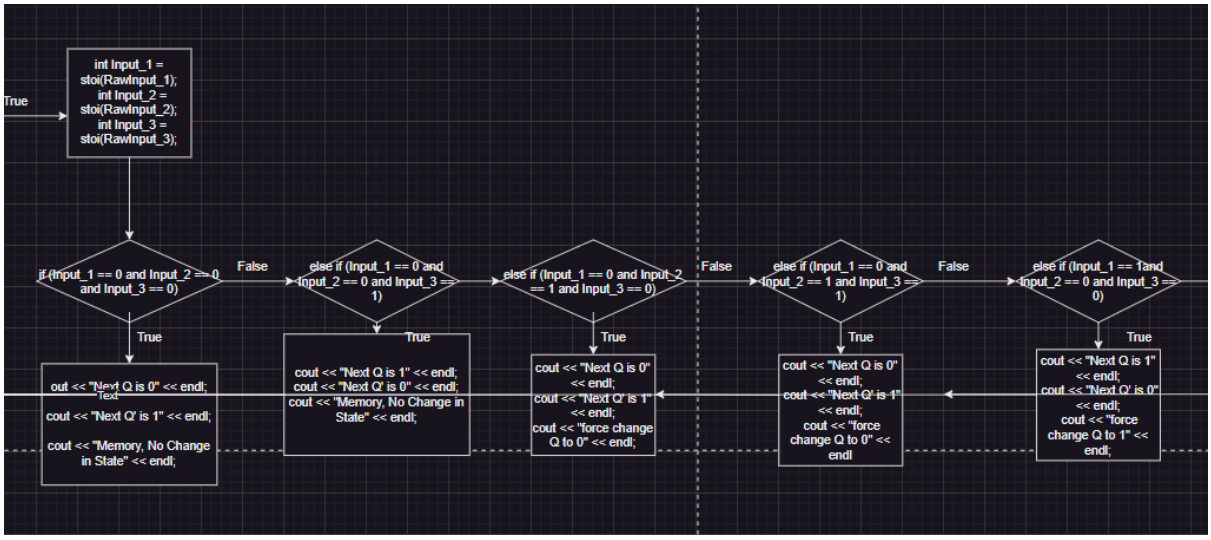


Figure 3.53 JK Flip Flop Part 3

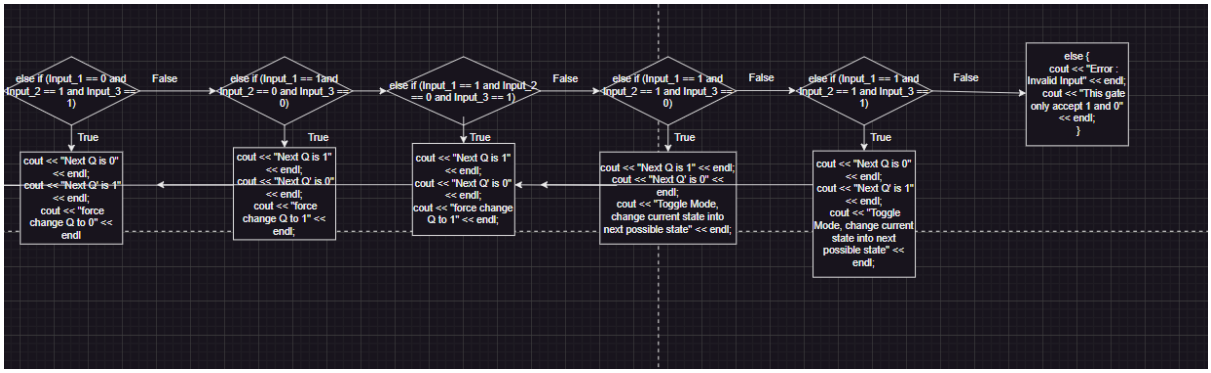


Figure 3.54 Exit

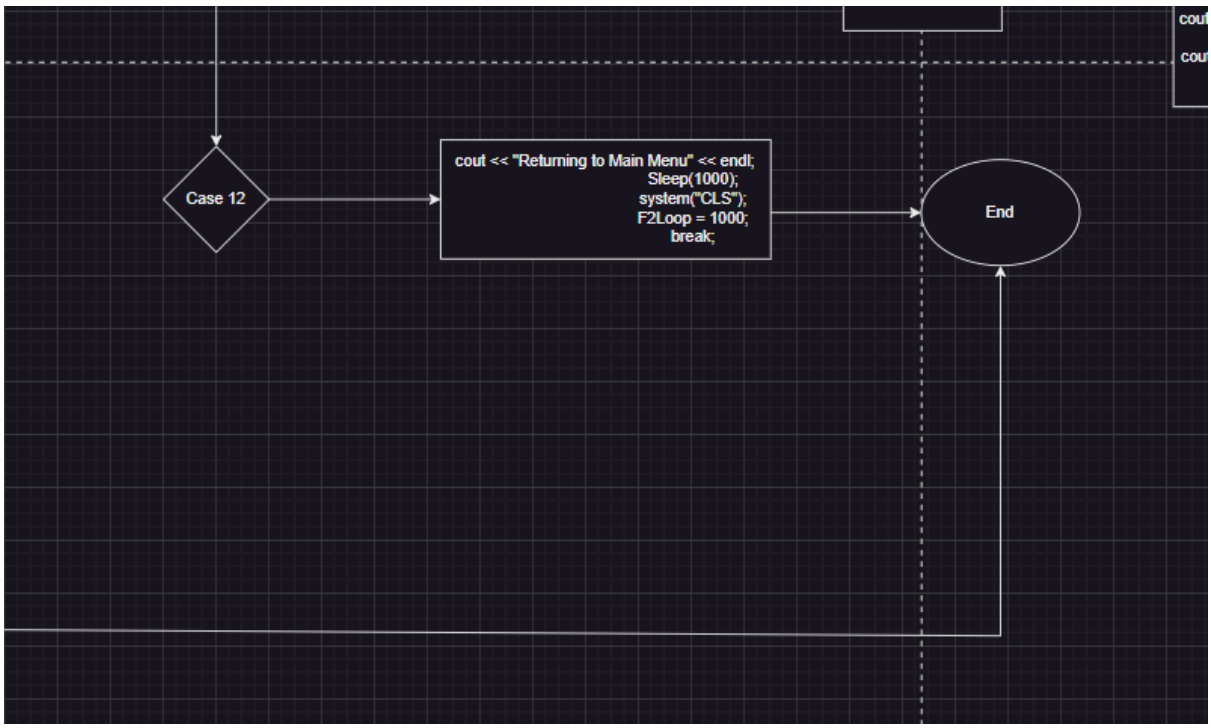
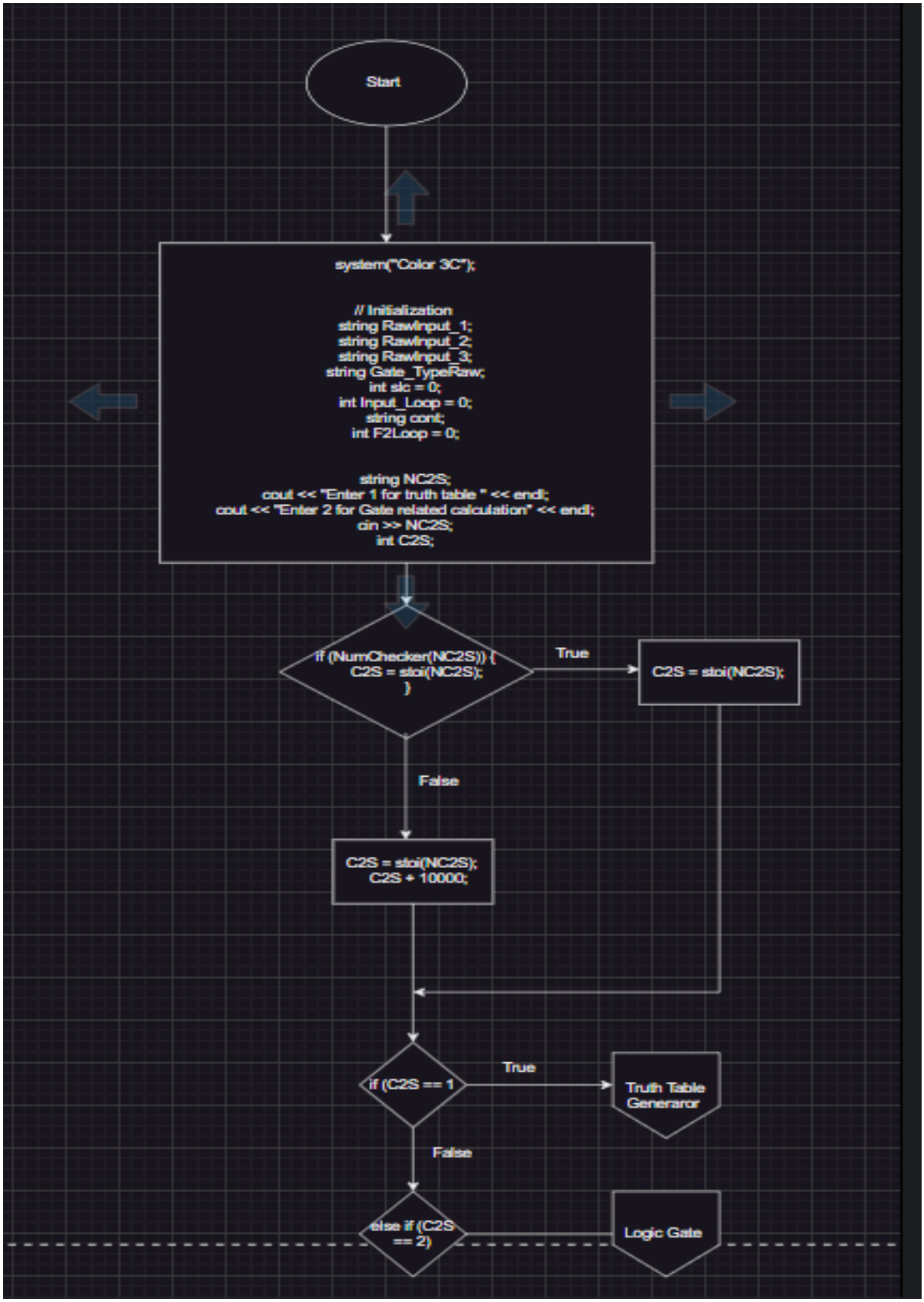


Figure 3.55 Case 2 Main Framework



Code

The RawInput will be processed by the number checking system which forces the compiler to declare as an error if an alphabetical input was keyed in

The concept of the part is very simple: Identify the types of input and verify them through numerous if and else if cases. If the input fits any of the if/else if cases, it will display the output stated in the case. If the input does not fit any of the cases, it will display an error message depending on the type of the input entered.

Figure 3.56 Prompt user input (AND GATE)

```
case 1:
{   cout << "AND GATE" << endl;
cout << "Provide First Input" << endl;
cin >> RawInput_1;
cout << "Provide Second Input" << endl;
cin >> RawInput_2;
```

Figure 3.57 Selection and output (AND Gate)

```
if (NumChecker(RawInput_1) && NumChecker(RawInput_2))
{
    int Input_1 = stoi(RawInput_1);
    int Input_2 = stoi(RawInput_2);
    if (Input_1 == 0 and Input_2 == 0)
    {
        cout << "Output is 0" << endl;

    }
    else if (Input_1 == 0 and Input_2 == 1)
    {
        cout << " Output is 0" << endl;
    }
    else if (Input_1 == 1 and Input_2 == 0)
    {
        cout << " Output is 0" << endl;
    }
    else if (Input_1 == 1 and Input_2 == 1)
    {
        cout << "Output is 1" << endl;
    }
```

Figure 3.58 Elimination of non 1s and 0s (AND Gate)

```
        }
        else {
            cout << "Error : Invalid Input" << endl;
            cout << "This gate only accept 1 and 0" << endl;
        }
    }
```

Figure 3.59 Elimination of Alphabetical input Error(AND Gate)

```
    else {
        cout << "Error" << endl;
        cout << "Accept integer only " << endl;
    }
    cout << " Press anything to continue" << endl;
    cin >> cont;
    system("CLS");
```

Figure 3.60 Prompt user input (OR GATE)

```
    case 2:
    {   cout << "OR GATE" << endl;
    cout << "Provide First Input" << endl;
    cin >> RawInput_1;
    cout << "Provide Second Input" << endl;
    cin >> RawInput_2;
```

Figure 3.61 Selection and output (OR Gate)

```
if (NumChecker(RawInput_1) && NumChecker(RawInput_2))
{
    int Input_1 = stoi(RawInput_1);
    int Input_2 = stoi(RawInput_2);
    if (Input_1 == 0 and Input_2 == 0)
    {
        cout << "Output is 0" << endl;

    }
    else if (Input_1 == 0 and Input_2 == 1)
    {
        cout << " Output is 1" << endl;
    }
    else if (Input_1 == 1 and Input_2 == 0)
    {
        cout << " Output is 1" << endl;
    }
    else if (Input_1 == 1 and Input_2 == 1)
    {
        cout << "Output is 1" << endl;
    }
```

Figure 3.62 Elimination of non 1s and 0s (OR Gate)

```
    else {
        cout << "Error : Invalid Input" << endl;
        cout << "This gate only accept 1 and 0" << endl;
    }
}
```

Figure 3.63 Elimination of Alphabetical input Error(OR Gate)

```
        else {
            cout << "Error" << endl;
            cout << "Accept integer only " << endl;
        }
        cout << " Press anything to continue" << endl;
        cin >> cont;
        system("CLS");
        break;
        }
        case 3:
```

Figure 3.64 Prompt user input (NAND GATE)

```
{   cout << "NAND Gate" << endl;
cout << "Provide First Input" << endl;
cin >> RawInput_1;
cout << "Provide Second Input" << endl;
cin >> RawInput_2;
```

Figure 3.65 Selection and output (NAND Gate)

```
if (Input_1 == 0 and Input_2 == 0)
{
    cout << "Output is 1" << endl;

}
else if (Input_1 == 0 and Input_2 == 1)
{
    cout << " Output is 1" << endl;
}
else if (Input_1 == 1 and Input_2 == 0)
{
    cout << " Output is 1" << endl;
}
else if (Input_1 == 1 and Input_2 == 1)
{
    cout << "Output is 0" << endl;
}
else {
```

Figure 3.66 Elimination of non 1s and 0s (NAND Gate)

```
else {
    cout << "Error : Invalid Input" << endl;
    cout << "This gate only accept 1 and 0" << endl;
}
}
```

Figure 3.67 Elimination of Alphabetical input Error(NAND Gate)

```
else {
    cout << "Error" << endl;
    cout << "Accept integer only " << endl;
}
cout << " Press anything to continue" << endl;
cin >> cont;
system("CLS");
```

Figure 3.68 Prompt user input (NOR GATE)

```
case 4:
{   cout << "NOR GATE" << endl;
cout << "Provide First Input" << endl;
cin >> RawInput_1;
cout << "Provide Second Input" << endl;
cin >> RawInput_2;
```

Figure 3.69 Selection and output (NOR Gate)

```
if (Input_1 == 0 and Input_2 == 0)
{
    cout << "Output is 1" << endl;

}
else if (Input_1 == 0 and Input_2 == 1)
{
    cout << " Output is 0" << endl;
}
else if (Input_1 == 1 and Input_2 == 0)
{
    cout << " Output is 0" << endl;
}
else if (Input_1 == 1 and Input_2 == 1)
{
    cout << "Output is 0" << endl;
}
```

Figure 3.70 Elimination of non 1s and 0s (NOR Gate)

```
else {
    cout << "Error : Invalid Input" << endl;
    cout << "This gate only accept 1 and 0" << endl;
}
```

Figure 3.71 Elimination of Alphabetical input Error(NOR Gate)

```
}
else {
    cout << "Error : Invalid Input" << endl;
    cout << "This gate only accept 1 and 0" << endl;
}
cout << " Press anything to continue" << endl;
cin >> cont;
system("CLS");
```

Figure 3.72 Prompt user input (FULL ADDER)

```
{   cout << "Full Adder" << endl;
cout << "Provide First Input (A) " << endl;
cin >> RawInput_1;
cout << "Provide Second Input (B) " << endl;
cin >> RawInput_2;
cout << "Provide Third Input (C)" << endl;
cin >> RawInput_3;
```

Figure 3.73 Selection and output(FULL ADDER)

```
if (Input_1 == 0 and Input_2 == 0 and Input_3 == 0)
{
    cout << "Sum is 0" << endl;
    cout << "C-Out is 0" << endl;

}
else if (Input_1 == 0 and Input_2 == 0 and Input_3 == 1)
{
    cout << "Sum is 1" << endl;
    cout << "C-Out is 0" << endl;

}
else if (Input_1 == 0 and Input_2 == 1 and Input_3 == 0)
{
    cout << "Sum is 1" << endl;
    cout << "C-Out is 0" << endl;

}
else if (Input_1 == 0 and Input_2 == 1 and Input_3 == 1)
{
    cout << "Sum is 0" << endl;
    cout << "C-Out is 1" << endl;

}
else if (Input_1 == 1 and Input_2 == 0 and Input_3 == 0)
{
    cout << "Sum is 1" << endl;
    cout << "C-Out is 0" << endl;

}
else if (Input_1 == 1 and Input_2 == 0 and Input_3 == 1)
{
    cout << "Sum is 0" << endl;
    cout << "C-Out is 1" << endl;

}
else if (Input_1 == 1 and Input_2 == 1 and Input_3 == 0)
{
    cout << "Sum is 0" << endl;
    cout << "C-Out is 1" << endl;

}
else if (Input_1 == 1 and Input_2 == 1 and Input_3 == 1)
{
    cout << "Sum is 1" << endl;
    cout << "C-Out is 1" << endl;
```

Figure 3.74 Elimination of non 1s and 0s (FULL ADDER)

```
    }
    else {
        cout << "Error : Invalid Input" << endl;
        cout << "This gate only accept 1 and 0" << endl;
    }
```

Figure 3.75 Elimination of Alphabetical input Error(FULL ADDER)

```
    else {
        cout << "Error" << endl;
        cout << "Accept integer only " << endl;
    }
    cout << " Press anything to continue" << endl;
    cin >> cont;
```

Figure 3.76 Prompt user input (HALF ADDER)

```
{   cout << "Half Adder" << endl;
cout << "Provide First Input" << endl;
cin >> RawInput_1;
cout << "Provide Second Input" << endl;
cin >> RawInput_2;
```

Figure 3.77 Selection and output (HALF ADDER)

```
if (Input_1 == 0 and Input_2 == 0)
{
    cout << "Sum is 0" << endl;
    cout << "Carry is 0" << endl;

}
else if (Input_1 == 0 and Input_2 == 1)
{
    cout << "Sum is 1" << endl;
    cout << "Carry is 0" << endl;
}
else if (Input_1 == 1 and Input_2 == 0)
{
    cout << "Sum is 1" << endl;
    cout << "Carry is 0" << endl;
}
else if (Input_1 == 1 and Input_2 == 1)
{
    cout << "Sum is 1" << endl;
    cout << "Carry is 1" << endl;
}
```

Figure 3.78 Elimination of non 1s and 0s (HALF ADDER)

```
    else {
        cout << "Error : Invalid Input" << endl;
        cout << "This gate only accept 1 and 0" << endl;
    }
```

Figure 3.79 Elimination of Alphabetical input Error (HALF ADDER)

```
    else {
        cout << "Error : Invalid Input" << endl;
        cout << "This gate only accept 1 and 0" << endl;
    }
    cout << " Press anything to continue" << endl;
    cin >> cont;
```

Figure 3.80 Prompt user input (BINARY SUBTRACTOR)

```
{   cout << "Binary Subtractor" << endl;
cout << "Provide First Input (A) " << endl;
cin >> RawInput_1;
cout << "Provide Second Input (B) " << endl;
cin >> RawInput_2;
cout << "Provide Third Input (Borrow-In)" << endl;
cin >> RawInput_3;
```

Figure 3.81 Selection and output (HALF ADDER)

```
if (Input_1 == 0 and Input_2 == 0 and Input_3 == 0)
{
    cout << "Diff is 0" << endl;
    cout << "Borrow is 0" << endl;

}
else if (Input_1 == 0 and Input_2 == 0 and Input_3 == 1)
{
    cout << "Diff is 1" << endl;
    cout << "Borrow is 1" << endl;

}
else if (Input_1 == 0 and Input_2 == 1 and Input_3 == 0)
{
    cout << "Diff is 1" << endl;
    cout << "Borrow is 1" << endl;

}
else if (Input_1 == 0 and Input_2 == 1 and Input_3 == 1)
{
    cout << "Diff is 0" << endl;
    cout << "Borrow is 1" << endl;

}
else if (Input_1 == 1 and Input_2 == 0 and Input_3 == 0)
{
    cout << "Diff is 1" << endl;
    cout << "Borrow is 0" << endl;

}
else if (Input_1 == 1 and Input_2 == 0 and Input_3 == 1)
{
    cout << "Diff is 0" << endl;
    cout << "Borrow is 0" << endl;

}
else if (Input_1 == 1 and Input_2 == 1 and Input_3 == 0)
{
    cout << "Diff is 0" << endl;
    cout << "Borrow is 0" << endl;

}
else if (Input_1 == 1 and Input_2 == 1 and Input_3 == 1)
{
    cout << "Diff is 1" << endl;
    cout << "Borrow is 1" << endl;
```

Figure 3.82 Elimination of non 1s and 0s (HALF ADDER)

```
else {
    cout << "Error : Invalid Input" << endl;
    cout << "This gate only accept 1 and 0" << endl;
}
```

Figure 3.83 Elimination of Alphabetical input Error (HALF ADDER)

```
else {
    cout << "Error" << endl;
    cout << "Accept integer only " << endl;
}
cout << " Press anything to continue" << endl;
cin >> cont;
system("CLS");
```

Figure 3.84 Prompt user input (SR Flip Flop)

```
case 8:
{   cout << "SR FLip Flop" << endl;
cout << "Provide First Input S" << endl;
cin >> RawInput_1;
cout << "Provide Second Input R" << endl;
cin >> RawInput_2;
```

Figure 3.85 Selection and output (SR Flip Flop)

```
if (Input_1 == 0 and Input_2 == 0)
{
    cout << "Q is 0" << endl;
    cout << "Q' is 1" << endl;

}
else if (Input_1 == 0 and Input_2 == 1)
{
    cout << "Q is 0" << endl;
    cout << "Q' is 1" << endl;
}
else if (Input_1 == 1 and Input_2 == 0)
{
    cout << "Q is 1" << endl;
    cout << "Q' is 0" << endl;
}
else if (Input_1 == 1 and Input_2 == 1)
{
    cout << "Output is indeterminable" << endl;
}
```

Figure 3.86 Elimination of non 1s and 0s (SR Flip Flop)

```
else {
    cout << "Error : Invalid Input" << endl;
    cout << "This gate only accept 1 and 0" << endl;
}
```

Figure 3.87 Elimination of Alphabetical input Error (SR Flip Flop)

```
else {
    cout << "Error" << endl;
    cout << "Accept integer only " << endl;
}
cout << " Press anything to continue" << endl;
cin >> cont;
system("CLS");
```

Figure 3.88 Prompt user input (D Flip Flop)

```
    {   cout << "D Flip Flop" << endl;
    cout << "Provide First Input D" << endl;
    cin >> RawInput_1;
    cout << "Provide current state " << endl;
    cin >> RawInput_2;
```

Figure 3.89 Selection and output (D Flip Flop)

```
if (Input_1 == 0 and Input_2 == 0)
{
    cout << "next state is 0" << endl;
}
else if (Input_1 == 0 and Input_2 == 1)
{
    cout << "next state is 0" << endl;
}
else if (Input_1 == 1 and Input_2 == 0)
{
    cout << "next state is 0" << endl;
}
else if (Input_1 == 1 and Input_2 == 1)
{
    cout << "next state is 0" << endl;
}
```

Figure 3.90 Elimination of non 1s and 0s (D Flip Flop)

```
else {
    cout << "Error : Invalid Input" << endl;
    cout << "This gate only accept 1 and 0" << endl;
}
```

Figure 3.91 Elimination of Alphabetical input Error (D Flip Flop)

```
else {
    cout << "Error" << endl;
    cout << "Accept integer only " << endl;
}
cout << " Press anything to continue" << endl;
cin >> cont;
```

Figure

Figure 3.92 Prompt user input (T Flip Flop)

```
{   cout << "T Flip Flop" << endl;
cout << "Provide First Input t" << endl;
cin >> RawInput_1;
cout << "Provide current state " << endl;
cin >> RawInput_2;
```

Figure 3.93 Selection and output (T Flip Flop)

```
if (Input_1 == 0 and Input_2 == 0)
{
    cout << "next state is 0" << endl;
}
else if (Input_1 == 0 and Input_2 == 1)
{
    cout << "next state is 1" << endl;
}
else if (Input_1 == 1 and Input_2 == 0)
{
    cout << "next state is 1" << endl;
}
else if (Input_1 == 1 and Input_2 == 1)
{
    cout << "next state is 0" << endl;
}
```

Figure 3.94 Elimination of non 1s and 0s (T Flip Flop)

```
else {
    cout << "Error : Invalid Input" << endl;
    cout << "This gate only accept 1 and 0" << endl;
}
```

Figure 3.95 Elimination of Alphabetical input Error(T Flip Flop)

```
else {
    cout << "Error" << endl;
    cout << "Accept integer only " << endl;
}
cout << " Press anything to continue" << endl;
cin >> cont;
system("CLS");
```

Figure 3.96 Prompt user input (JK Flip Flop)

```
{    cout << "JK Flip Flop" << endl;
cout << "Provide First Input J" << endl;
cin >> RawInput_1;
cout << "Provide Second Input K " << endl;
cin >> RawInput_2;
cout << "Provide Current State" << endl;
cin >> RawInput_3;
```

Figure 3.97 Selection and output (JK Flip Flop)

```
if (Input_1 == 0 and Input_2 == 0 and Input_3 == 0)
{
    cout << "Next Q is 0" << endl;
    cout << "Next Q' is 1" << endl;
    cout << "Memory, No Change in State" << endl;

}
else if (Input_1 == 0 and Input_2 == 0 and Input_3 == 1)
{
    cout << "Next Q is 1" << endl;
    cout << "Next Q' is 0" << endl;
    cout << "Memory, No Change in State" << endl;

}
else if (Input_1 == 0 and Input_2 == 1 and Input_3 == 0)
{
    cout << "Next Q is 0" << endl;
    cout << "Next Q' is 1" << endl;
    cout << "force change Q to 0" << endl;

}
else if (Input_1 == 0 and Input_2 == 1 and Input_3 == 1)
{
    cout << "Next Q is 0" << endl;
    cout << "Next Q' is 1" << endl;
    cout << "force change Q to 0" << endl;

}
else if (Input_1 == 1 and Input_2 == 0 and Input_3 == 0)
{
    cout << "Next Q is 1" << endl;
    cout << "Next Q' is 0" << endl;
    cout << "force change Q to 1" << endl;

}
else if (Input_1 == 1 and Input_2 == 0 and Input_3 == 1)
{
    cout << "Next Q is 1" << endl;
    cout << "Next Q' is 0" << endl;
    cout << "force change Q to 1" << endl;

}
else if (Input_1 == 1 and Input_2 == 1 and Input_3 == 0)
{
    cout << "Next Q is 1" << endl;
    cout << "Next Q' is 0" << endl;
    cout << "Toggle Mode, change current state into next possible state" << endl;
```

Figure 3.98 Elimination of non 1s and 0s (JK Flip Flop)

```
        }
        else {
            cout << "Error : Invalid Input" << endl;
            cout << "This gate only accept 1 and 0" << endl;
        }
    }
```

Figure 3.99 Elimination of Alphabetical input Error(JK Flip Flop)

```
        }
        else {
            cout << "Error" << endl;
            cout << "Accept integer only " << endl;
        }
        cout << " Press anything to continue" << endl;
        cin >> cont;
```

Figure 3.100 Exit to menu

```
case 12:
{cout << "Returning to Main Menu" << endl;
Sleep(1000);
system("CLS");
F2Loop = 1000;
break;
}

default: {
    cout << "Error" << endl;
    cout << "Enter a value from 1 to 12" << endl;
    Sleep(1000);
    system("CLS");
}

}
```

Program

Figure 3.101 AND GATE



Figure 3.102 OR GATE

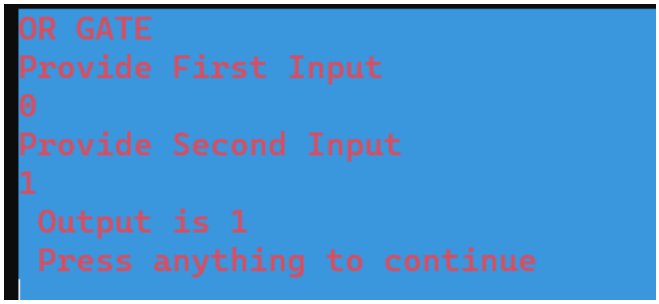


Figure 3.103 NAND GATE

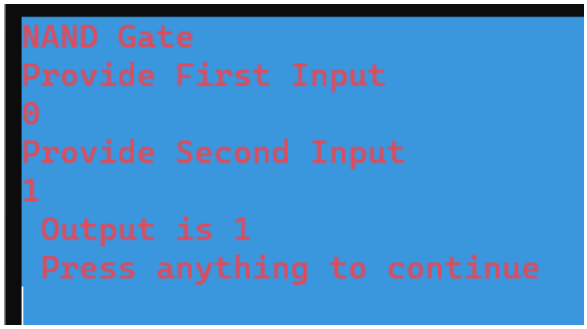


Figure 3.104 NOR GATE

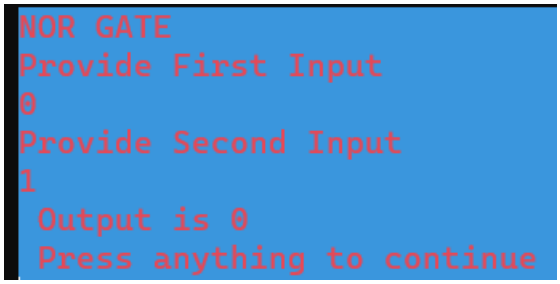


Figure 3.105 Full Adder

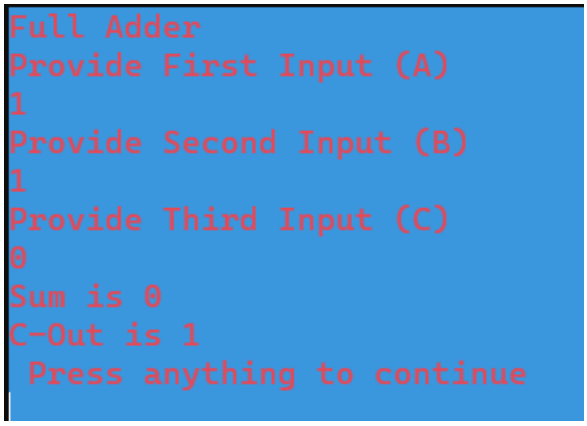


Figure 3.106 Half Adder

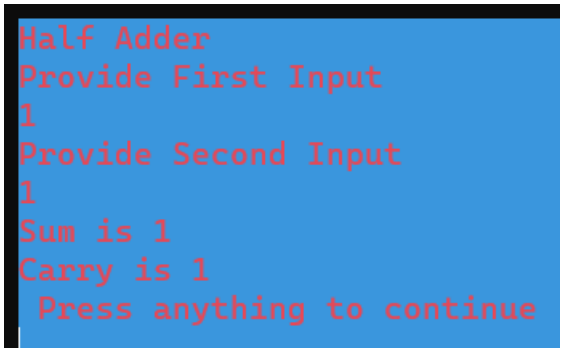


Figure 3.107 Binary Subtractor

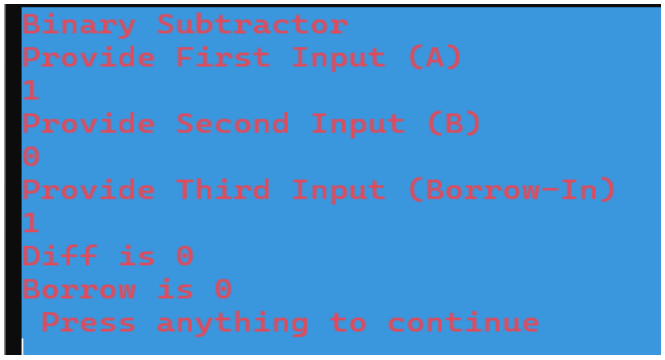


Figure 3.108 SR Flip Flop

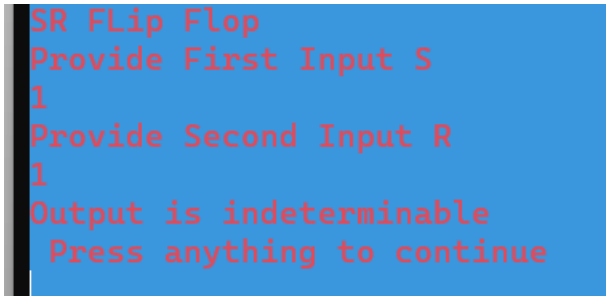


Figure 3.109 D Flip Flop

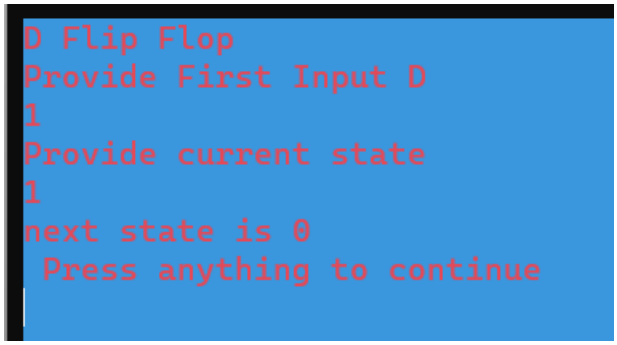


Figure 3.110 T Flip Flop

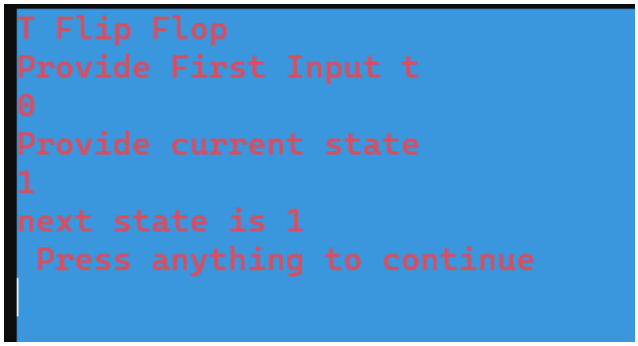


Figure 3.111 JK Flip Flop



Specialty and limitation for the program

This program only serves one purpose: which is to display all types of gates and flip flops. There are 4 types of gates and 3 types of binary calculator and 4 types of flip flops present in the system. They are AND OR NAND NOR 4 types of gates and Full adder half adder binary subtractor 3 types of calculators and SR D T JK 4 types of flip flop.

For the gate types AND gate refers to an electrical circuit that combines two signals so that the output is on if both signals are present. (Collins Dictionary) An OR gate is a logic gate that performs logical OR operation. A logical OR operation has a high output (1) if one or both the inputs to the gate are high (1). If neither input is high, a low output (0) result. (Electrical4U 2020). NAND Gate on the other hand is an inverted AND Gate which displays the complete opposite result in respect to AND Gate. Hence it will provide a HIGH as an output unless both inputs are HIGH. (Electronic4U 2022) When NOR gate is the inverse of OR gate which provides a HIGH output if both input are LOW and provide an output of LOW when one or more inputs are HIGH (Priyanshu Vaish 2022)

For the Binary Calculators, A Full Adder is a type of adder that takes three inputs and produces two outputs. The initial two inputs are denoted as A and B, while the third input is referred to as the input carry, labelled as C-IN. The output carry is designated as C-OUT, and the standard output is identified as S, which represents the sum. C-OUT is also recognized as the majority 1's detector, as its output becomes high when more than one input is set to high. (GeeksforGeeks). A half adder is similar to full adder just that it can only accept two inputs. It sums up two input and provide the output. Hence as long as one of the inputs is 0 when the other input is 1, it will provide a HIGH as the output. (Margaret Rouse, November 2020) Lastly, the binary subtractor refers to a combinational circuit which performs subtraction of single bit binary numbers.(Electircal4U 2020) It has the truth table as follows

Minuend (A)	Subtrahend (B)	Difference (D)	Borrow (b)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

For the Flip flops, they refer to an electronic component that store binary data and has two stable states. (Electronics4U 2023) SR Flip flops store the data as binary until it is being used. It can store up to only one bit. (Priyanshu Vaish 2022) D Flip flop refers to a component that delas the change until next rise of the clock at AC current provision. T Flip flop refers to the toggle flip flop which toggles the change in output when “t” is ON. Lastly JK flip flop is a SR flip flop with an added layer of feedback that affects the input of the flip flop. (Nidhi Agarwal 2023)

Binary Conversion:

Number System Converter:

In this program, user is able to convert between different number system. The number systems included in this program are Binary (Base 2), Octal (Base 8), Decimal (Base 10) and Hexadecimal (Base 16). This versatile tool empowers users to effortlessly switch between these number systems, making it an essential utility for students, programmers, and anyone working with different numeral bases. Whether you need to convert binary to decimal, octal to hexadecimal, or any other combination, this program simplifies the process and ensures accurate results.

Flowchart

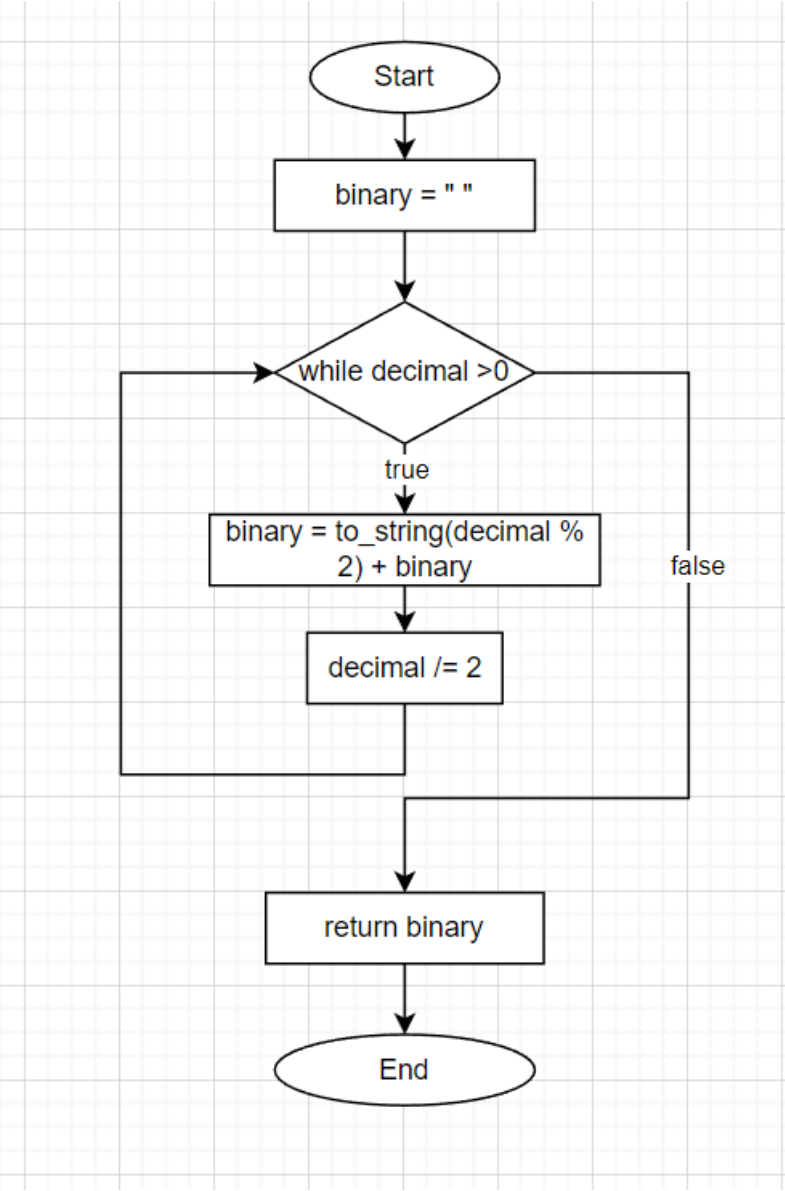


Figure 6.1: Flowchart of decimal to binary.

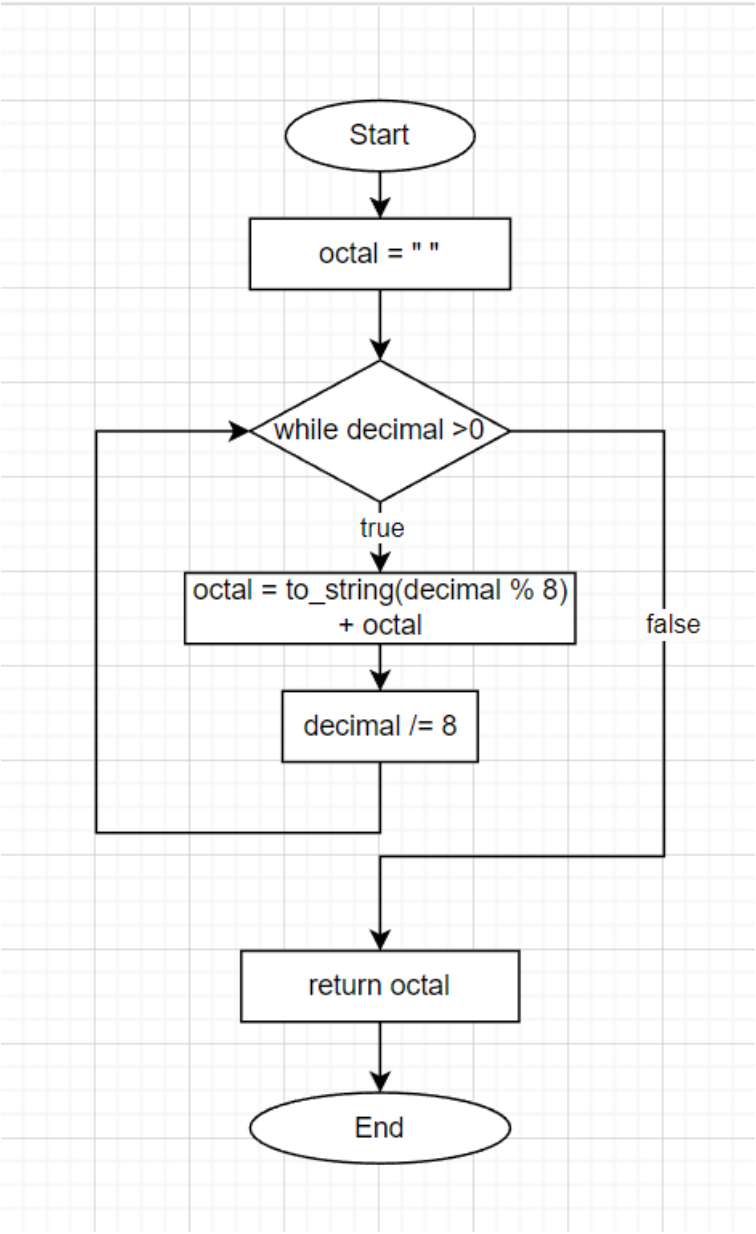


Figure 6.2: Flowchart of decimal to octal.

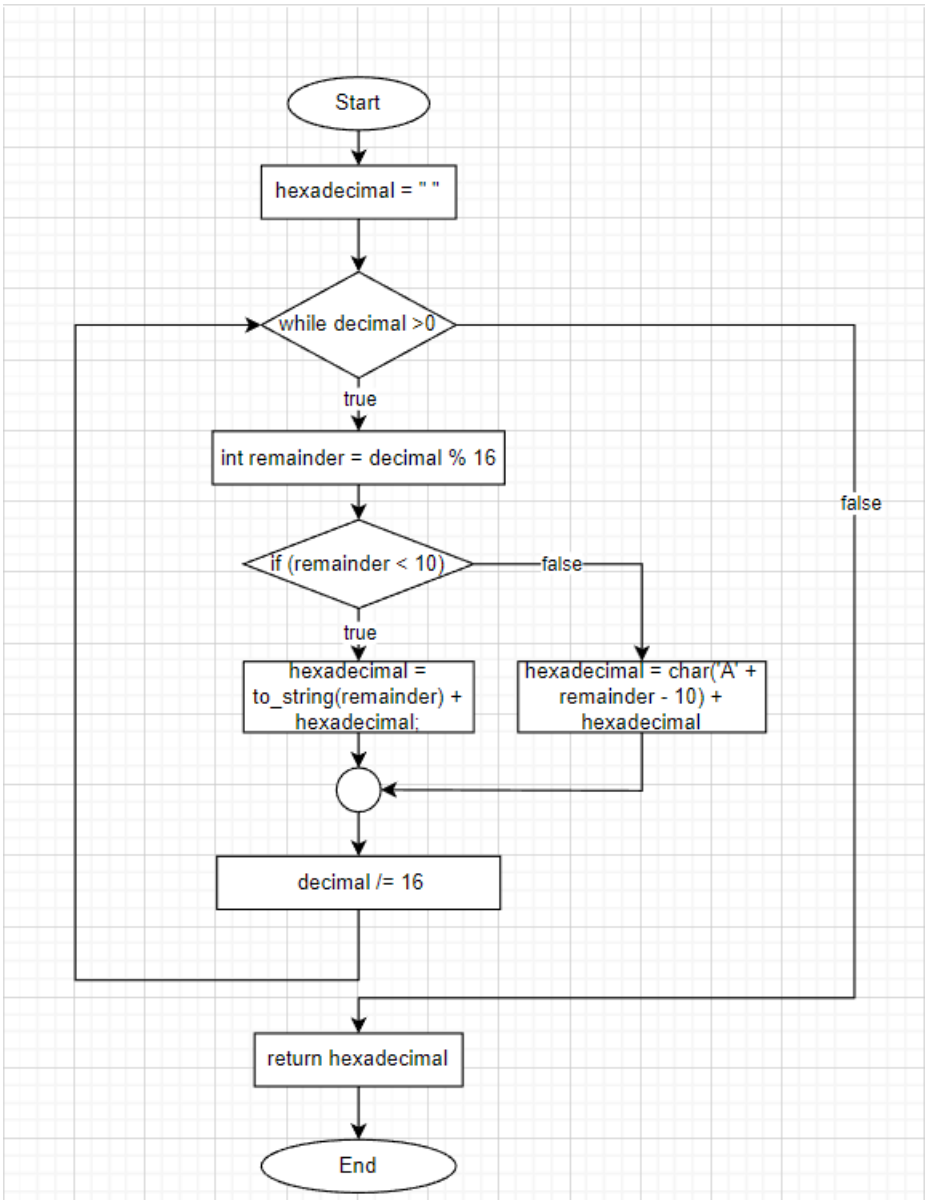


Figure 6.3: Flowchart of decimal to hexadecimal.

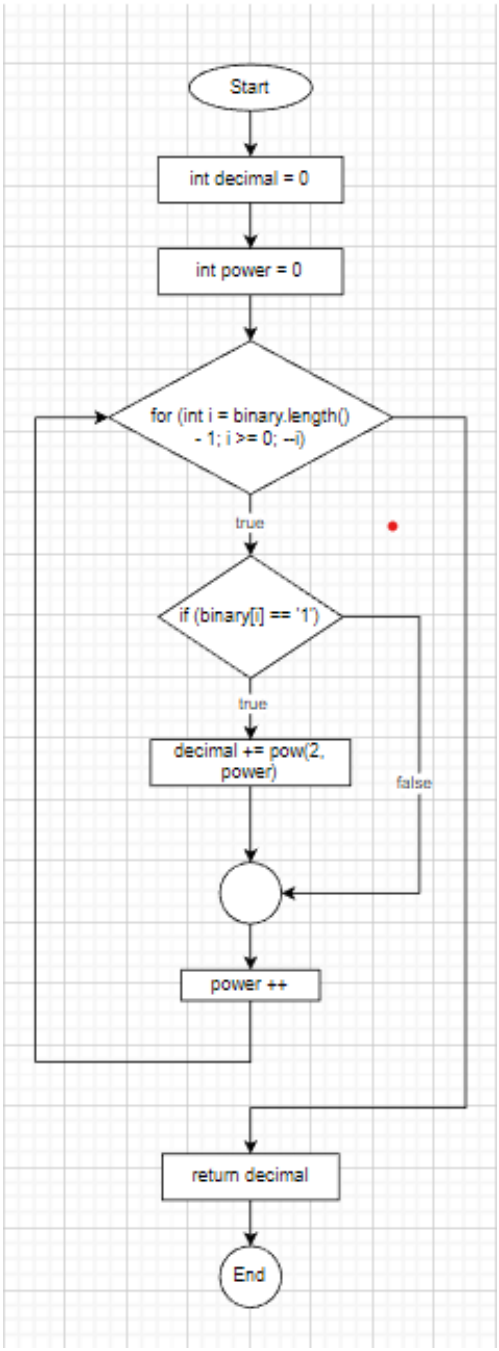


Figure 6.4: Flowchart of binary to decimal

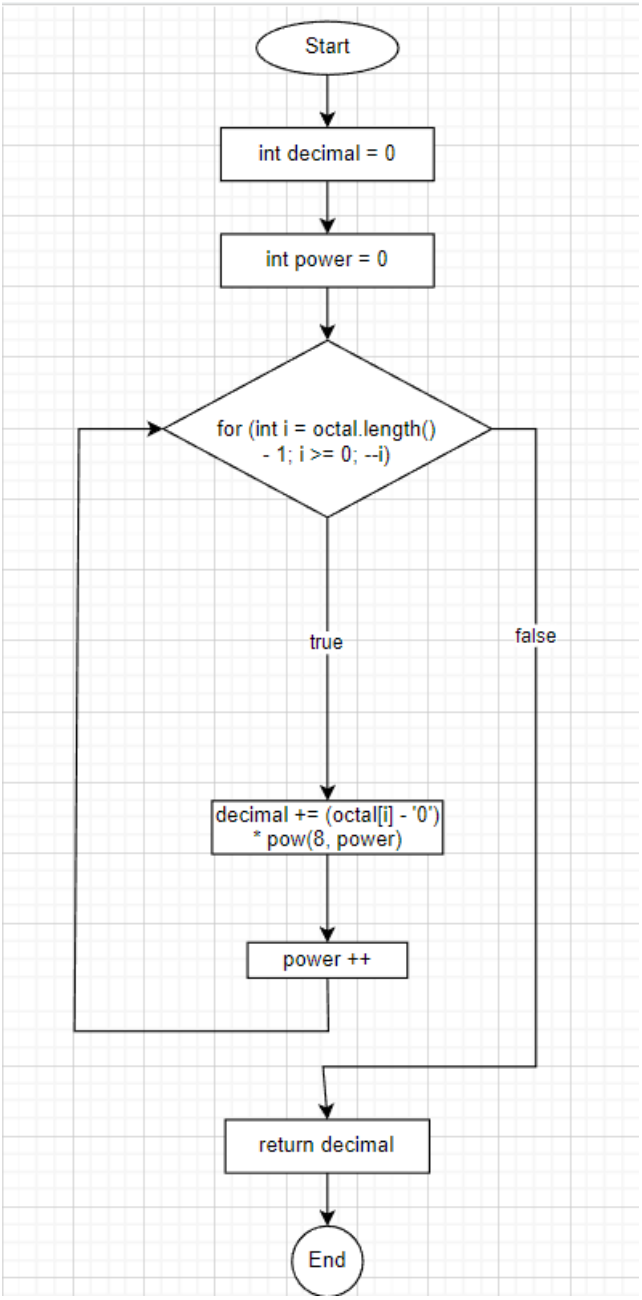


Figure 6.5: Flowchart of octal to decimal.

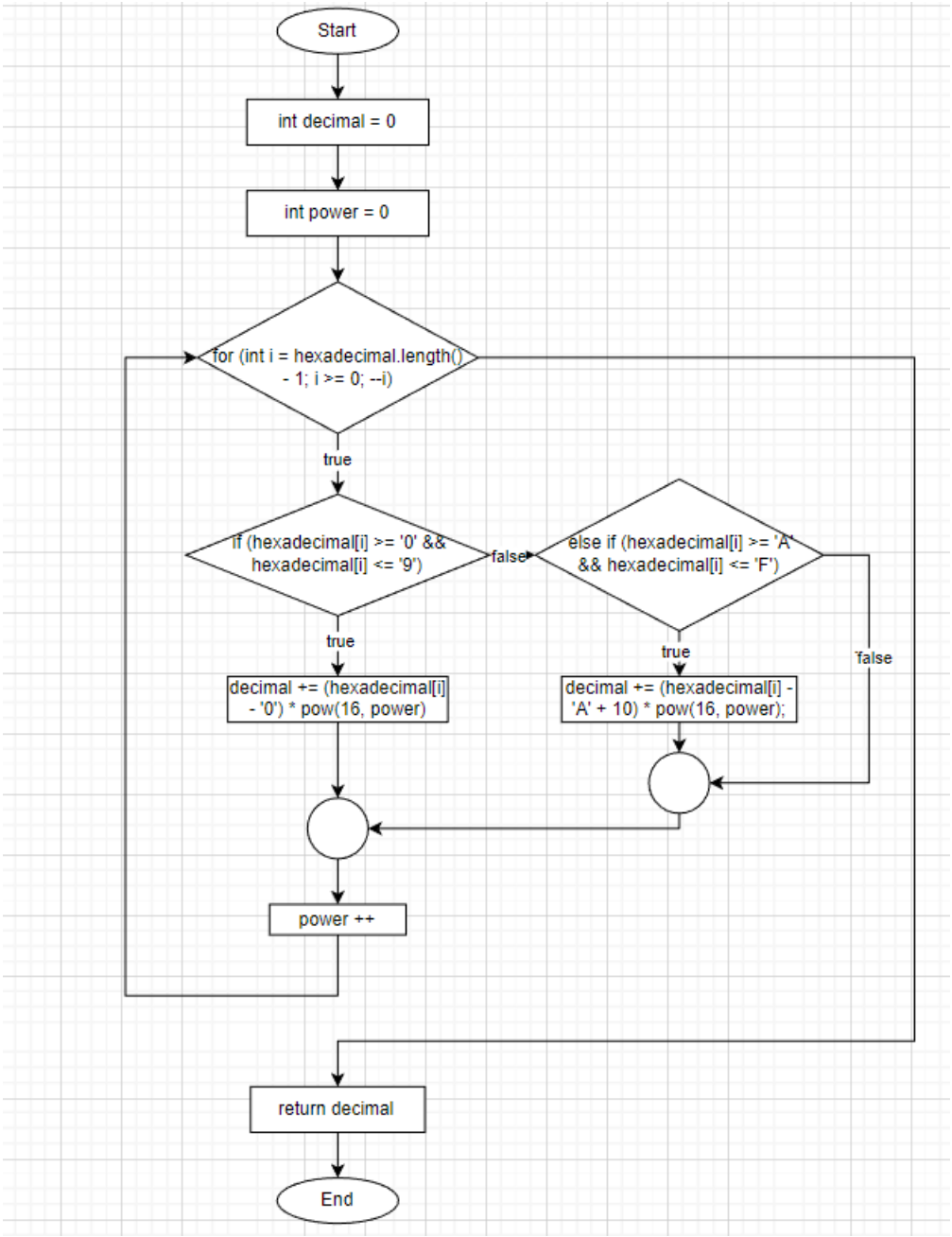


Figure 6.6: Flowchart of hexadecimal to decimal.

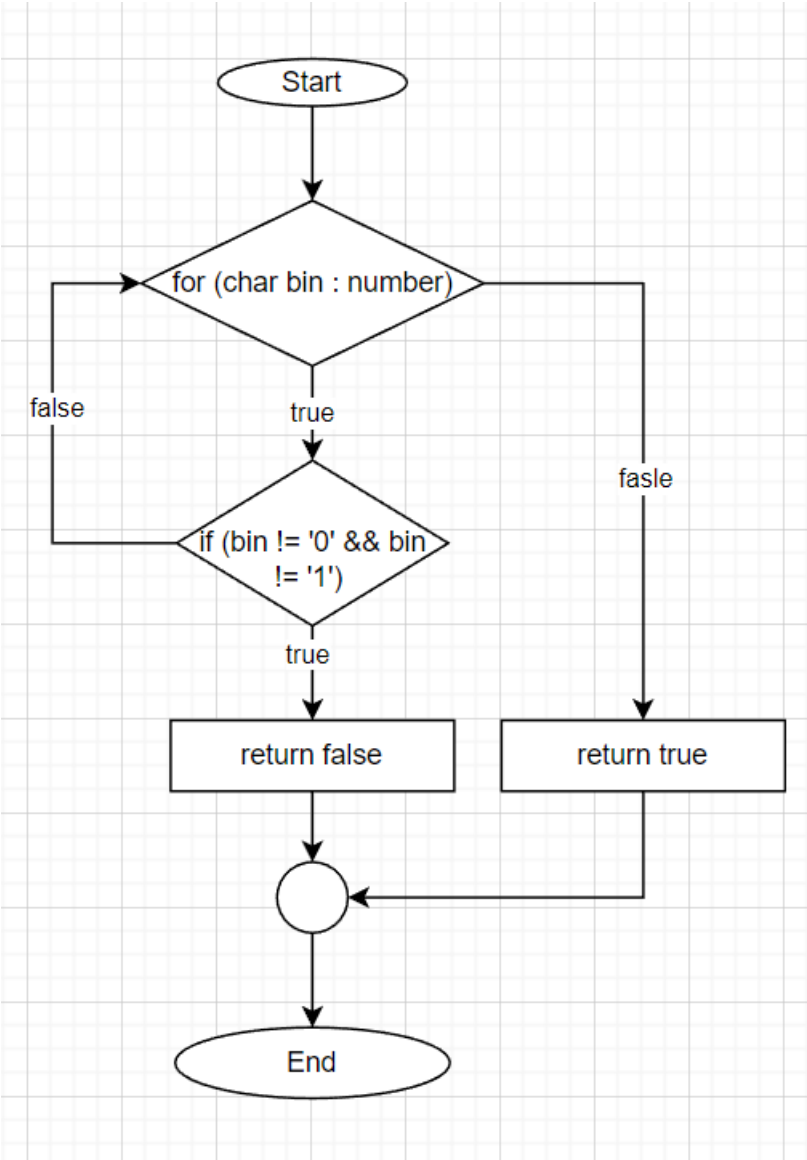


Figure 6.7: Flowchart of binary checking.

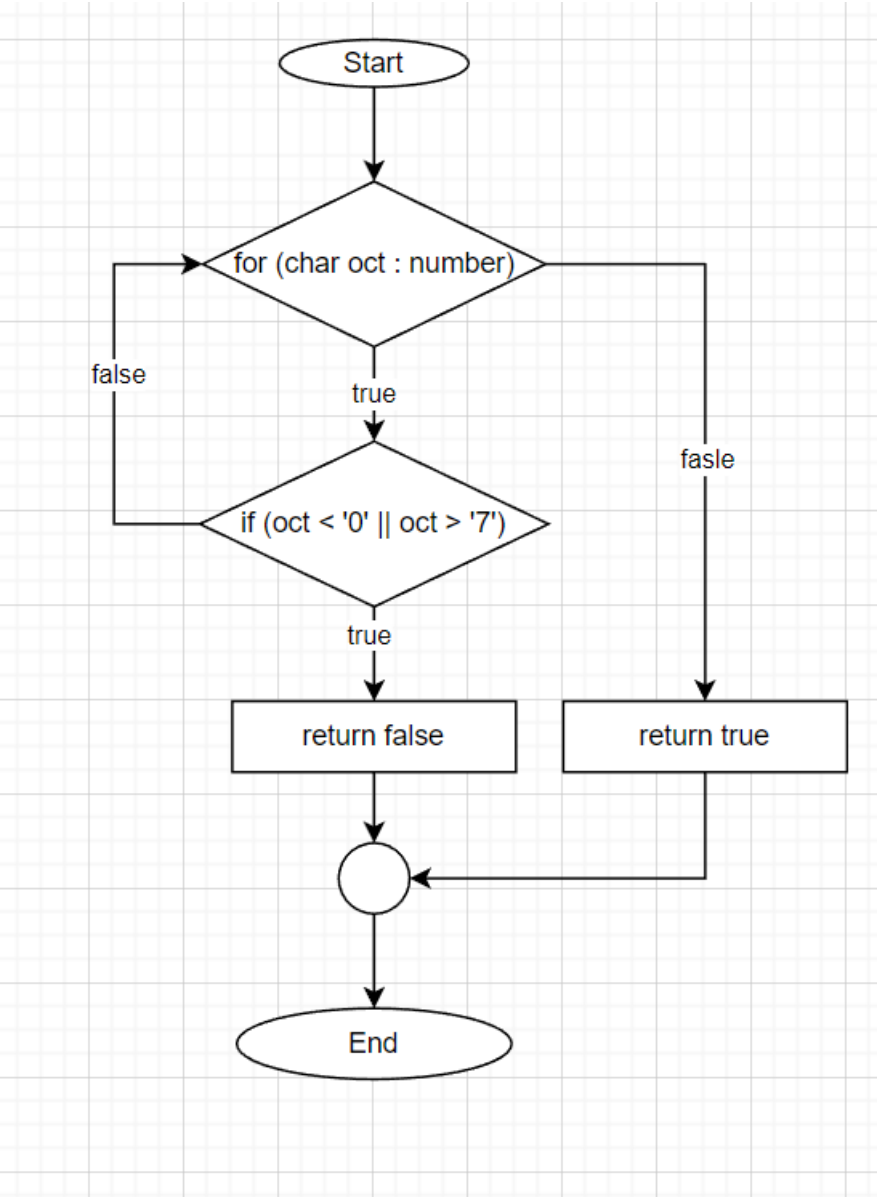


Figure 6.8: Flowchart of octal checking.

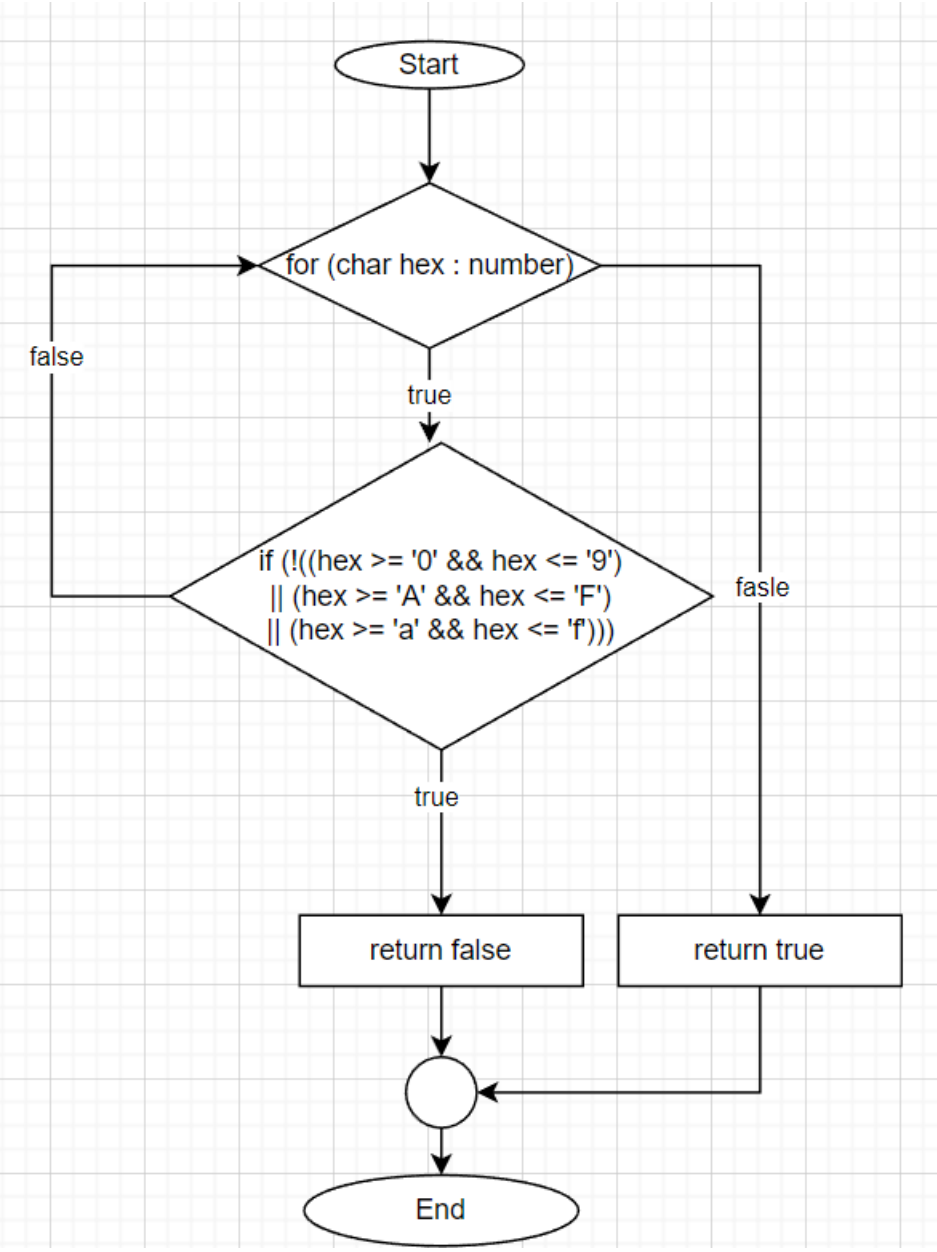


Figure 6.9: Flowchart of hexadecimal checking.



Figure 6.10: Flowchart of Main Program of Number system conversion. (Part1)

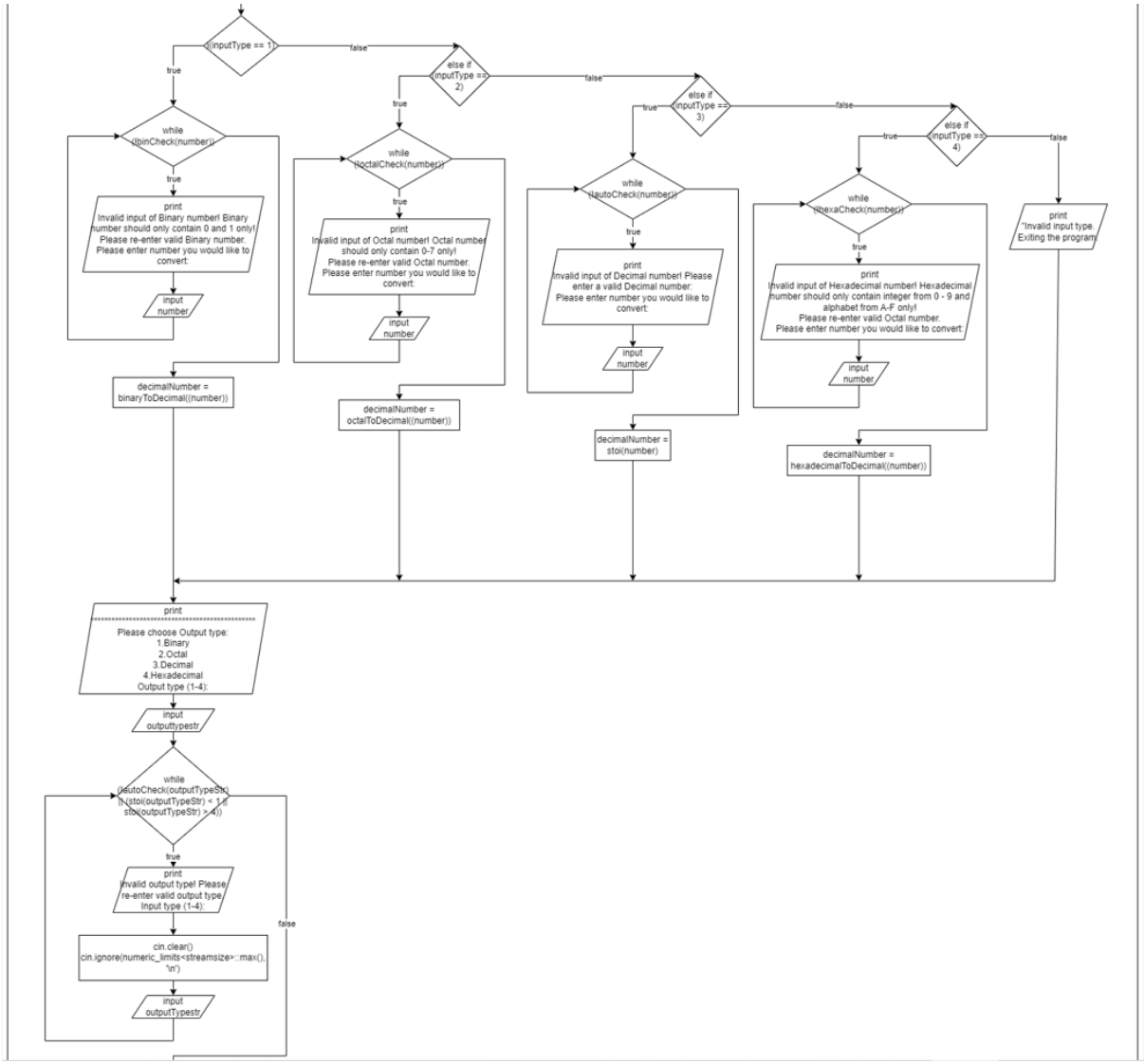


Figure 6.10(b): Flowchart of Main Program of Number system conversion. (Part2)

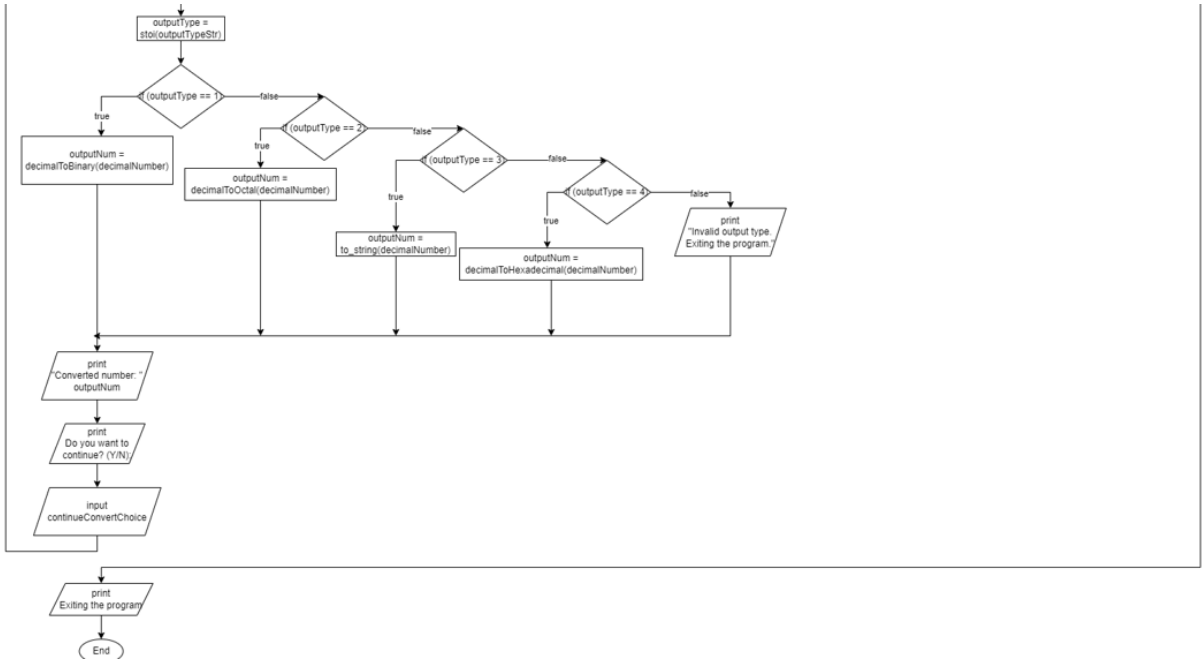


Figure 6.10(c): Flowchart of Main Program of Number system conversion. (Part3)

Screenshots of program

```
// Function to convert decimal to binary
string decimalToBinary(int decimal)
{
    string binary = "";
    while (decimal > 0)
    {
        binary = to_string(decimal % 2) + binary;
        decimal /= 2;
    }
    return binary;
}
```

Figure 6.11: Function to convert decimal to binary.

```
// Function to convert decimal to octal
string decimalToOctal(int decimal)
{
    string octal = "";
    while (decimal > 0)
    {
        octal = to_string(decimal % 8) + octal;
        decimal /= 8;
    }
    return octal;
}
```

Figure 6.12: Function to convert decimal to octal.

```
// Function to convert decimal to hexadecimal
string decimalToHexadecimal(int decimal)
{
    string hexadecimal = "";
    while (decimal > 0)
    {
        int remainder = decimal % 16;
        if (remainder < 10)
            hexadecimal = to_string(remainder) + hexadecimal;
        else
            hexadecimal = char('A' + remainder - 10) + hexadecimal;
        decimal /= 16;
    }
    return hexadecimal;
}
```

Figure 6.13: Function to convert decimal to hexadecimal.

```cpp
// Function to convert binary to decimal
int binaryToDecimal(const string& binary)
{
    int decimal = 0;
    int power = 0;
    for (int i = binary.length() - 1; i >= 0; --i)
    {
        if (binary[i] == '1')
            decimal += pow(2, power);
        power++;
    }
    return decimal;
}
```

Figure 6.14: Function to convert binary to decimal.

```cpp
// Function to convert octal to decimal
int octalToDecimal(const string& octal)
{
    int decimal = 0;
    int power = 0;
    for (int i = octal.length() - 1; i >= 0; --i)
    {
        decimal += (octal[i] - '0') * pow(8, power);
        power++;
    }
    return decimal;
}
```

Figure 6.15: Function to convert octal to decimal.

```cpp
// Function to convert hexadecimal to decimal
int hexadecimalToDecimal(const string& hexadecimal)
{
    int decimal = 0;
    int power = 0;
    for (int i = hexadecimal.length() - 1; i >= 0; --i)
    {
        if (hexadecimal[i] >= '0' && hexadecimal[i] <= '9')
            decimal += (hexadecimal[i] - '0') * pow(16, power);
        else if (hexadecimal[i] >= 'A' && hexadecimal[i] <= 'F')
            decimal += (hexadecimal[i] - 'A' + 10) * pow(16, power);
        power++;
    }
    return decimal;
}
```

Figure 6.16: Function to convert hexadecimal to decimal.

```
// Function to check binary input
bool binCheck(string number)
{
    for (char bin : number)
    {
        if (bin != '0' && bin != '1')
        {
            return false;
        }
    }
    return true;
}
```

Figure 6.17: Function to check binary input.

```
// Function to check octal input
bool octalCheck(string number)
{
    for (char oct : number)
    {
        if (oct < '0' || oct > '7')
            return false;
    }
    return true;
}
```

Figure 6.18: Function to check octal input.

```
// Function to check hexadecimal input
bool hexaCheck(string number)
{
    for (char hex : number)
    {
        if (!((hex >= '0' && hex <= '9') || (hex >= 'A' && hex <= 'F') || (hex >= 'a' && hex <= 'f')))
        {
            return false;
        }
    }
    return true;
}
```

Figure 6.19: Function to check hexadecimal input.

```
// Function to check if a string can be converted to an integer using stoi, for error input checking purpose
bool autoCheck(const string& str)
{
    try
    {
        stoi(str);
        return true;
    }
    catch (const invalid_argument& e)
    {
        return false;
    }
}
```

Figure 6.20: Function to check error alphabetical input.

```
int main()
{
    string number, outputNum;
    int inputType, outputType;
    int decimalNumber;
    bool continueConvert = true;

    while (continueConvert)
    {
        cout << "*********************************************" << endl;
        cout << "Please choose Input type:\n1.Binary\n2.Octal\n3.Decimal\n4.Hexadecimal" << endl;
        cout << "*********************************************" << endl;
        cout << "Input type (1-4): ";
        string inputTypeStr;
        cin >> inputTypeStr;
        cout << "*********************************************" << endl;
        while (!autoCheck(inputTypeStr) || (stoi(inputTypeStr) < 1 || stoi(inputTypeStr) > 4))
        {
            cout << "Invalid Input type! Please re-enter valid Input type." << endl;
            cout << "Input type (1-4): ";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cin >> inputTypeStr;
        }
        inputType = stoi(inputTypeStr);

        cout << "Please enter number you would like to convert: ";
        cin >> number;
```

Figure 6.21: Main function for number system converter (Part 1)

```
        cout << "*********************************************" << endl;
        if (inputType == 1)
        {
            while (!binCheck(number))
            {
                cout << "Invalid input of Binary number! Binary number should only contain 0 and 1 only!" << endl;
                cout << "Please re-enter valid Binary number." << endl;
                cout << "Please enter number you would like to convert: ";
                cin >> number;
            }
            decimalNumber = binaryToDecimal((number));
        }
        else if (inputType == 2)
        {
            while (!octalCheck(number))
            {
                cout << "Invalid input of Octal number! Octal number should only contain integer from 0 - 7 only!" << endl;
                cout << "Please re-enter valid Octal number." << endl;
                cout << "Please enter number you would like to convert: ";
                cin >> number;
            }
            decimalNumber = octalToDecimal((number));
        }
        else if (inputType == 3)
        {
            while (!autoCheck(number))
            {
                cout << "Invalid input of Decimal number! Please enter a valid Decimal number: ";
```

Figure 6.22: Main function for number system converter (Part 2)

```
            cin >> number;
        }
        decimalNumber = stoi(number);
    }
    else if (inputType == 4)
    {
        while (!hexaCheck(number))
        {
            cout << "Invalid input of Hexadecimal number! Hexadecimal number should only contain integer from 0 - 9 and alphabet from
            cout << "Please re-enter valid Hexadecimal number." << endl;
            cout << "Please enter number you would like to convert: ";
            cin >> number;
        }
        decimalNumber = hexadecimalToDecimal((number));
    }
    else
    {
        cout << "Invalid input type. Exiting the program." << endl;
        return 0;
    }

    cout << "Please choose Output type:\n1.Binary\n2.Octal\n3.Decimal\n4.Hexadecimal" << endl;
    cout << "Output type (1-4): ";
    string outputTypeStr;
    cin >> outputTypeStr;
    cout << "*****************************************" << endl;
    while (!autoCheck(outputTypeStr) || (stoi(outputTypeStr) < 1 || stoi(outputTypeStr) > 4))
    {
        cout << "Invalid Output type! Please re-enter valid Output type." << endl;
        cout << "Output type (1-4): ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cin >> outputTypeStr;
    }
    outputType = stoi(outputTypeStr);
```

Figure 6.23: Main function for number system converter (Part 3)

```
    if (outputType == 1)
    {
        outputNum = decimalToBinary(decimalNumber);
    }
    else if (outputType == 2)
    {
        outputNum = decimalToOctal(decimalNumber);
    }
    else if (outputType == 3)
    {
        outputNum = to_string(decimalNumber);
    }
    else if (outputType == 4)
    {
        outputNum = decimalToHexadecimal(decimalNumber);
    }
    else
    {
        cout << "Invalid output type. Exiting the program." << endl;
        return 0;
    }

    cout << "Converted number: " << outputNum << endl;
    cout << "*****************************************" << endl;
    char continueConvertChoice;
    cout << "Do you want to continue? (Y/N): ";
    cin >> continueConvertChoice;
    if (toupper(continueConvertChoice) != 'Y')
    {
        continueConvert = false;
    }
}
cout << "*****************************************" << endl;
cout << "Exiting the program." << endl;
return 0;
```

Figure 6.24: Main function for number system converter (Part 4)

Test plan and results

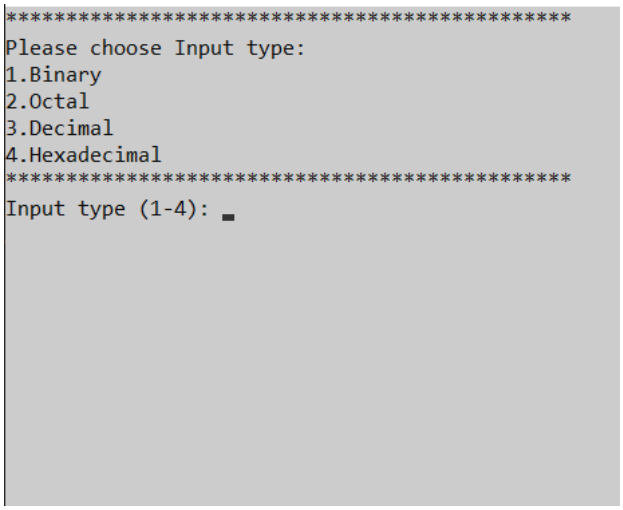


Figure 6.25: User interface of number system conversion.

First the program will show which number system is including in the program. The program will require the user to enter the type of number system they choose as an input. In this case, the program only accepts 1 – 4 as input, 1 stands for binary number system, 2 stands for octal number system, 3 stands for decimal number system, 4 stands for hexadecimal number system. If the input is out of range from 1 – 4, the program will show error message and require the user to re-enter the valid input.

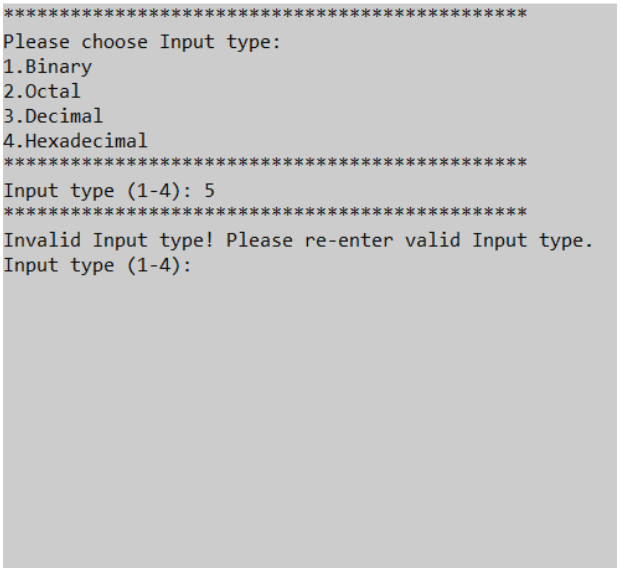


Figure 6.26: Error message for invalid input.

Once the user enters the correct input. The program will again require the user to enter number they would like to convert.

```
*********************************************
Please choose Input type:
1.Binary
2.Octal
3.Decimal
4.Hexadecimal
*********************************************
Input type (1-4): 1
*********************************************
Please enter number you would like to convert:
```

Figure 6.27: Number input requirement.

Since binary number system only consists of 1 and 0, while octal number system only consists of 0 until 7, and hexadecimal number system only consists of 0-9 and A-F, which A stand for 10, B stand for 11 and so on. The program will treat other input as invalid input. For example, if user choose Binary number system as input type, and insert an input except than 1 and 0, the program will show error message and require user to reinsert the valid input. This same for other number systems too.

```
*********************************************
Please choose Input type:
1.Binary
2.Octal
3.Decimal
4.Hexadecimal
*********************************************
Input type (1-4): 1
*********************************************
Please enter number you would like to convert: 1234567
*********************************************
Invalid input of Binary number! Binary number should only contains 0 and 1 only!
Please re-enter valid Binary number.
Please enter number you would like to convert:
```

Figure 6.28: Error message for Binary number system.

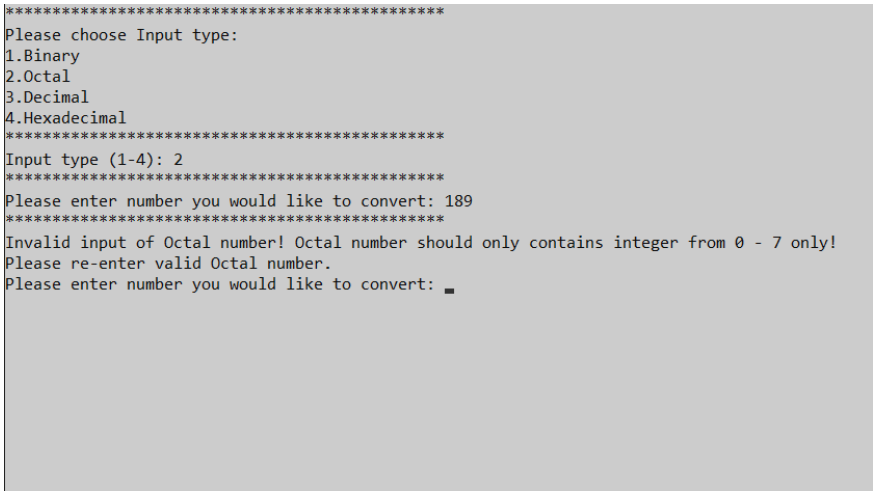


Figure 6.29: Error message for octal number system.

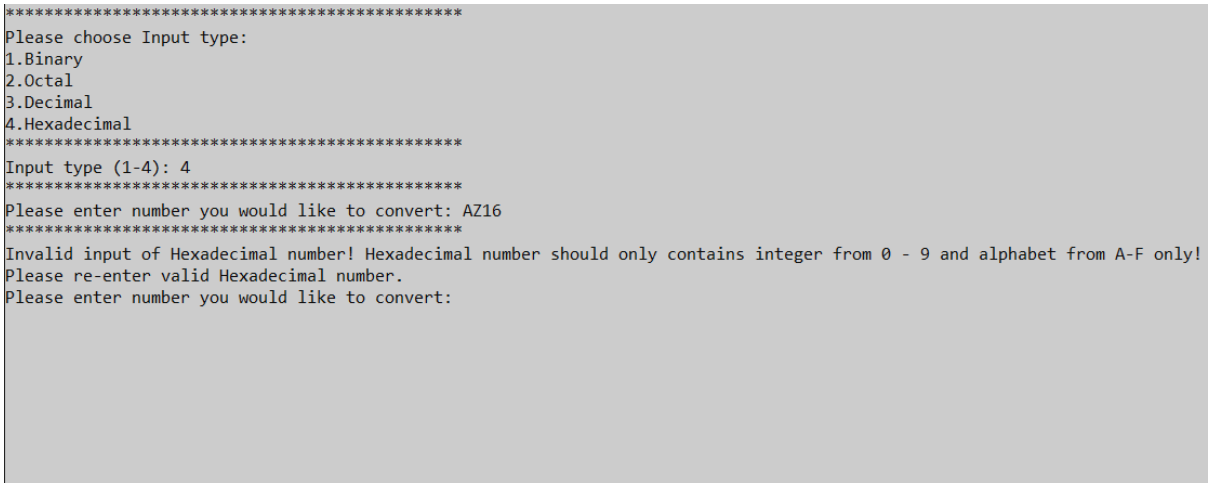


Figure 6.30: Error message for hexadecimal number system.

After user enter the correct number input. The program will require user to enter the output type they would like to convert to.

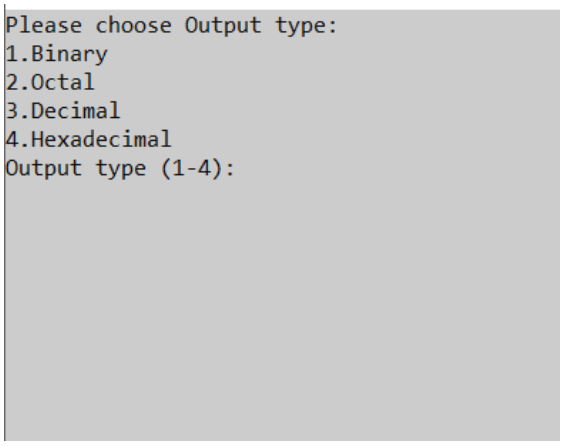


Figure 6.31: Output type requirement.

They are error messages too for the output type just like the input type if users enter invalid input. After entering a valid output, the program will do the calculation and shows the

result/converted number. The program will then ask the user whether they would like to continue using the number conversion program.

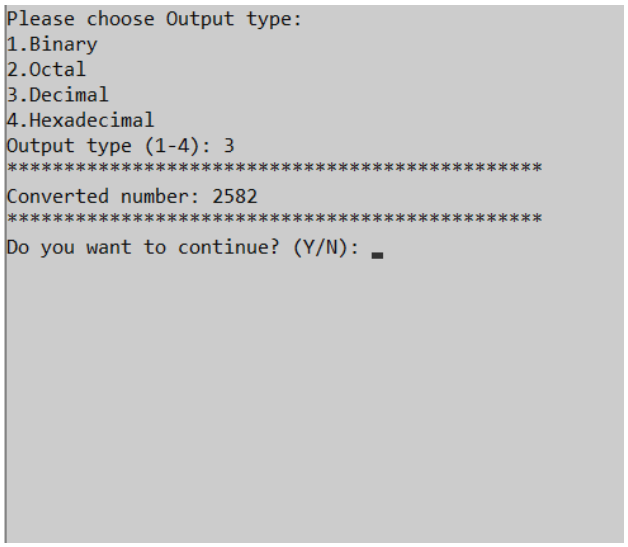


Figure 6.32: Result is generated.

The program will bring user back to Input type requirements (Figure 6.1) if user choose yes for continue, else the program will return user back to main menu if user choose no.

Specialty and limitation for the program

This program exhibits several key specialties that make it an invaluable tool for anyone dealing with numerical conversions. Firstly, its versatility sets it apart. The program is tailored to seamlessly convert between four fundamental number systems: Binary (Base 2), Octal (Base 8), Decimal (Base 10), and Hexadecimal (Base 16). This versatility ensures it can cater to a wide range of applications, from computer programming to advanced mathematics, simplifying conversions across these essential bases. A hallmark feature of number system converter is its unwavering commitment to accuracy. By automating conversions, it effectively eliminates the possibility of human error in manual calculations. This means users can rely on precise results for all their conversions, regardless of complexity or scale. Furthermore, Number system converter shines in terms of timesaving. It significantly reduces the time required for number system conversions, which can be especially beneficial for programmers working with multiple bases. This time efficiency allows users to allocate more of their focus and energy toward more critical tasks, further underscoring the program's utility.

Despite its numerous strengths, Number system converter does come with a set of limitations. Notably, its support is primarily focused on the four major number systems: Binary, Octal, Decimal, and Hexadecimal. Users requiring conversions involving less common or specialized numeral bases may find the program inadequate for their specific needs, necessitating alternative solutions. Lastly, for users seeking advanced mathematical or computational features beyond the core functionality of number system conversions, Number system converter may fall short. It primarily focuses on these conversions and may not satisfy the requirements of users with more specialized needs.

Self-review and future improvement

Upon reflection on the development of the Number System Converter program, it's evident that the tool holds significant value in simplifying number system conversions. Its strengths lie in its versatile base support, specializing in the four primary number systems: Binary, Octal, Decimal, and Hexadecimal. This specialization allows it to adeptly handle conversions between these bases, catering to a wide range of applications. The program's user-friendly and intuitive interface further enhances its appeal, making it accessible to users of varying skill levels. Notably, its commitment to delivering precise and automated conversions stands out, eliminating the risk of human error and ensuring accurate results.

In the future, Number System Converter aims to build on its current strengths and address these identified areas for improvement. Expanding number system support will be a top priority. Future versions of Number System Converter will include support for a wider range of number systems, accommodating specialized needs beyond Binary, Octal, Decimal, and Hexadecimal. These additions will ensure that users can seamlessly convert to and from these bases. Comprehensive learning resources will be created, offering a rich set of educational materials that cater to users of all skill levels. This will include video tutorials, interactive examples, and extensive documentation, empowering users to master number system conversions.

Conclusion

In summary, our journey through the development of the electronic engineering toolkit "Helix" has been truly rewarding. It's been an educational experience that not only enhanced our programming skills but also deepened our understanding of electronic engineering principles. Moreover, it emphasized the significance of effective teamwork in bringing innovative solutions to life.

"Helix" stands as a valuable asset for electronic engineers, offering a range of powerful features such as binary function analysis, truth table generation, number system conversion, and logic gate analysis. We believe it has the potential to significantly enhance the efficiency and effectiveness of engineers' work in various applications.

Above all, this assignment has underscored the importance of collaboration and innovation in solving complex engineering challenges. As we continue to refine "Helix" and explore its possibilities, we are excited about the positive impact it can have on the electronic engineering community. It is a testament to what can be achieved through teamwork, creativity, and a commitment to advancing technology in our field.

References

Admin. (2023, February 20). *Number system (definition, types, conversion & examples)*. BYJUS. https://byjus.com/maths/number-system/#:~:text=A%20number%20system%20is%20defined,algebraic%20structure%20of%20the%20figures.

BinaryHexConverter. (n.d.). *Decimal to Binary Converter.* https://www.binaryhexconverter.com/decimal-to-binary-converter

Kamayani. (n.d.). *Number system conversion in C++*. Number system conversion in C++ | kamayaniR. https://coderspacket.com/number-system-conversion

Kumar, D. (2020, June 22). *C++ program to find parity of a number efficiently*. CodeSpeedy. https://www.codespeedy.com/cpp-program-to-find-parity-of-a-number-efficiently/

Definition of 'AND in Electrical Engineering https://www.collinsdictionary.com/dictionary/english/and-gate

Electrical4U(2020, September 27) OR Gate: What is it? (Working Principle & Circuit Diagram) https://www.electrical4u.com/logical-or-gate/

Electronic4U(2020, September 27) NAND Gate: What is it? (Working Principle & Circuit Diagram) https://www.electrical4u.com/nand-gate/

Priyanshu Vaish(2022, September 30) NOR Gate Truth Table | Nor Gate Circuit Diagram https://byjusexamprep.com/nor-gate-truth-table-i

Margaret Rouse (2022, November 13) Half Adder

https://www.techopedia.com/definition/7509/half-adder

Electrical4U (2020, October 28) Binary Subtractor

https://www.electrical4u.com/binary-substractor/

Electronicsforu(2023, August 21) Flip Flop Basics – Types, Truth Table, Circuit, and Applications

https://www.electronicsforu.com/technology-trends/learn-electronics/flip-flop-rs-jk-t-d

Priyanshu Vaish(2022 August 26) SR Flip-Flop: What is SR Flip-Flop Truth Table?

https://byjusexamprep.com/sr-flip-flop-truth-table-i

Nidhi Agarwal (2023, March 23) Introduction to JK Flip-Flops and Its Working

https://www.electronicsforu.com/technology-trends/learn-electronics/jk-flip-flop-circuit-truth-table-limitations-applications